



NATIONAL ECONOMICS UNIVERSITY

NEU COLLEGE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY

CHAPTER IV

WEB DEVELOPMENT WITH PYTHON

PHAM THAO (PHD)

SENIOR LECTURE

 0966.986.689



THAOP@NEU.EDU.VN



CHAPTER IV PYTHON IN WEB DEVELOPMENT WITH FLASK

1. Flask Development Server:

1. Setting up the Flask Development Server.

2. Setting Up Flask:

1. Creating a Python environment folder.
2. Establishing the Flask Templates folder.

3. Routes:

1. Defining and configuring routes.

4. Working with Jinja2:

1. Leveraging the Jinja2 templating engine for rendering dynamic content.

5. Using Blueprints:

1. Implementing Blueprints for modularizing and organizing your Flask application.

1. Working with Sessions and State Management:

1. Managing user sessions and application state effectively.

• Working with Databases:

- Managing database interactions.

• Development of Key Features:

- Implementing essential features such as:
 - User Login
 - Displaying Products/Goods/Items
 - CRUD (Create, Read, Update, Delete) Operations
 - Admin Dashboard Management
 - ...



OUTLINE OF THIS LESSON

1. PYTHON IN WEB DEVELOPMENT

2. REVIEW

1. HTML
2. CSS
3. Javascript

1. Flask Development Server:

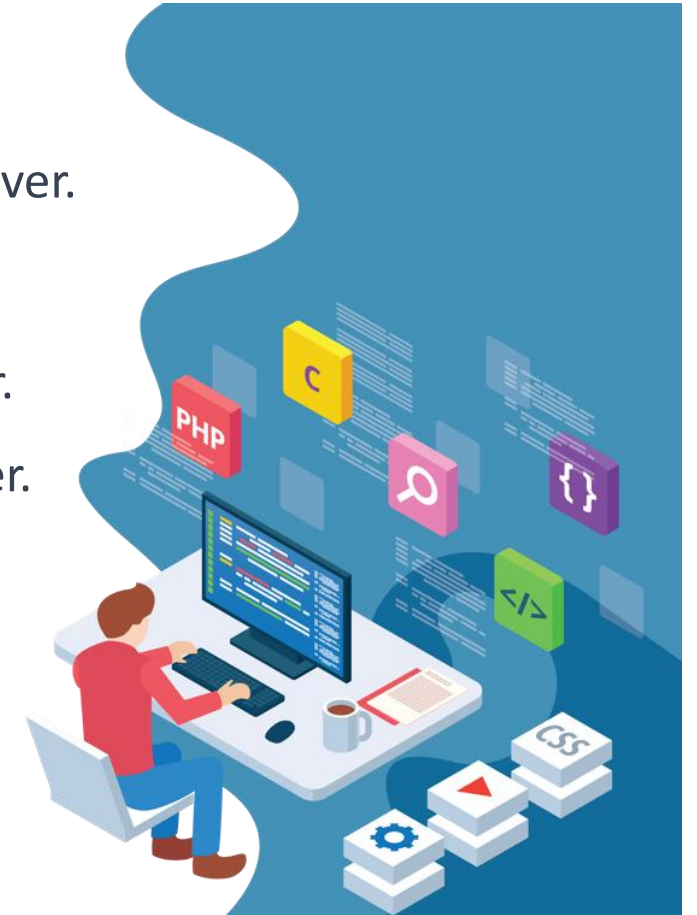
1. Setting up the Flask Development Server.

2. Setting Up Flask:

1. Creating a Python environment folder.
2. Establishing the Flask Templates folder.

3. Routes:

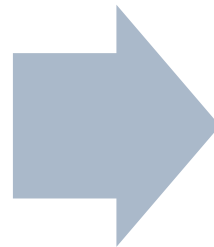
1. Defining and configuring routes.



PYTHON IN WEB DEVELOPMENT

Staple technologies of web development: There's HTML, CSS, and JavaScript

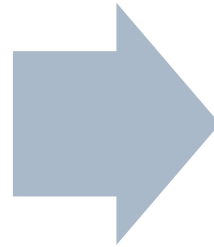
- HTML is the building block of a website. What does that mean? Well, everything you see online was created by HTML. The images, the text, the buttons, everything came from HTML.
- CSS Now CSS styled all of it. It's called "Cascading Style Sheet", and the elements (building blocks) that you create with HTML can be designed (colored, aligned, etc.) using CSS
- JavaScript JavaScript makes everything dynamic. When you click a button on a site, something happens right? Maybe another site opens up, or maybe you just get a popup that gives you some information. JavaScript lets you do things like that on your site.



What role does
Python
play in web development?

PYTHON IN WEB DEVELOPMENT

Front-end web development is what we talked about just now: HTML, CSS, JavaScript, and all together we get the user-facing end of your website; that is, what the user sees.



Back-end web development is just the opposite.

- It's what the user does not see: server-side development. Most applications need a lot of information transfer and retrieval, am I right?
- You have an account on a site, and when you log in to that site, your account detail should be retrieved.
- You search for something on Google, and they give you a list of related websites that pertain to your search.
- ...

AN EXAMPLE (HTML)

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
  </body>
</html>
```



```
<!DOCTYPE html>
<html>
  <head>
    <title>My first website</title>
  </head>
  <body>
  </body>
</html>
```



```
<body>
  <h1>My introduction</h1>
  <p>Hello there! I'm Susan. I'm 8 years old. I have a
  puppy named Barky. I love him so much! :) </p>
  <button>Click Me!</button>
  <a href='google.com'>Look me up!</a>
  <img src='susan.png'>
</body>
```

My introduction

Hello there! I'm Susan. I'm 8 years old. I have a puppy named Barky. I love him so much! :)

AN EXAMPLE (CSS-I)

```
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
  </body>
</html>
```



```
<!DOCTYPE html>
<html>
  <head>
    <title>My first website</title>
  </head>
  <body>
  </body>
</html>
```



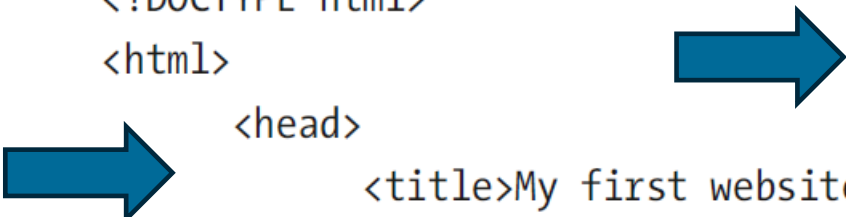
```
<head>
  <title>My first website</title>
  <style>
    html {
      background-color: lightgray;
    }
    h1 {
      color: darkgreen;
    }
    p {
      color: darkred;
    }
  </style>
</head>
```

My introduction

Hello there! I'm Susan. I'm 8 years old. I have a puppy named Barky. I love him so much! :)

AN EXAMPLE (CSS-2)

```
<!DOCTYPE html>      <!DOCTYPE html>
<html>                <html>
  <head>              <head>
    </head>           <title>My first website</title>
  <body>              </head>
    </body>           <body>
  </html>             </body>
                    </html>
```



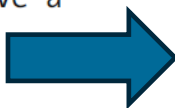
```
<!DOCTYPE html>
<link rel="stylesheet" href="style.css" >
<html lang="en">
<head>
```

"style.css":

```
body {
  background-color: powderblue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

AN EXAMPLE (JAVA SCRIPT)

```
<body>
  <h1>My introduction</h1>
  <p>Hello there! I'm Susan. I'm 8 years old. I have a
  puppy named Barky. I love him so much! :) </p>
  <button>Click Me!</button>
  <a href='google.com'>Look me up!</a>
  <img src='susan.png'>
  <script>
  </script>
</body>
```



```
<button id='btn'>Click Me!</button>
```

You can create variables using the keyword “let”

```
let variableName;
```

```
<script>
  let button = document.getElementById('btn');
  function buttonClick() {
    alert("Hello there! I'm Susan!");
  }
  button.addEventListener('click',buttonClick);
</script>
```

:/BMCNTT/Python%20LTWeb/Web%2030%20HTML%20CSS%20JS/Main.html

ion

8 years old. I hav

Click Me!

This page says

Hello there! I'm Susan!

OK

MAIN.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My first website</title>
  <style>
    html {
      background-color: lightgray;
    }
    h1 {
      color: darkgreen;
    }
    p {
      color: darkred;
    }
  </style>
</head>
```

```
<body>
  <h1>My introduction</h1>
  <p>Hello there! I'm Susan. I'm 8 years old. I have a
  puppy named Barky. I love him so much! :) </p>

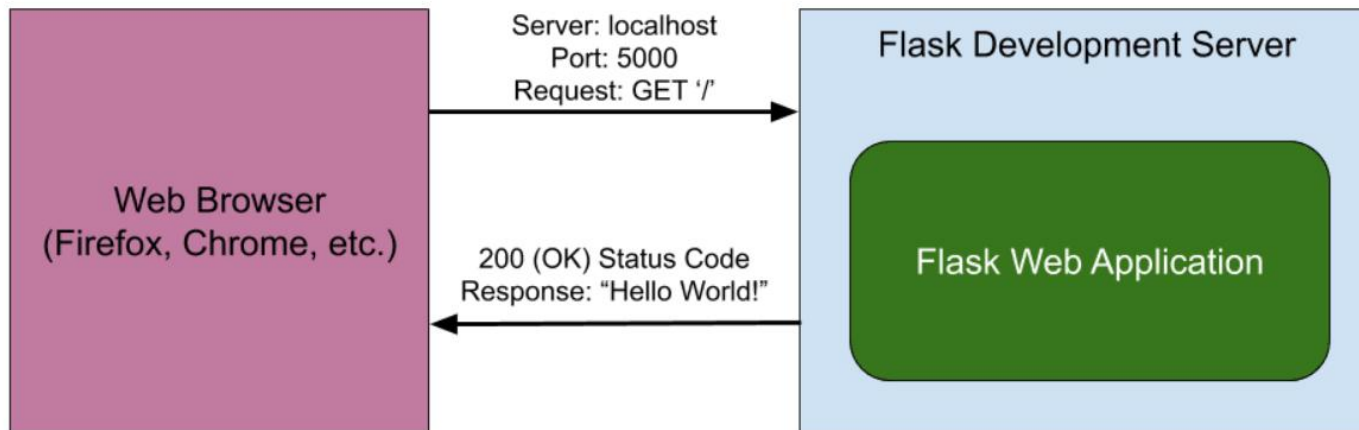
  <button>Click Me!</button>
  <a href='google.com'>Look me up!</a>
  <img src='susan.png'>

  <button id='btn'>Click Me!</button>

  <script>
    let button = document.getElementById('btn');
    function buttonClick() {
      alert("Hello there! I'm Susan!");
    }
    button.addEventListener('click',buttonClick);
  </script>
</body>
</html>
```

FLASK DEVELOPMENT SERVER

Run the Flask Development Server to serve your Flask app

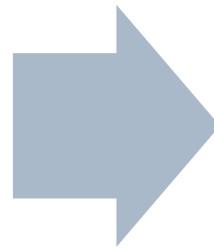


The Flask is a framework that uses Python language with easy-to-understand code writing.

1. The browser made a request to the Flask development server (running as '127.0.0.1:5000')
2. The server received the request and responded with a status of 200 (OK) along with the HTML to display "Hello World!" in the web browser:

PYTHON IN WEB DEVELOPMENT

Front-end web development is what we talked about just now: HTML, CSS, JavaScript, and all together we get the user-facing end of your website; that is, what the user sees.

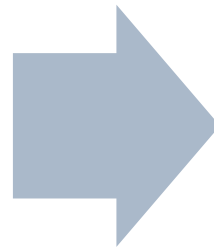


Back-end web development is just the opposite.

- It's what the user does not see: server-side development. Most applications need a lot of information transfer and retrieval, am I right?
- You have an account on a site, and when you log in to that site, your account detail should be retrieved.
- You search for something on Google, and they give you a list of related websites that pertain to your search.
- ...

PYTHON IN WEB DEVELOPMENT

Flask is a lightweight WSGI web application framework. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.



It began as a simple wrapper around Werkzeug and Jinja and has become one of the most popular Python web application frameworks.

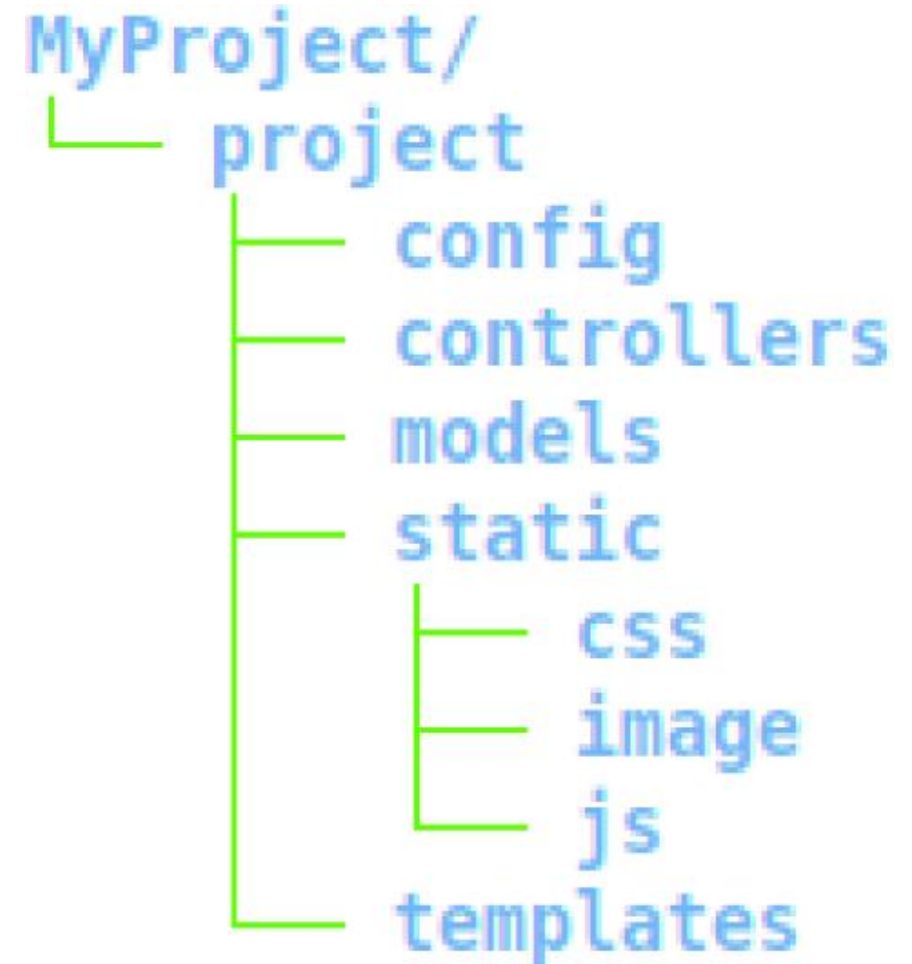
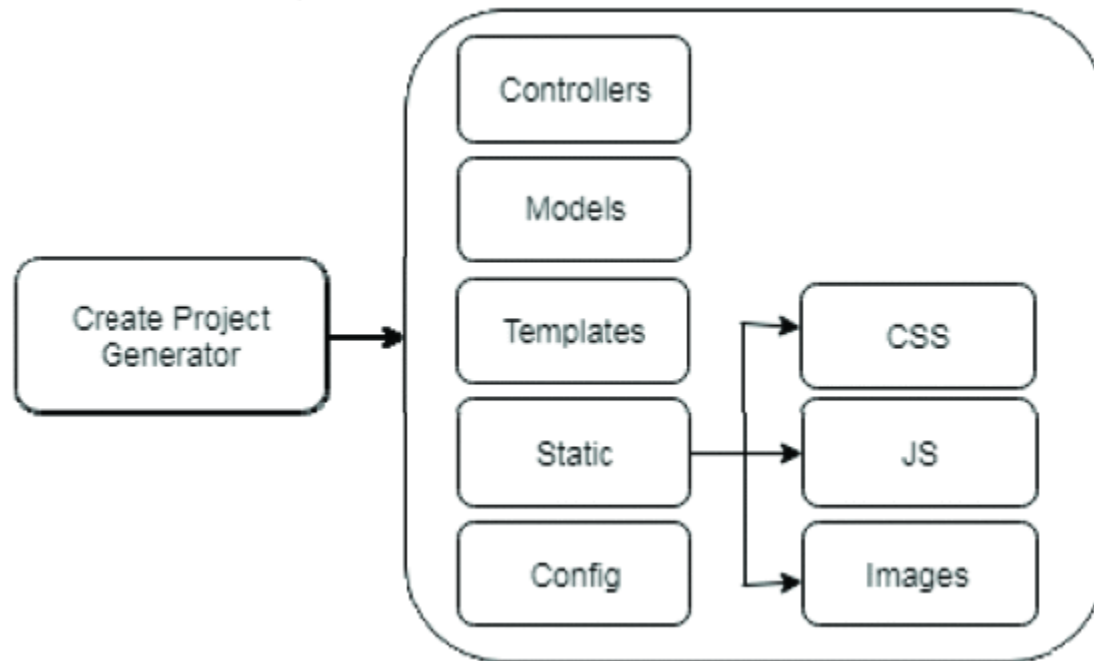
FLASK DEVELOPMENT SERVER

Flask is a Python web framework that offers a set of tools and pre-written code to facilitate website development, without the need to do everything from scratch.

We chose Flask because Flask is a micro-framework that does not contain many tools and libraries, so it is more portable and does not utilize a lot of resources (Anggoro and Aziz, 2021). With its streamlined features, Flask is lightweight and doesn't rely heavily on numerous external libraries (Mufid et al., 2021).

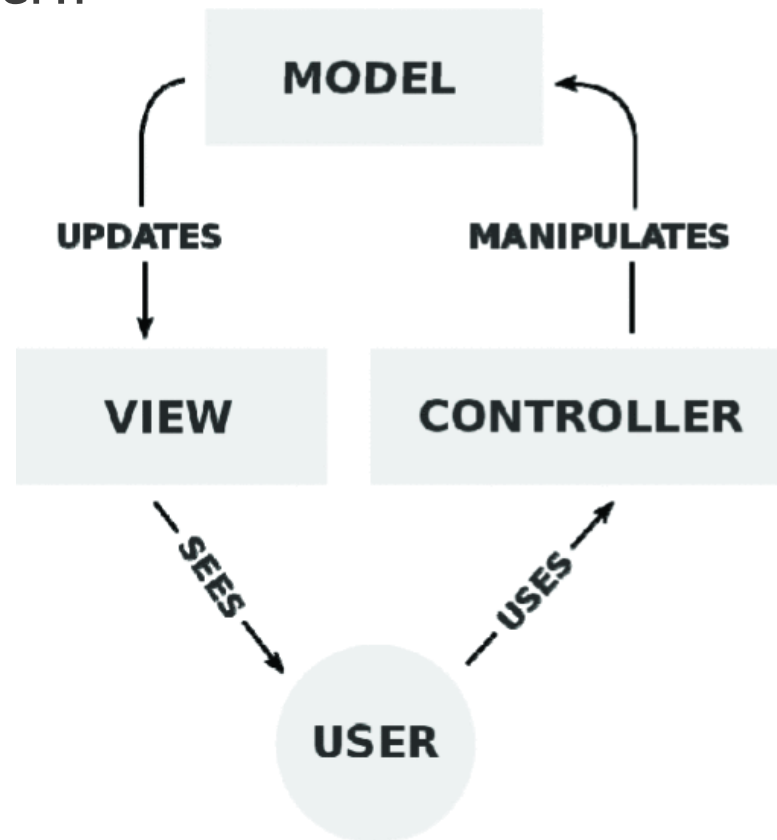
PYTHON IN WEB DEVELOPMENT

■ System Process Diagram



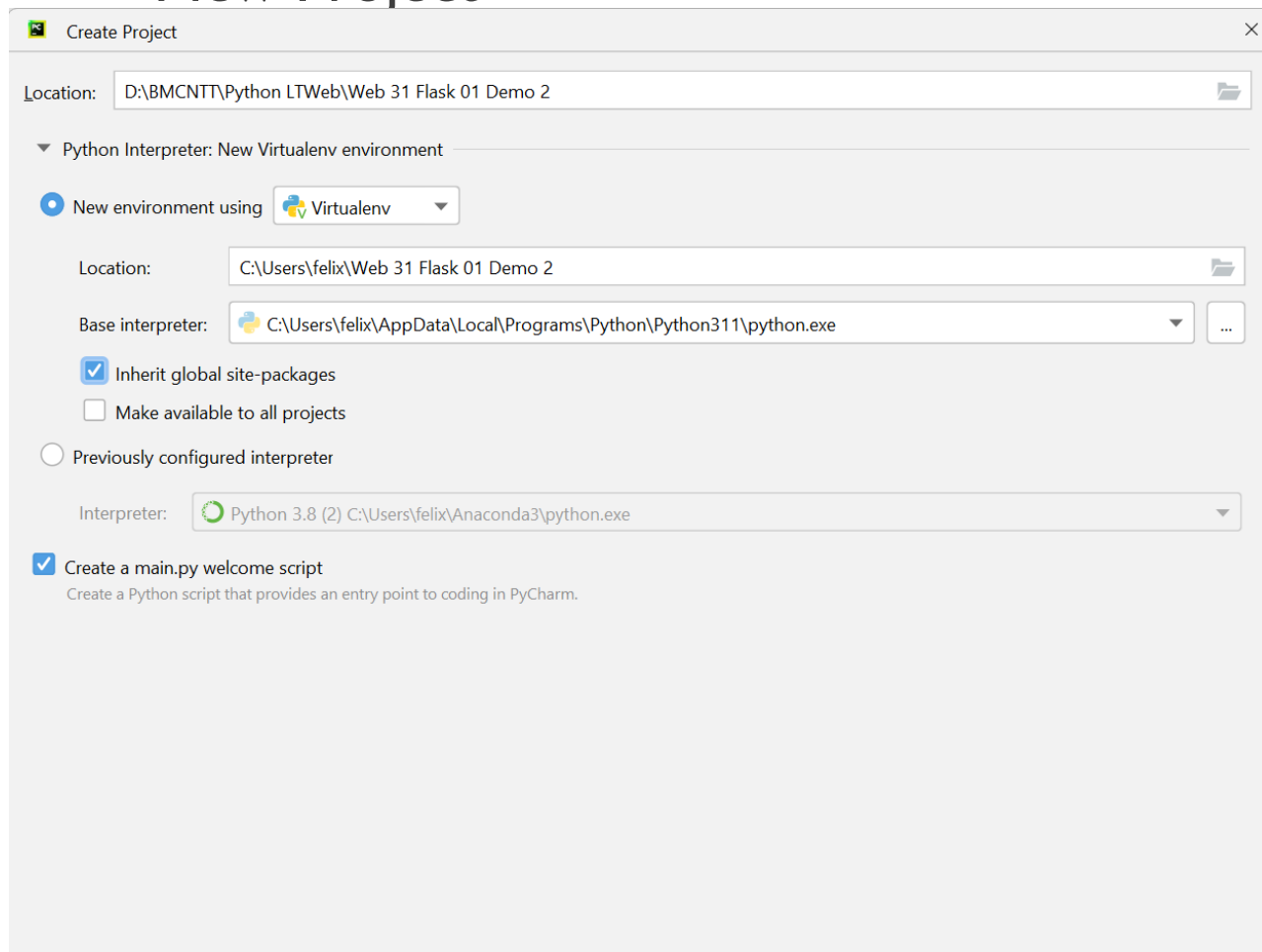
PYTHON IN WEB DEVELOPMENT

- The MVC pattern



CREATE NEW WEBSITE WITH FLASK

■ New Project

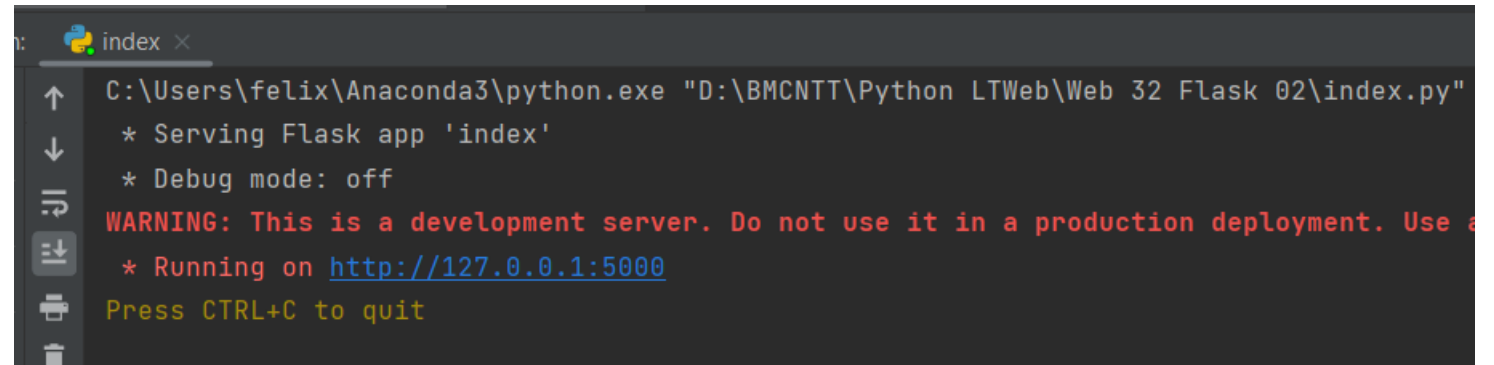


CREATE NEW WEBSITE WITH FLASK

■ New Project

```
#import các thư viện
from flask import Flask, render_template
#Khởi tạo
app = Flask(__name__)
#Chạy ứng dụng
if __name__ == "__main__":
    app.run();
```

In this Python code, we import Flask and create a simple Flask web application.



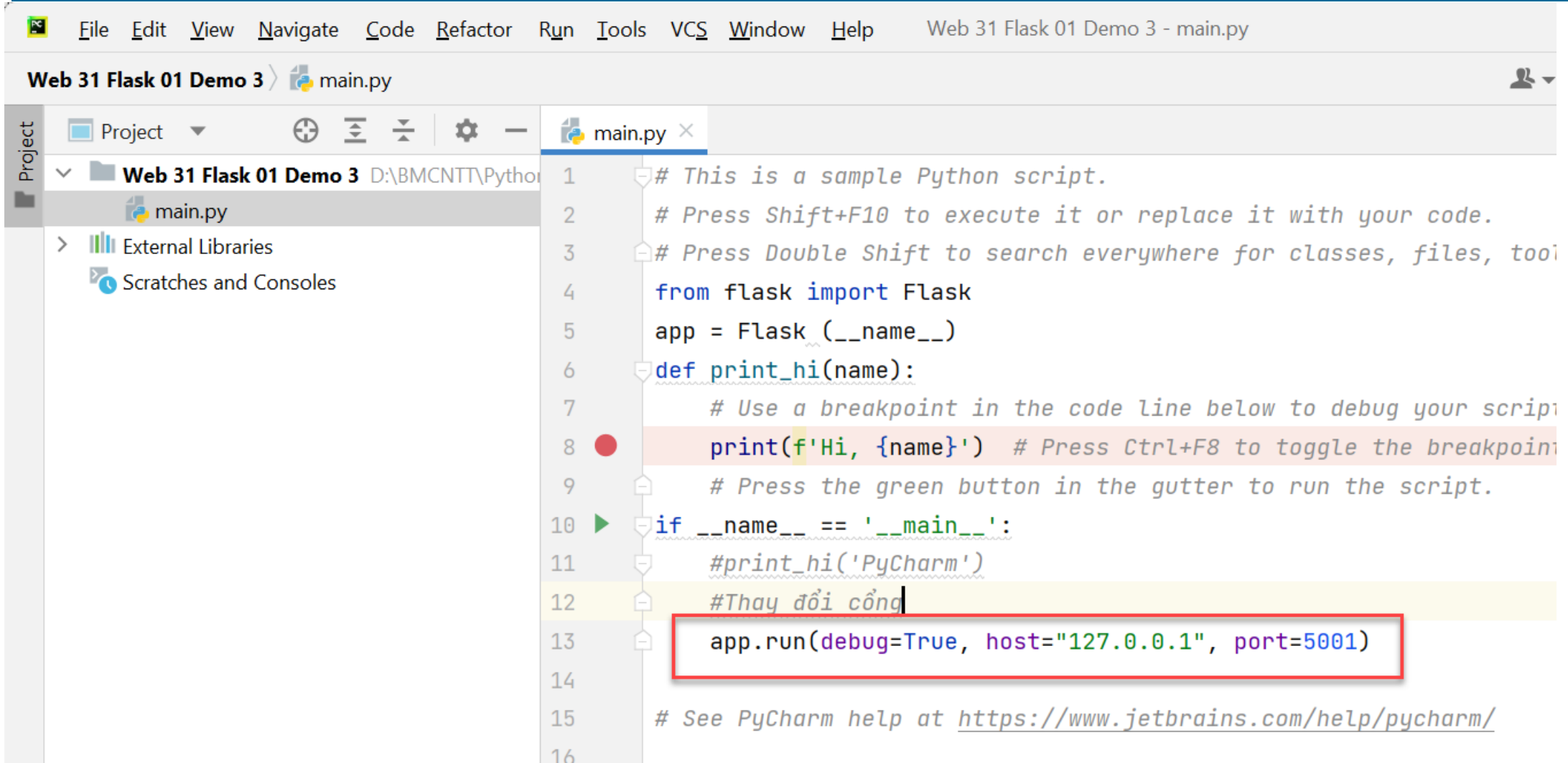
```
index x
C:\Users\felix\Anaconda3\python.exe "D:\BMCNTT\Python LTWeb\Web 32 Flask 02\index.py"
* Serving Flask app 'index'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

← → ↻ ⓘ 127.0.0.1:5000

Not Found

The requested URL was not found on the server. If you entered the URL manually

CREATE NEW WEBSITE WITH FLASK

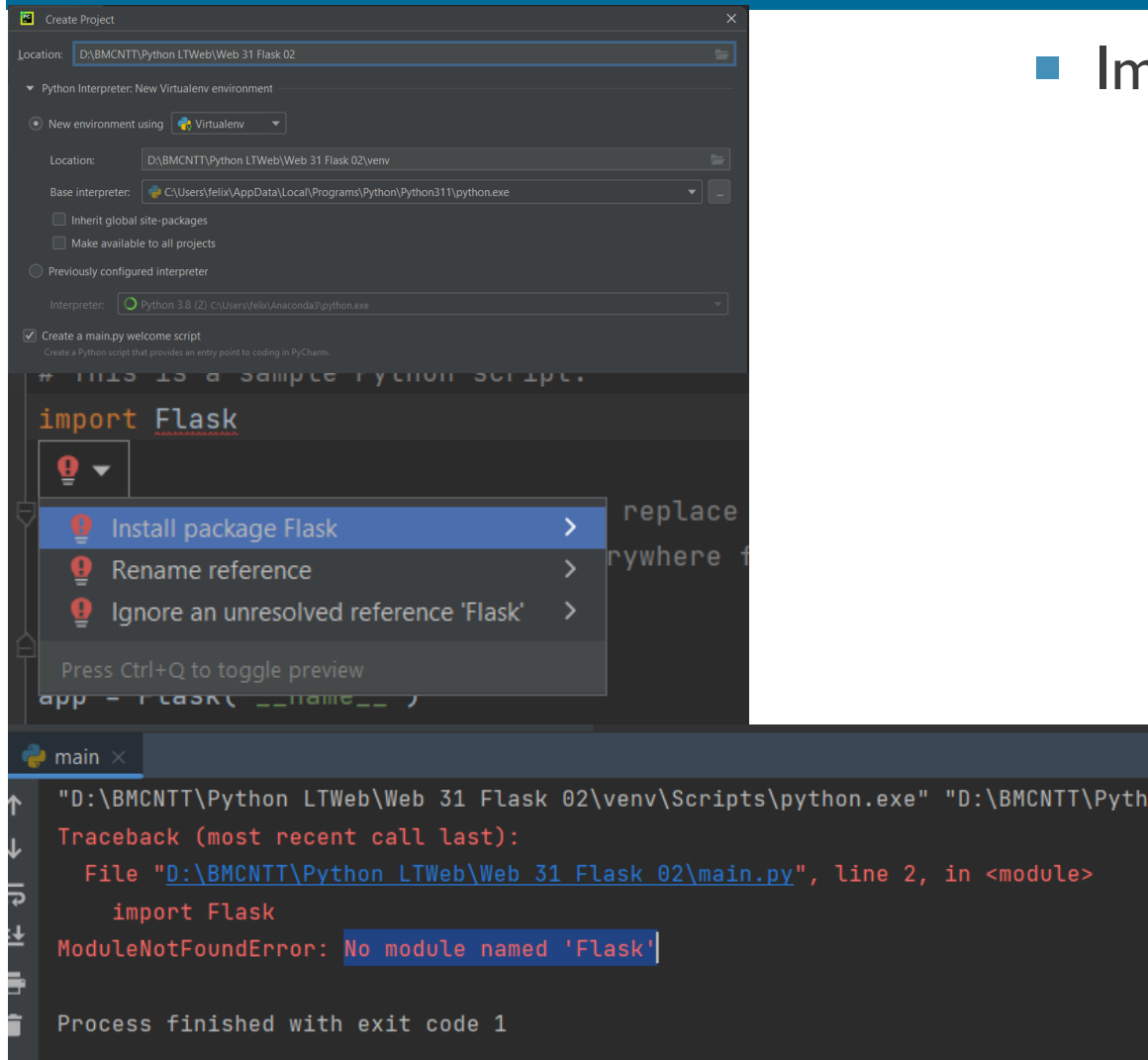


The image shows the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Run, Tools, VCS, Window, and Help. The title bar reads "Web 31 Flask 01 Demo 3 - main.py". The breadcrumb navigation shows "Web 31 Flask 01 Demo 3" > "main.py". The left sidebar contains a "Project" view with a tree structure: "Web 31 Flask 01 Demo 3" (expanded) containing "main.py", "External Libraries", and "Scratches and Consoles". The main editor window displays the code for "main.py". The code includes comments in Vietnamese and English, imports Flask, creates an app, defines a print_hi function, and has a main block that calls app.run with debug=True, host="127.0.0.1", and port=5001. A red circle is in the gutter next to line 8, and a green play button is next to line 10. The line `app.run(debug=True, host="127.0.0.1", port=5001)` is highlighted with a red box.

```
1 # This is a sample Python script.
2 # Press Shift+F10 to execute it or replace it with your code.
3 # Press Double Shift to search everywhere for classes, files, tool
4 from flask import Flask
5 app = Flask(__name__)
6 def print_hi(name):
7     # Use a breakpoint in the code line below to debug your script
8     print(f'Hi, {name}') # Press Ctrl+F8 to toggle the breakpoint
9     # Press the green button in the gutter to run the script.
10 if __name__ == '__main__':
11     #print_hi('PyCharm')
12     #Thay đổi cổng
13     app.run(debug=True, host="127.0.0.1", port=5001)
14
15 # See PyCharm help at https://www.jetbrains.com/help/pycharm/
16
```

CREATE NEW WEBSITE WITH FLASK

SET UP FLASK IF NOT EXISTS



- Import and setup Flask by using the suggestion

```
# This is a sample Python script.  
from flask import Flask  
  
#Khởi tạo biến app  
app = Flask(__name__)  
  
@app.route("/")  
def Introduction():  
    return "Hello!"  
  
# Press the green button in the gutter to run the script.  
if __name__ == '__main__':  
    #print_hi('PyCharm')  
    app.run(debug=True)  
# See PyCharm help at https://www.jetbrains.com/help/pycharm/
```

Terminal

- python.exe -m pip install --upgrade pi
- pip install Flask

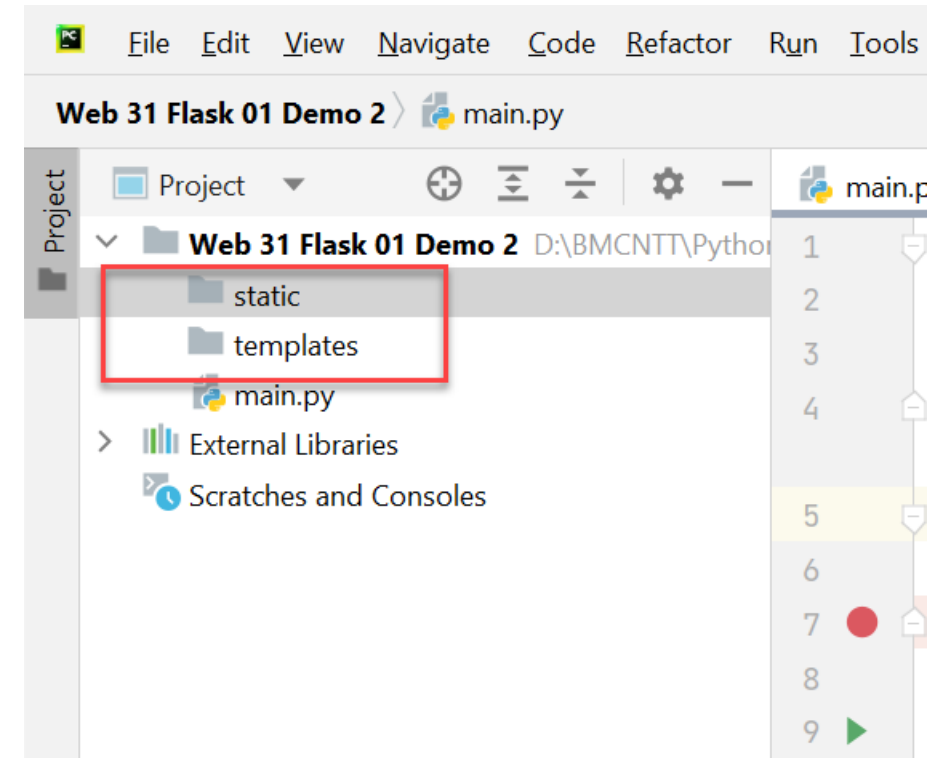
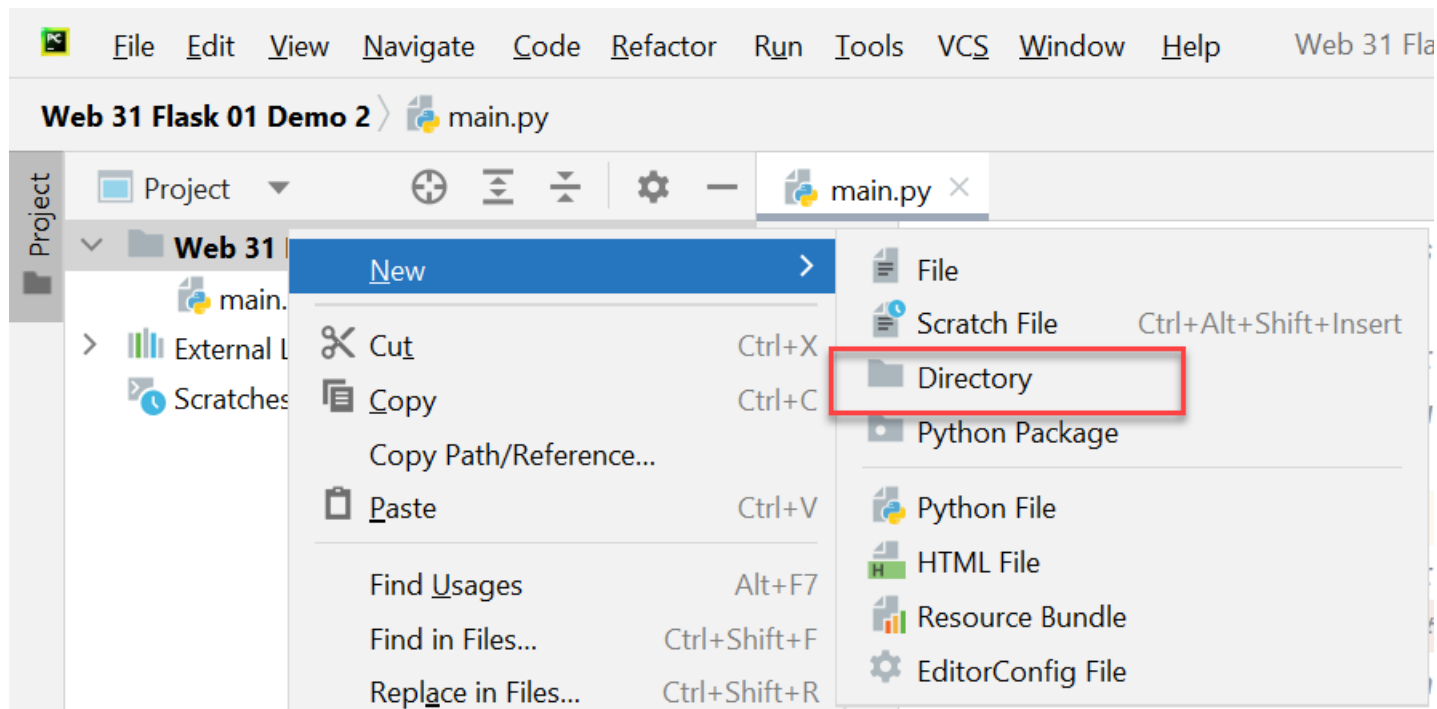
INTRODUCTION JINJA2 AND WEB TEMPLATE

- Jinja2 is a popular templating engine for Python that allows you to embed dynamic content in templates, such as HTML files.
 - It's commonly used in web development with frameworks like Flask and Django.
 - Jinja2 uses a syntax similar to Python to include dynamic elements in templates.
 - Link: <https://jinja.palletsprojects.com/en/3.1.x/templates/>
- flask file structure
- Generate an index.html file within the 'templates' folder.
 - Static folder stores css, image file

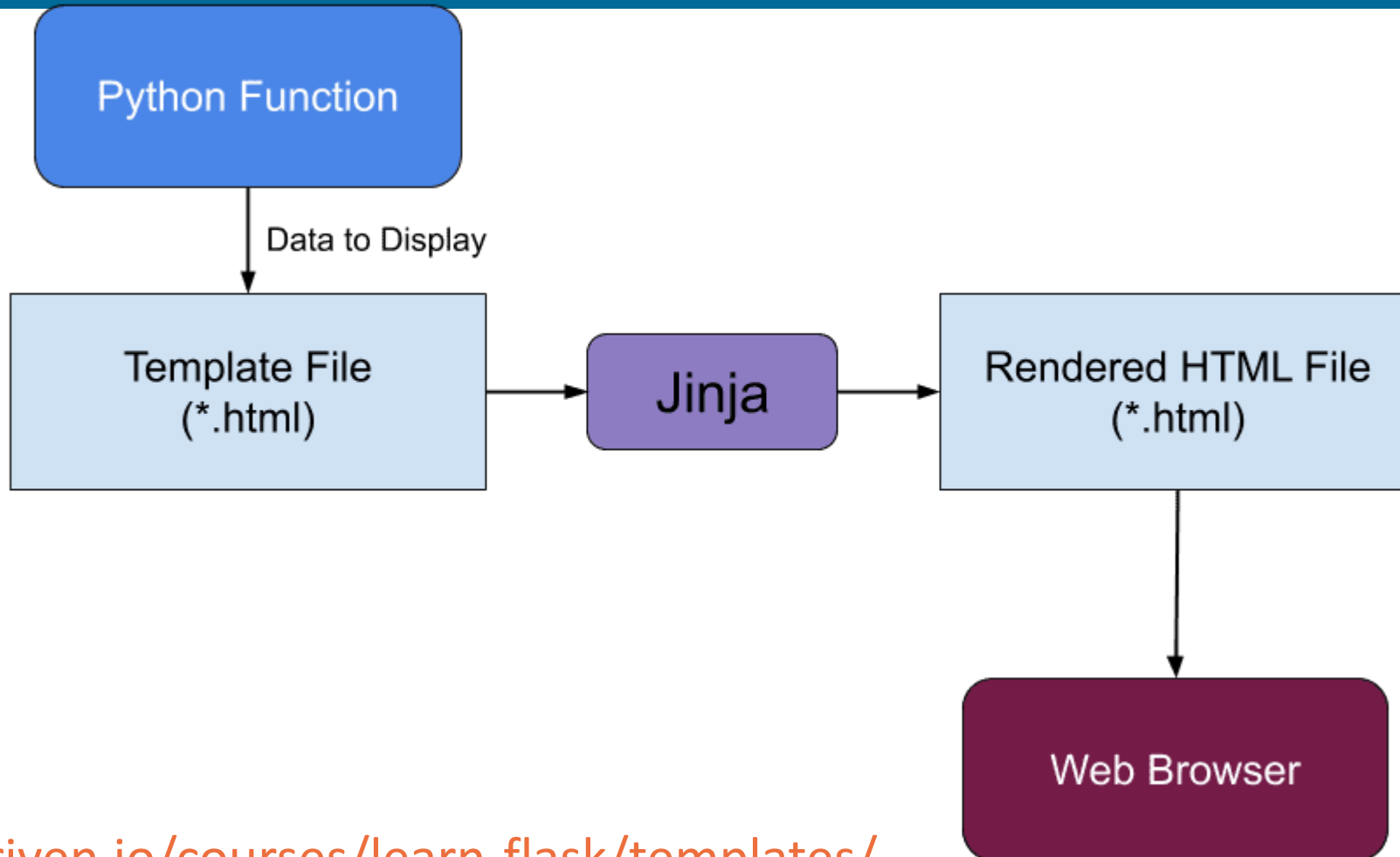
```
| -app  
|--templates // where your html files must be in  
|--static // where your js and css files must be in  
|--.py files  
|--other packages
```

CREATE NEW WEBSITE WITH FLASK

■ New Project



INTRODUCTION JINJA2 AND WEB TEMPLATE



INTRODUCTION JINJA2 AND WEB TEMPLATE

← → ↻ jinja.palletsprojects.com/en/3.1.x/templates/

🔍 📄 ☆ ⚙️ ⬇️



Contents

[Template Designer Documentation](#)

[Synopsis](#)

[Template File Extension](#)

[Variables](#)

[Filters](#)

[Tests](#)

[Comments](#)

[Whitespace Control](#)

[Escaping](#)

[Line Statements](#)

[Template Inheritance](#)

[Base Template](#)

Template Designer Documentation

This document describes the syntax and semantics of the template engine and will be most useful as reference to those creating Jinja templates. As the template engine is very flexible, the configuration from the application can be slightly different from the code presented here in terms of delimiters and behavior of undefined values.

Synopsis

A Jinja template is simply a text file. Jinja can generate any text-based format (HTML, XML, CSV, LaTeX, etc.). A Jinja template doesn't need to have a specific extension: `.html`, `.xml`, or any other extension is just fine.

A template contains **variables** and/or **expressions**, which get replaced with values when a template is *rendered*, and **tags**, which control the logic of the template. The template syntax is heavily inspired by Django and Python.

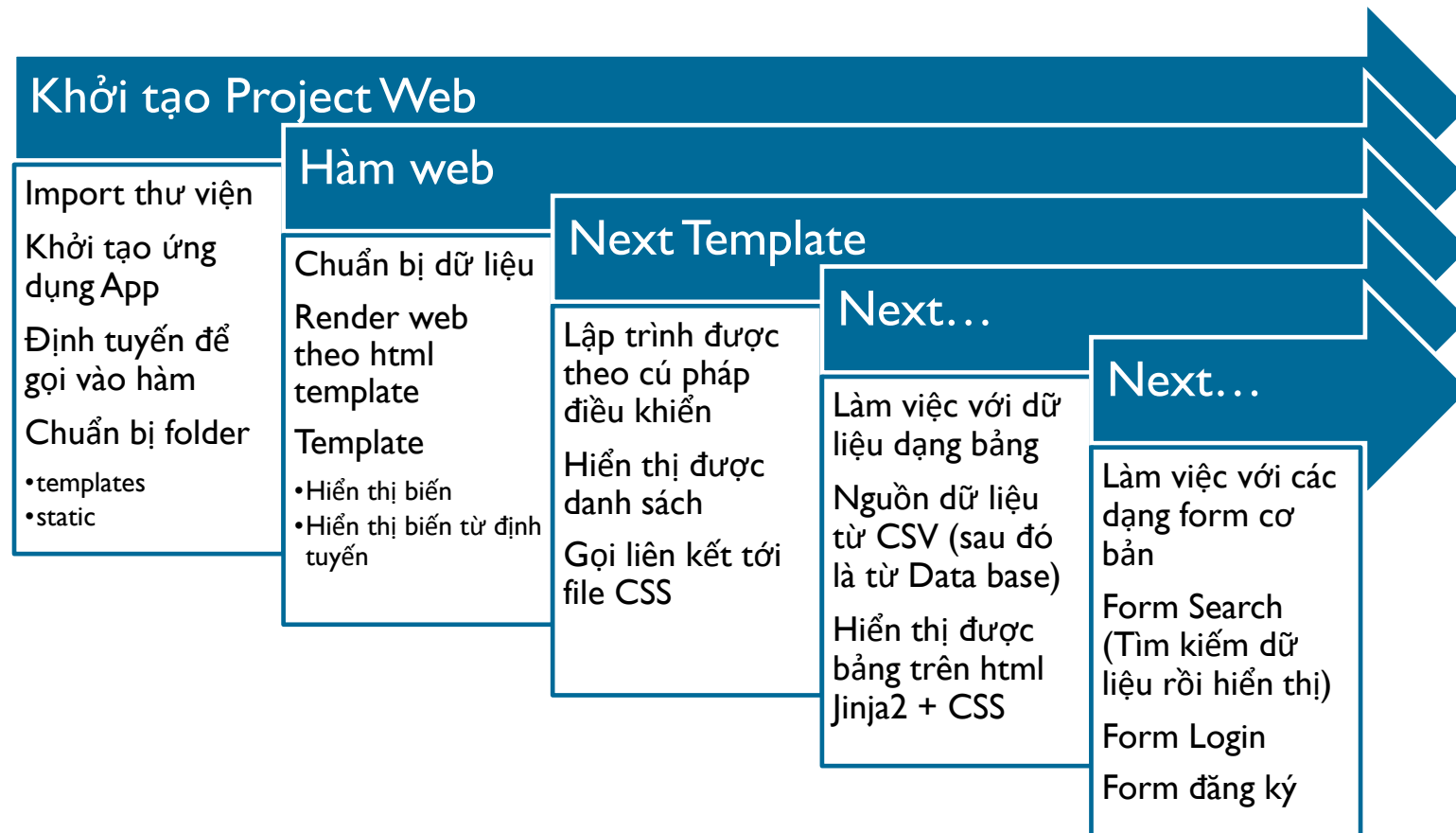
Below is a minimal template that illustrates a few basics using the default Jinja configuration. We will cover the details later in this document:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

INTRODUCTION JINJA2 AND WEB TEMPLATE

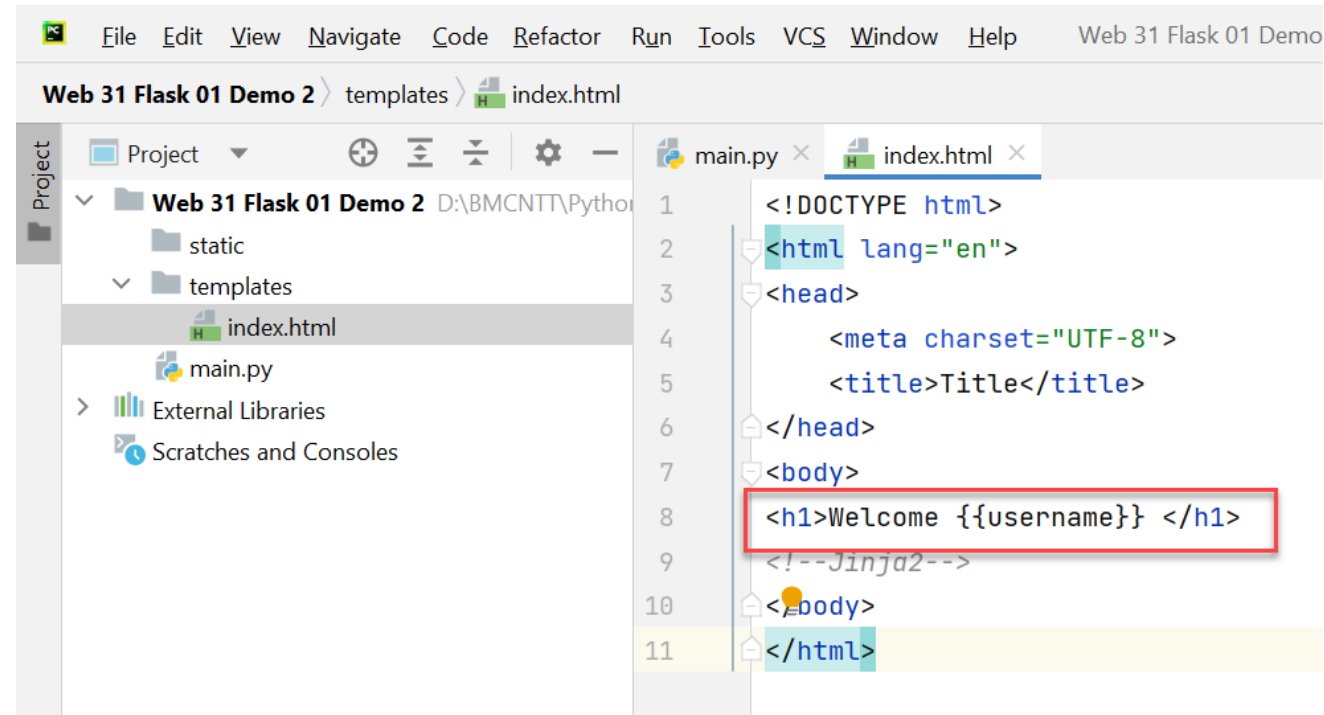
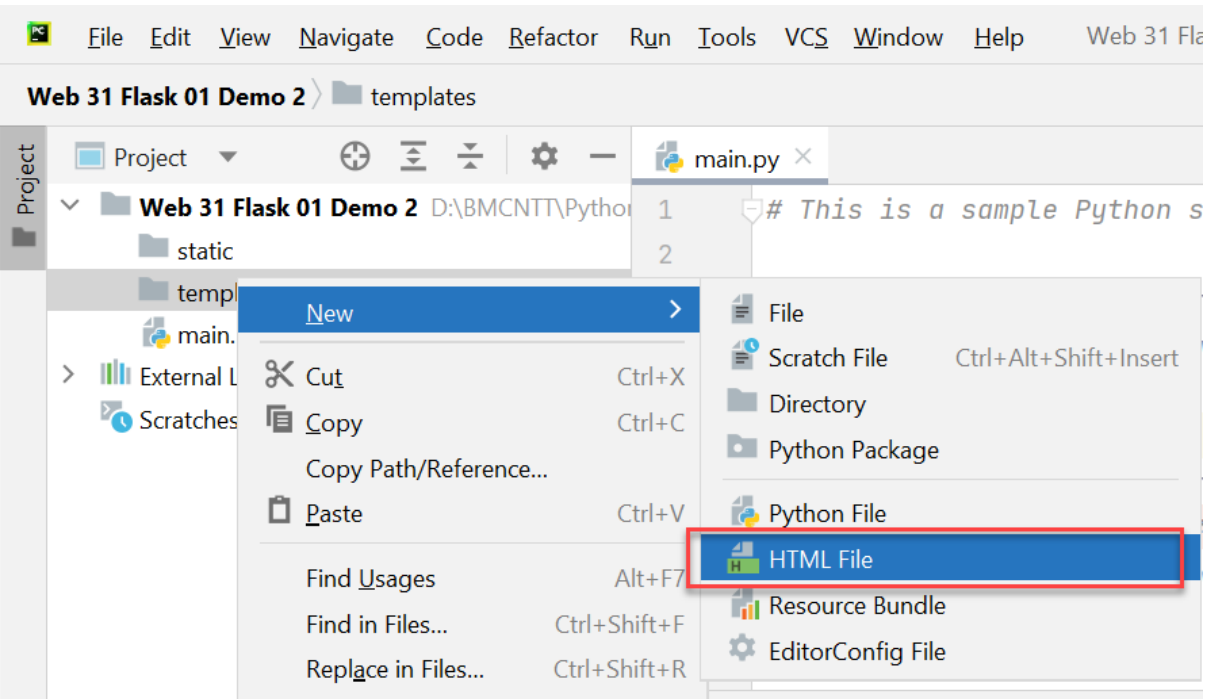
- Jinja is a fast, expressive, extensible templating engine. Special placeholders in the template allow writing code similar to Python syntax. Then the template is passed data to render the final document.
- Jinja2 allows you to do much more, such as loops, conditionals, and template inheritance, making it a powerful tool for generating dynamic content in web applications.

CREATE NEW WEBSITE WITH FLASK



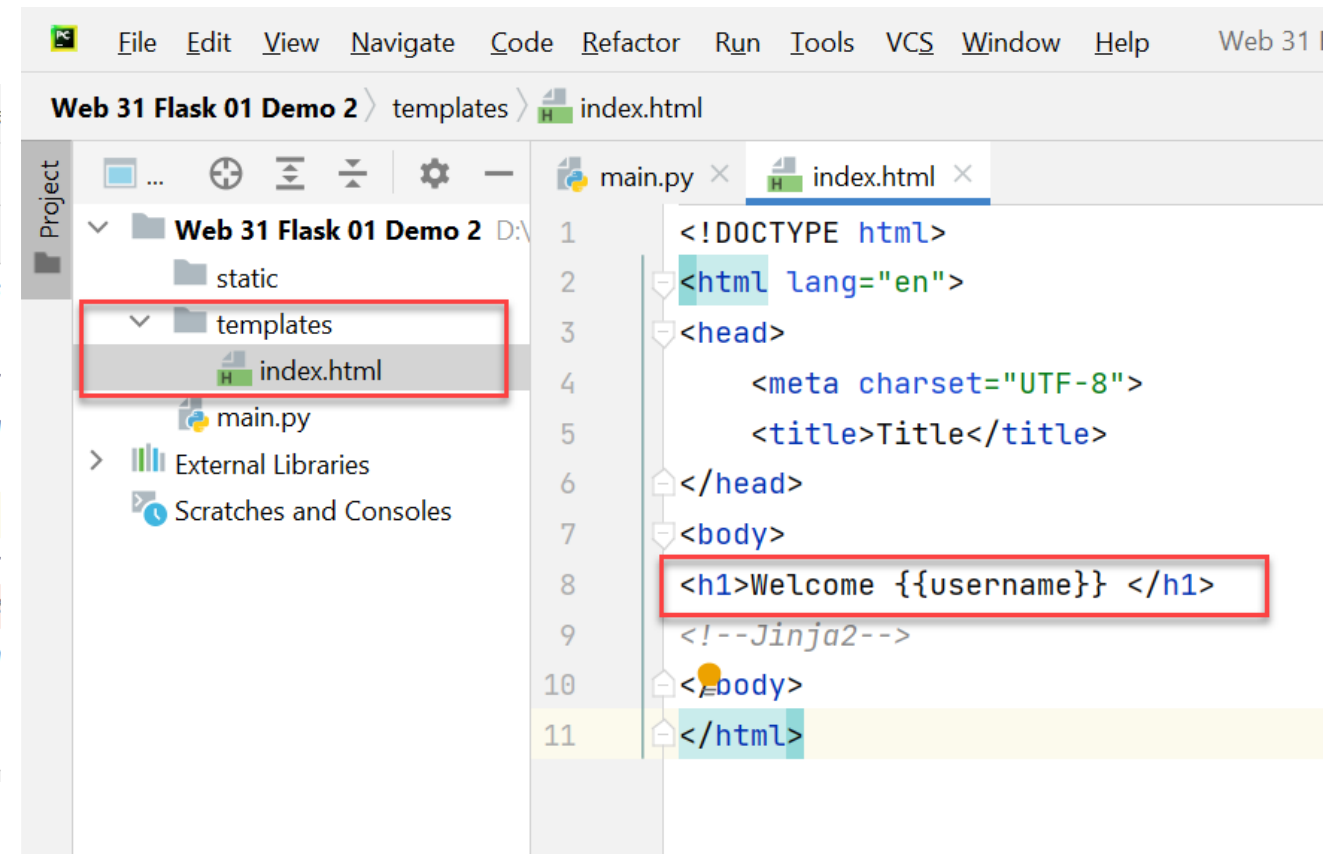
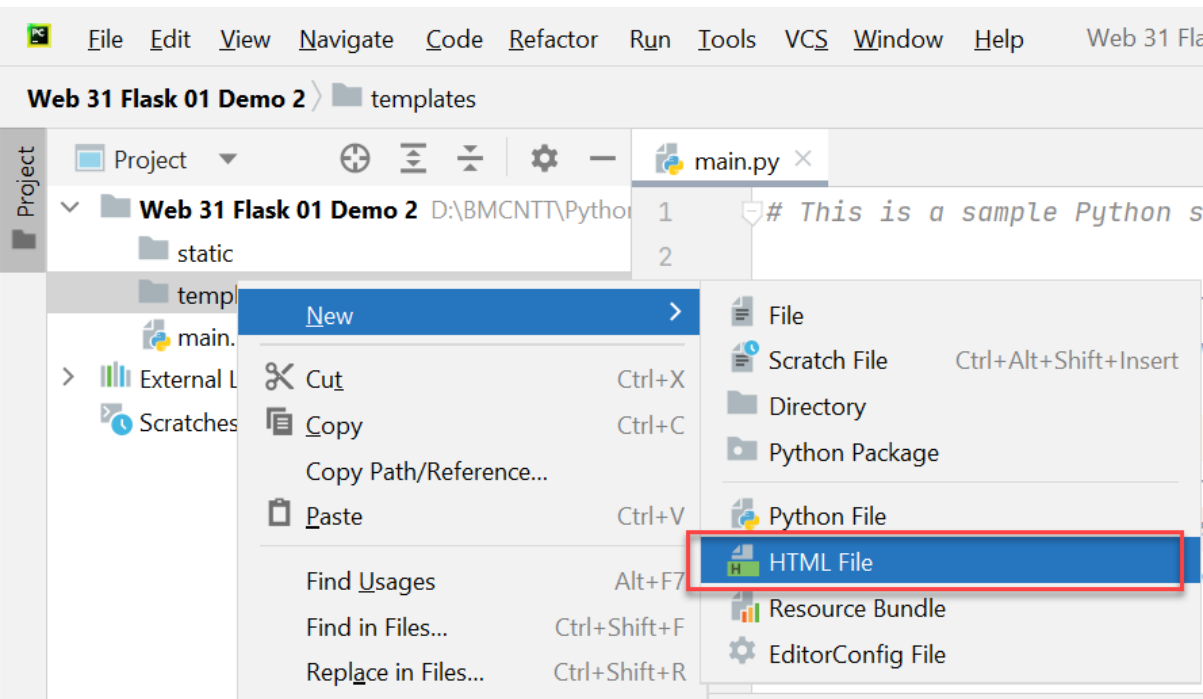
CREATE NEW WEBSITE WITH FLASK

■ New Project



CREATE NEW WEBSITE WITH FLASK

■ New Project



CREATE NEW PROJECT

The screenshot displays the PyCharm IDE interface for a new Flask project named "Web 31 Flask 01 Demo 2".

Project View (Left): Shows the project structure with folders "static" and "templates", and files "index.html" and "main.py". The "main.py" file is highlighted.

Code Editor (Main): Displays the content of "main.py". The code is as follows:

```
1 # This is a sample Python script.
2 # Press Shift+F10 to execute it or rep
3 # Press Double Shift to search everywh
4 from flask import Flask
5 # Khởi tạo biến app
6 app = Flask(__name__)
7 def print_hi(name):
8     # Use a breakpoint in the code lin
9     print(f'Hi, {name}') # Press Ctrl
10    # Press the green button in the gutter
11 if __name__ == '__main__':
12     #print_hi('PyCharm')
13     #Chạy ứng dụng web
14 app.run()
```

The code includes comments in Vietnamese: "# Khởi tạo biến app" (Initialize app variable), "# Chạy ứng dụng web" (Run web application), and "# Press the green button in the gutter".

Run View (Bottom): Shows the command to run the application:

```
"C:\Users\felix\Web 31 Flask 01 Demo 2\Scripts\python.exe" "D:\BMCNTT\Pyt
```

The output indicates that the Flask app is running on <http://127.0.0.1:5000>. A warning message states: "WARNING: This is a development server. Do not use it in a production depl".

CREATE NEW PROJECT

The image shows a screenshot of a code editor (likely PyCharm) with a project named "Web 31 Flask 01 Demo 2". The editor displays the file `main.py` with the following Python code:

```
1 # This is a sample Python script.
2 # Press Shift+F10 to execute it or replace it with your code
3 # Press Double Shift to search everywhere for classes, files, and code snippets
4 from flask import Flask, render_template
5 # Khởi tạo biến app
6 app = Flask(__name__)
7
8 @app.route("/")
9 def Index():
10     # Khai báo mặc định một giá trị username
11     name = "Felix"
12     return render_template(template_name_or_list="index.html",
13                            username=name)
```

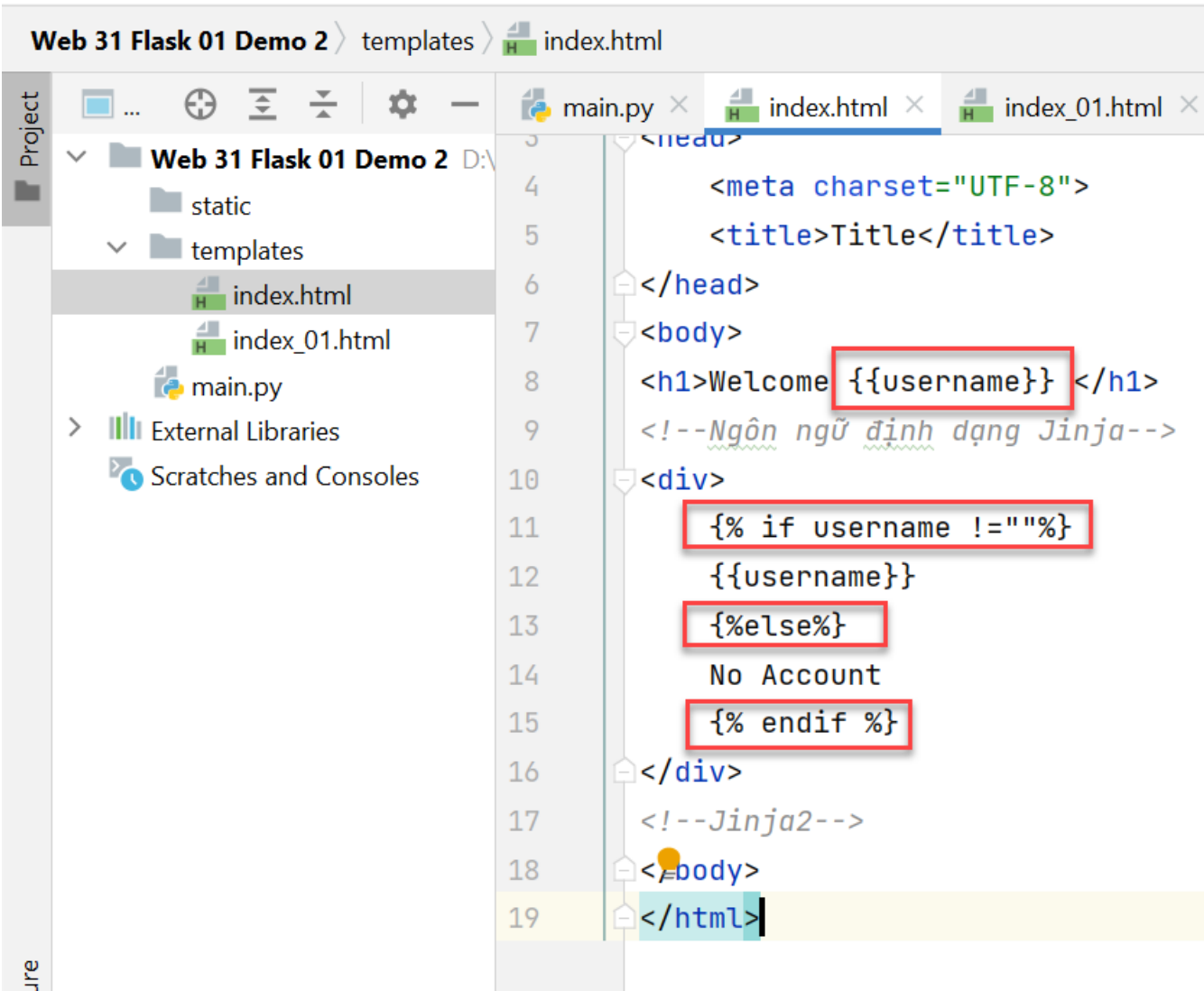
The code is annotated with red boxes highlighting the following elements:

- `render_template` in the `from flask import` statement.
- `@app.route("/")`.
- `def Index():`.
- `name = "Felix"`.
- The `render_template` function call and its arguments: `render_template(template_name_or_list="index.html", username=name)`.

Below the code editor, a web browser window is visible, showing the output of the application. The browser's address bar displays `127.0.0.1:5000`, and the page content shows "Welcome Felix".

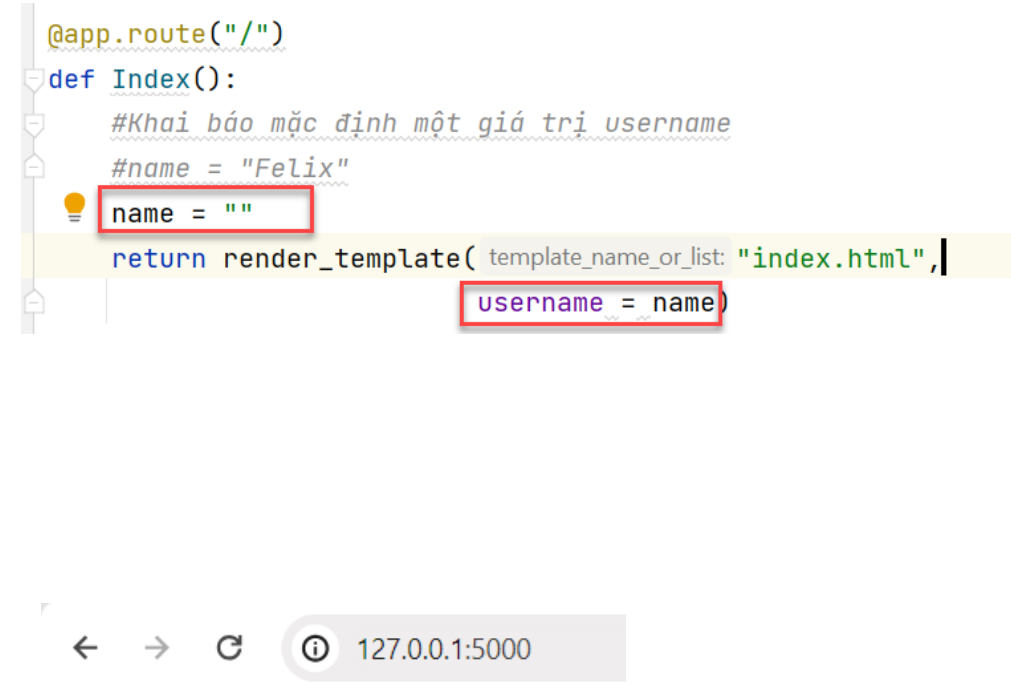
Welcome Felix

CREATE NEW PROJECT



The screenshot shows a code editor with a project named "Web 31 Flask 01 Demo 2". The file explorer on the left shows the project structure: "static" and "templates" folders. The "templates" folder is expanded, showing "index.html" and "index_01.html". The "index.html" file is open in the editor. The code is as follows:

```
1 <!--Ngôn ngữ định dạng Jinja-->
2 <!--Jinja2-->
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6 </head>
7 <body>
8   <h1>Welcome {{username}} </h1>
9   <!--Ngôn ngữ định dạng Jinja-->
10  <div>
11    {% if username != "" %}
12      {{username}}
13    {% else %}
14      No Account
15    {% endif %}
16  </div>
17  <!--Jinja2-->
18  </body>
19 </html>
```



The screenshot shows a code editor with a project named "Web 31 Flask 01 Demo 2". The file explorer on the left shows the project structure: "static" and "templates" folders. The "main.py" file is open in the editor. The code is as follows:

```
@app.route("/")
def Index():
    #Khai báo mặc định một giá trị username
    #name = "Felix"
    name = ""
    return render_template(template_name_or_list="index.html",|
                           username = name)
```

Welcome

No Account

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

- When a user visits the root URL ('/'), the index function is called. Inside the index function, we define a variable username with the value "Felix" and pass it to the HTML template.

```
#import các thư viện
from flask import Flask, render_template
#Khởi tạo
app = Flask(__name__)

@app.route("/")
def Index():
    username = "Felix"
    return render_template(template_name_or_list="index.html", username=username)

#Chạy ứng dụng
if __name__ == "__main__":
    app.run();
```

- In the HTML template, we use double curly braces {{ }} to enclose the username variable. This syntax tells Jinja2 to replace {{ username }} with the value of the username variable that we passed from the Python code.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Jinja2 Example</title>
</head>
<body>
    <h1>Welcome {{ username }}!</h1>
</body>
</html>
```

← → ↻ ⓘ 127.0.0.1:5000

Welcome Felix!

CREATE NEW WEBSITE WITH FLASK

Khởi tạo Project Web

Import thư viện

Khởi tạo ứng dụng App

Định tuyến để gọi vào hàm

Chuẩn bị folder

- templates
- static

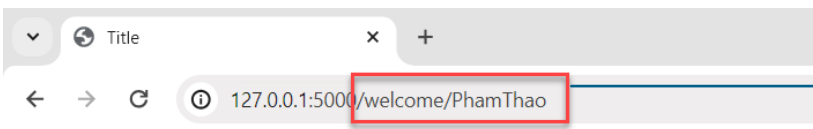
Hàm web

Chuẩn bị dữ liệu

Render web theo html template

Template

- Hiển thị biến
- Hiển thị biến từ định tuyến



Welcome PhamThao

PhamThao

```
from flask import Flask, render_template

# Khởi tạo biến app
app = Flask(__name__)

@app.route("/")
def Index():
    # Khai báo mặc định một giá trị username
    # name = "Felix"
    name = ""
    return render_template(template_name_or_list="index.html",
                           username = name)

# pname là tham số sẽ được truyền từ địa chỉ url.
# pname sẽ nhận là phần tham số sau welcome
@app.route("/welcome/<pname>")
def welcome(pname):
    # Khai báo mặc định một giá trị username
    return render_template(template_name_or_list="welcome.html",
                           username = pname)
```

CREATE NEW WEBSITE WITH FLASK

Khởi tạo Project Web

Import thư viện

Khởi tạo ứng dụng App

Định tuyến để gọi vào hàm

Chuẩn bị folder

- templates
- static

Hàm web

Chuẩn bị dữ liệu

Render web theo html template

Template

- Hiển thị biến
- Hiển thị biến từ định tuyến

→ × ngoisao.vnexpress.net/4-thoi-quen-pho-bien-lam-suy-giam-tri-nho-4719009.html

← → https://ngoisao.vnexpress.net/abc-4719009.html

← → https://vnexpress.net/nghe-si-xuc-dong-nhan-danh-hieu-nsnd-nsut-4719009.html

ngoisao.vnexpress.net/error.html?/4-thoi-quen-pho-bien-lam-suy-giam-tri-nho-4719009ABC.html

Liên hệ quảng cáo



Showbiz Thời trang Làm đẹp Xem Ăn chơi Lối sống Thể thao Thời cuộc P

404

Không tìm thấy đường dẫn này

Bạn có thể truy cập vào [trang chủ](#) hoặc sử dụng ô dưới đây để tìm kiếm

Tim kiếm thông tin



← → × cafef.vn/giam-doc-chien-luoc-vpbanks-day-chua-phai-giai-doan-tien-re-188240306103749213.chn

CAFEF
TRANG THÔNG TIN ĐIỆN TỬ TỔNG HỢP

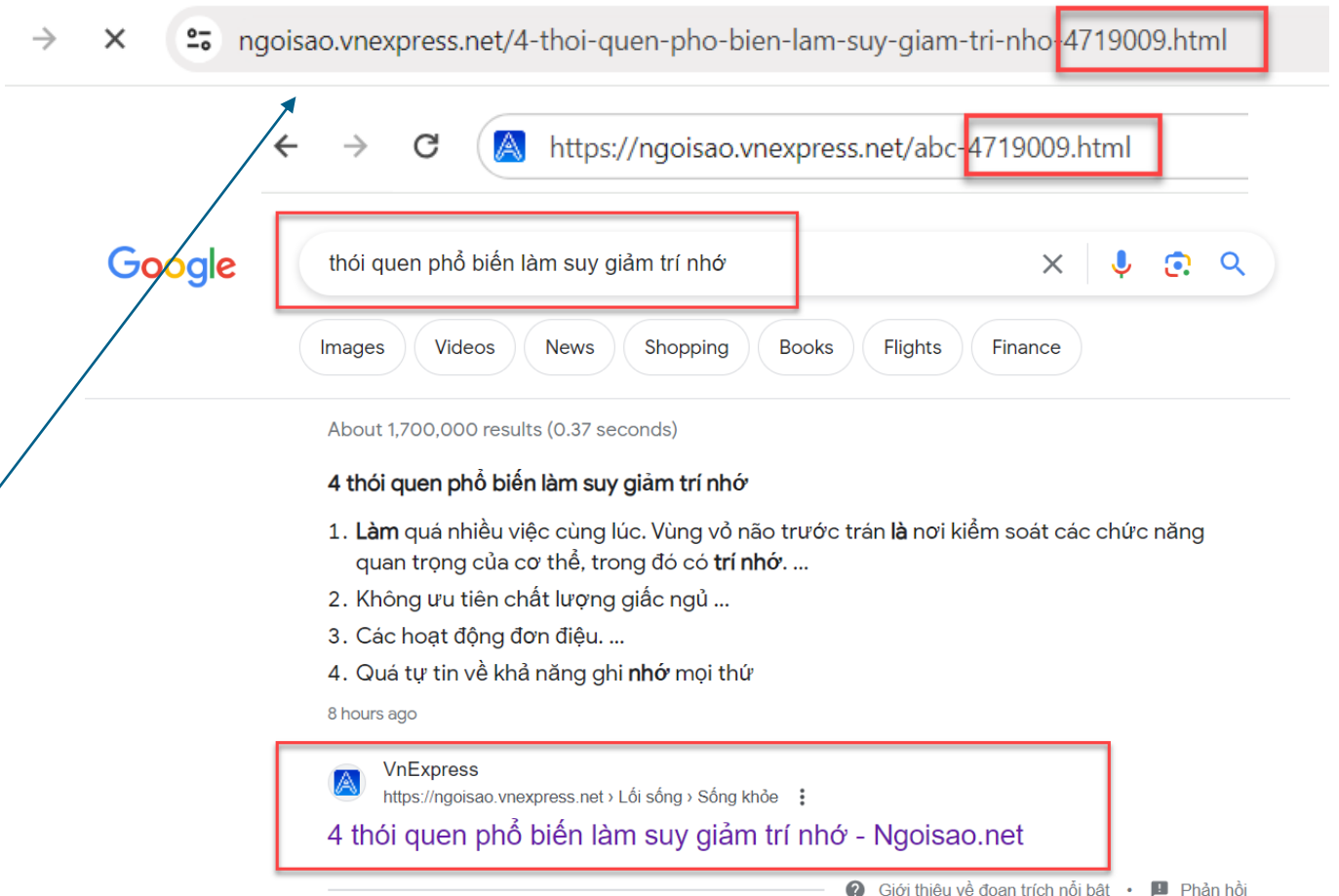
Chọn mã CK
cần theo dõi

Chọn mã CK
cần theo dõi

▼ Title × Hé lộ lý do khiến Facebook, Inst × +

← → https://dantri.com.vn/suc-manh-so/he-lo-ly-do-khien-facebook-instagram-bi-sap-tren-toan-cau-20240306151744574.htm

CREATE NEW WEBSITE WITH FLASK



USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

```
from flask import Flask, render_template, request
```

```
app = Flask(__name__)
```

```
app.template_folder = "templates"
```

```
app.static_folder = 'static'
```

```
@app.route("/")
```

```
def index():
```

```
    return render_template("index.html")
```

```
@app.route("/welcome/<pname>")
```

```
def welcome(pname):
```

```
    return render_template(template_name_or_list="welcome.html", account=pname);
```

```
if __name__ == '__main__':
```

```
    app.run()
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Hello World</title>
```

```
</head>
```

```
<body>
```

```
    <p>Hello felix!</p>
```

```
    <a href="{{ url_for('index') }}">Index</a><br>
```

```
    <a href="{{ url_for('loadPage00') }}">If</a><br>
```

```
    <a href="{{ url_for('loadPage01') }}">For</a><br>
```

```
    <a href="{{ url_for('loadPage02') }}">Menu</a><br>
```

```
    <a href="{{ url_for('loadPage03') }}">Table</a><br>
```

```
    <a>Welcome with parameter</a>
```

```
    {% block body %}
```

```
        <p>This is a Flask application.</p>
```

```
    {% endblock %}
```

```
</body>
```

```
</html>
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML IF SYNTAX

```
from flask import Flask, render_template, request
app = Flask(__name__)
app.template_folder = "templates"
app.static_folder = 'static'
@app.route("/")
def index():
    return render_template("index.html")

@app.route("/loadAccount")
def loadAccount():
    acc = "Felix"
    return render_template(template_name_or_list="account.html",
                           account=acc);

if __name__ == '__main__':
    app.run()
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Account</title>
6 </head>
7 <body>
8     <h1>
9         Account:
10    </h1>
11    <h2>{{account}}</h2>
12
13 </body>
14 </html>
```

```
<h2>
    <div>
        {% if account!=" " %}
        {{account}}
        {%else%}
        No Account
        {% endif %}
    </div>
</h2>
```

Account

127.0.0.1:5000/loadAccount

Account:

Felix

Felix

Web 32 Flask 03 - JinJa2 Demo - main.py

Web 32 Flask 03 - JinJa2 Demo - account.html

Project

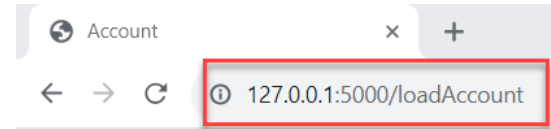
- menu.css
- style_table_dataframe.css
- templates
 - .idea
 - account.html
 - index.html
 - main_01.html
 - main_02.html
 - main_09.html
 - Page_00_If.html
 - Page_01_For.html
 - Page_02_For Menu.html
 - Page_03_Table.html
 - Page_04_Table.html
 - Page_06_MultipleTable.html
 - table.html
 - welcome.html
- gradedata.csv
- main.py
- main_02_stylefor.py
- main_02_styleformenu.py
- main_03_table.py
- main_04_table.py
- main_09_pandas.py
- External Libraries

```
3 # Press Shift+F10 to execute it or rep
4 # Press Double Shift to search everywhere for classes, f
5
6 from flask import Flask, render_template, request
7 app = Flask(__name__)
8 app.template_folder = "templates"
9 app.static_folder = 'static'
10 @app.route("/")
11 def index():
12     return render_template("index.html")
13 @app.route("/loadAccount")
14 def loadAccount():
15     acc = "Felix"
16     return render_template(
17         template_name_or_list: "account.html",
18         account=acc);
19 if __name__ == '__main__':
20     app.run()
```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Account</title>
6 </head>
7 <body>
8     <h1>
9         Account:
10     </h1>
11     <h2>{{account}}</h2>
12     <h2>
13         <div>
14             {% if account!=" " %}
15                 {{account}}
16             {%else%}
17                 No Account
18             {% endif %}
19         </div>
20     </h2>
21 </body>
22 </html>
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML IF SYNTAX

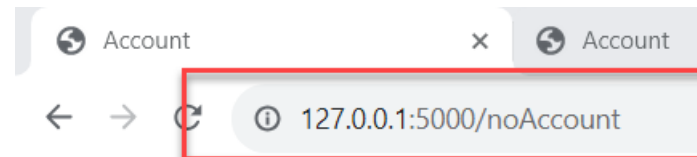
```
from flask import Flask, render_template, request
app = Flask(__name__)
app.template_folder = "templates"
app.static_folder = 'static'
@app.route("/")
def index():
    return render_template("index.html")
@app.route("/loadAccount")
def loadAccount():
    acc = "Felix"
    return render_template(
        template_name_or_list: "account.html",
        account=acc);
@app.route("/noAccount")
def noAccount():
    acc = ""
    return render_template(
        template_name_or_list: "account.html",
        account=acc);
```



Account:

Felix

Felix



Account:

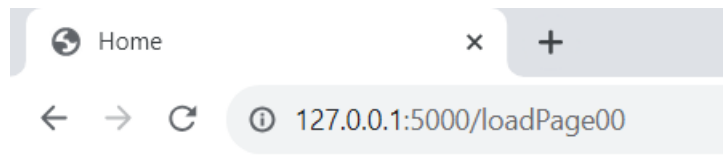
No Account

```
account.html x main.py x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <title>Account</title>
6 </head>
7 <body>
8 <h1>
9 Account:
10 </h1>
11 <h2>{{account}}</h2>
12 <h2>
13 <div>
14 {% if account!=" " %}
15 {{account}}
16 {%else%}
17 No Account
18 {% endif %}
19 </div>
20 </h2>
21 </body>
22 </html>
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML IF SYNTAX

```
@app.route("/loadPage00")
def loadPage00():
    acc = "Felix"
    return render_template(
        template_name_or_list: "Page_00_If.html",
        name=acc);

if __name__ == '__main__':
    app.run()
```

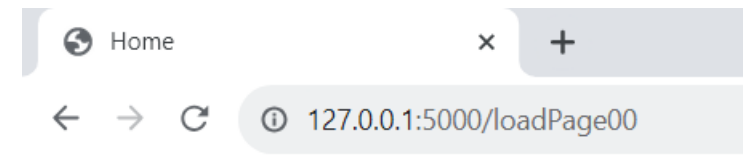


Welcome, Felix

Welcome, Felix

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Home</title>
</head>
<body>
    <!--Hiện tên -->
    <h3>Welcome, {{name}}</h3>
    <!--Kiểm tra rồi mới hiện tên -->
    {% if name!=" " %}
        <h2>Welcome, {{name}}</h2>
    {%else%}
        <h2>No Account</h2>
    {% endif %}
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<style>
    h2 {
        color: blue;
    }
    h3 {
        color: red;
    }
</style>
<head>
    <meta charset="UTF-8">
    <title>Home</title>
</head>
```



Welcome, Felix

Welcome, Felix

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML IF SYNTAX

The image shows a web development environment with three main components: a code editor on the left, a file explorer in the center, and a browser window on the right.

Code Editor (Left): Displays the HTML template structure. The `<style>` block contains CSS rules for `h2` (color: blue) and `h3` (color: red). The `<head>` block includes a `<link>` tag for the stylesheet, which is highlighted with a red box. The `</head>` tag is also highlighted.

File Explorer (Center): Shows the project structure. The `static/css` directory is expanded, and the `style.css` file is highlighted with a red box. Other files in the directory include `style_table.css`, `menu.css`, and `style_table_dataframe.css`.

Browser Window (Right): Displays the rendered page. The address bar shows the URL `127.0.0.1:5000/loadPage00`. The page content includes the text `Welcome, Felix` in blue and `Welcome, Felix` in red.

Code Snippets:

```
<!DOCTYPE html>
<html lang="en">
<style>
  h2 {
    color: blue;
  }
  h3 {
    color: red;
  }
</style>

<head>
  <meta charset="UTF-8">
  <title>Home</title>
  <!-- Cach 1-->
  <link rel="stylesheet" href="static/style.css">

  <!-- Cach 2-->
  <link rel="stylesheet" type="text/css"
        href="{{ url_for('static',
                          filename='/style.css') }}">
</head>
```

Browser Output:

Home

127.0.0.1:5000/loadPage00

Welcome, Felix

Welcome, Felix

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML FOR SYNTAX

```
3 <head>
4   <meta charset="UTF-8">
5   <title>Title</title>
6 </head>
7 <body>
8   <div>
9     <ul>
10      {% for item in seq %}
11        <li>{{ item }}</li>
12      {% endfor %}
13    </ul>
14  </div>
15 </body>
16 </html>
```

```
@app.route("/loadPage01")
def loadPage01():
    lst = [0, 1, 2, 3]
    return render_template(template_name_or_list="page_01_for.html", seq=lst);

if __name__ == '__main__':
    app.run()
```

← → ↻ ⓘ 127.0.0.1:5000/Page01

-
- {% for item in seq %}
- {{ item }}
- {% endfor %}
-

- 0
- 1
- 2
- 3

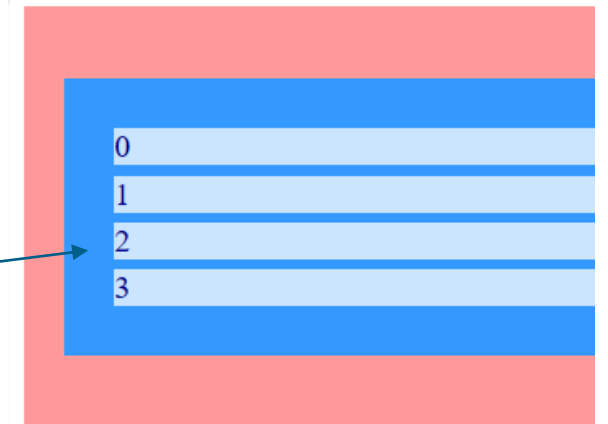
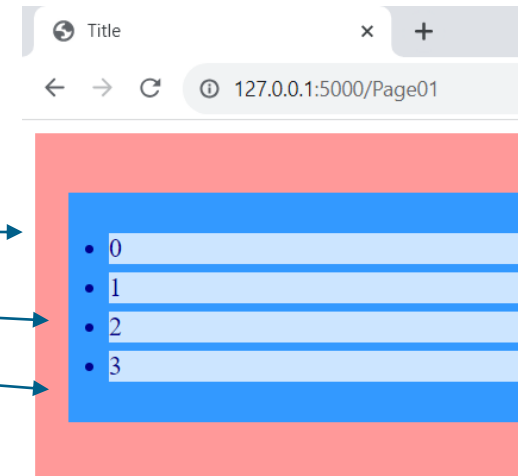
USING JINJA2 – DYNAMIC DATA RENDERING IN HTML FORMAT WITH CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <style>
    .one { background: #ff9999;padding: 20px;}
    .one ul {background: #3399ff; padding: 20px;}
    .one ul li {background: #cce5ff;color: darkblue; margin: 5px;}
  </style>
</head>
<body>
<div class="one">
  <ul>
    {% for item in seq %}
      <li>{{ item }}</li>
    {% endfor %}
  </ul>
</div>
</body>
</html>
```

html

Jinja2

Format with CSS



USING JINJA2 – DYNAMIC DATA RENDERING IN HTML FORMAT WITH CSS

The image illustrates the dynamic rendering of an HTML template using Jinja2. It shows the template code, the CSS file, the Python render function, and the resulting rendered HTML output in a browser.

HTML Template (main_01.html):

```
<!DOCTYPE html>
<html lang="en">
  <link rel="stylesheet" href="static/main_01.css">
  <head>
    <title>{{mytitle}}</title>
  </head>
  <body>
    <h1>Hello world</h1>
    <h2>{{mycontent}}</h2>
    <p>content</p>
  </body>
</html>
```

CSS File (main_01.css):

```
body {
  background-color: powder blue;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Python Render Function:

```
return render_template(template_name_or_list: 'main_01.html',
                       mycontent="my name is Felix",
                       mytitle="Felix!")
```

Browser Output (127.0.0.1:5000):

Hello world

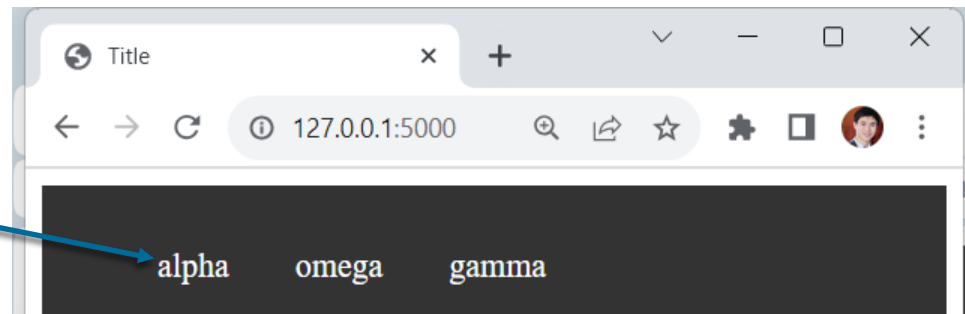
my name is Felix

content

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML FORMAT WITH CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Title</title>
  <link rel="stylesheet" href="static/menu.css">
</head>
<body>
<div class="menu">
  <ul>
    {% for item in seq %}
      <li><a href="{{ item[1] }}">
        {{ item[0] }}</a></li>
    {% endfor %}
  </ul>
</div>
</body>
</html>
```

```
@app.route("/loadPage02")
def loadPage02():
    lst = ["alpha", "alpha.html"],
          ["omega", "omega.html"],
          ["gamma", "gamma.html"]
    return render_template(
        template_name_or_list: "Page_02_For Menu.html",
        seq=lst);
if __name__ == '__main__':
    app.run()
```



USING JINJA2 – DYNAMIC DATA RENDERING IN HTML FORMAT WITH CSS

```
.menu {  
    list-style-type: none;  
    margin: 0;  
    padding: 0;  
    overflow: hidden;  
    background-color: #333;  
    color: white;  
}  
  
.menu ul {  
    list-style-type: none;  
}  
  
.menu ul li {  
    float: left;  
}
```

```
.menu ul li a {  
    display: block;  
    color: white;  
    text-align: center;  
    padding: 14px 16px;  
    text-decoration: none;  
}  
  
.menu ul li a:hover:not(.active) {  
    background-color: #111;  
}  
  
.menu ul li a.active {  
    float: right;  
}  
  
.active {  
    background-color: #4CAF50;  
}
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

DATA FROM PANDAS

■ Pass the Data Frame to the Template

```
from flask import Flask
from flask import render_template

# Khởi tạo biến app
app = Flask(__name__)

@app.route("/")
def loadPage03():
    import pandas as pd
    data = {
        'Name': ['John', 'Anna', 'Peter', 'Linda'],
        'Age': [28, 24, 35, 32],
        'City': ['New York', 'Paris', 'Berlin', 'London']
    }
    df = pd.DataFrame(data)
    html_data = df.to_html(classes='data', escape=False);
    return render_template(template_name_or_list: 'Page_03_Table.html',
                           table=html_data)

# Press the green button in the gutter to run the script.
if __name__ == '__main__':
    # app.run(debug="True")
    app.run()
```

The screenshot shows a code editor with three tabs: `Page_03_Table.html`, `style_table.css`, and `main_03_table.py`. The `Page_03_Table.html` file contains the following HTML code:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <title>Home</title>
5     <!--<link rel="stylesheet"
6         href="static/style_table_dataframe.css" >-->
7     <link rel="stylesheet" type="text/css"
8         href="{{ url_for('static',
9             filename='/css/style_table.css') }}">
10
11 </head>
12 <body>
13     <h1>Table from DataFrame with CSS</h1>
14     {{ table|safe }}
15 </body>
16 </html>
```

The `style_table.css` file is also visible in the file explorer on the right, showing the following CSS code:

```
body {
    font-family: sans-serif;
    font-size: 1.2em;
    color: #333;
    background-color: #f9f9f9;
}

table {
    width: 100%;
    border-collapse: collapse;
}

table tr {
    background-color: #fff;
}

table tr:nth-child(2n) {
    background-color: #f2f2f2;
}

table tr td {
    padding: 5px;
}
```

The file explorer on the right shows the project structure:

- Web 32 Flask 03 - JinJa2 Demo D:\BM
 - static
 - css
 - style_table.css
 - menu.css
 - style_table_dataframe.css

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML DATA FROM PANDAS AND FORMAT WITH CSS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Home</title>
  <link rel="stylesheet" type="text/css"
        href="{{ url_for('static',
                          filename='/css/style_table.css') }}">
</head>
<body>
  <h1>Table from DataFrame with CSS</h1>
  {{ table|safe }}
</body>
</html>
```

Table from DataFrame

	Name	Age	City
0	John	28	New York
1	Anna	24	Paris
2	Peter	35	Berlin
3	Linda	32	London

Table from DataFrame

	Name	Age	City
0	John	28	New York
1	Anna	24	Paris
2	Peter	35	Berlin
3	Linda	32	London

```
.dataframe {
  border-collapse: collapse;
  width: 100%;
  border: 1px solid #ddd;
}

/* Style for table headers */
.dataframe th {
  background-color: #f2f2f2;
  text-align: left;
  padding: 8px;
  border: 1px solid #ddd;
}
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

DATA FROM CSV TO PANDAS AND FORMAT WITH CSS

- Load data from a CSV file into a Pandas DataFrame and then pass the DataFrame to an HTML table
 - Import the necessary libraries
 - Load the data from the CSV file into a Pandas DataFrame
 - Create an HTML table from the DataFrame using the `to_html()` method
 - use the `html_table` variable to embed the DataFrame as an HTML table in your web application or save it to an HTML file:

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

DATA FROM CSV TO PANDAS AND FORMAT WITH CSS

```
from flask import Flask
from flask import render_template
# Khởi tạo biến app
app = Flask(__name__)
1 usage
@app.route("/")
def load_data():
    import pandas as pd
    df = pd.read_csv('gradedata.csv')
    #Chỉ hiện 5 bản ghi
    html_table = df.iloc[:5].to_html(classes='data', escape=False)
    return html_table
@app.route("/loadPage04")
def Introduction():
    html_table = load_data();
    return render_template(template_name_or_list='Page_04_Table.html',
                           tables=html_table,
                           titles=html_table.columns.values)
# Press the green button in the gutter to run the script.
if __name__ == '__main__':
    # app.run(debug="True")
    app.run()
```

```
<!DOCTYPE html>
<link rel="stylesheet" href="style_table_dataframe.css" >
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Title {{mytitle}}</title>
</head>
<body>
    <h1>Data from Data Frame</h1>
    <h2>{{mycontent}}</h2>
    {{ tables|safe }}
</body>
</html>
```

USING JINJA2 – DYNAMIC DATA RENDERING IN HTML

DATA FROM CSV TO PANDAS AND FORMAT WITH CSS

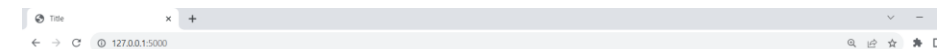
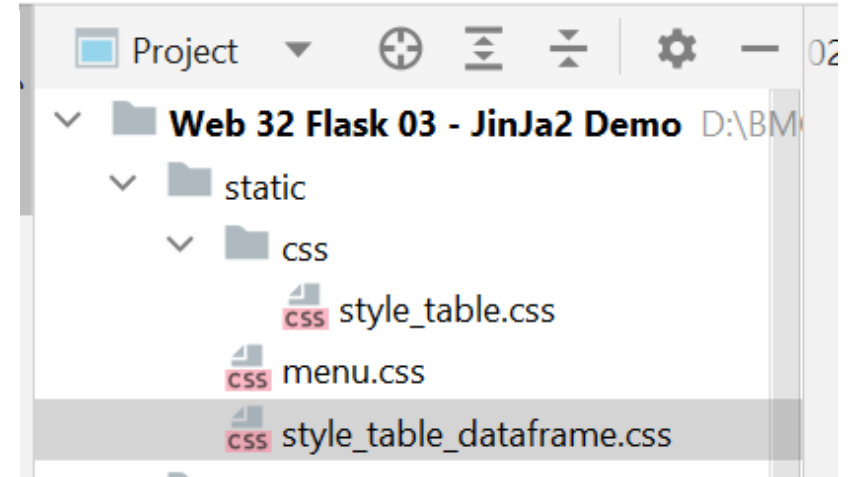
```
/* Style for the table */
.dataframe {
    border-collapse: collapse;
    width: 100%;
    border: 1px solid #ddd;
}

/* Style for table headers */
.dataframe th {
    background-color: #f2f2f2;
    text-align: left;
    padding: 8px;
    border: 1px solid #ddd;
}
```

```
/* Style for table data cells */
.dataframe td {
    text-align: left;
    padding: 8px;
    border: 1px solid #ddd;
}

/* Style for alternating row colors */
.dataframe tr:nth-child(even) {
    background-color: #f2f2f2;
}

/* Hover effect on rows */
.dataframe tr:hover {
    background-color: #ddd;
}
```



Data from Data Frame

	fname	lname	gender	age	exercise	hours	grade	address
0	Marcia	Pugh	female	17	2	7	82.4	9253 Richardson Road, Matawan, NJ 07747
1	Kadeem	Morrison	male	18	4	15	78.2	33 Spring Dr., Taunton, MA 02780
2	Nash	Powell	male	18	5	16	79.3	41 Hill Avenue, Mentor, OH 44060
3	Noelani	Wagner	female	14	2	7	83.2	8839 Marshall St., Miami, FL 33125
4	Noelani	Cherry	female	18	4	15	87.4	8304 Charles Rd., Lewis Center, OH 43035



NATIONAL ECONOMICS UNIVERSITY

NEU COLLEGE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY

CHAPTER IV

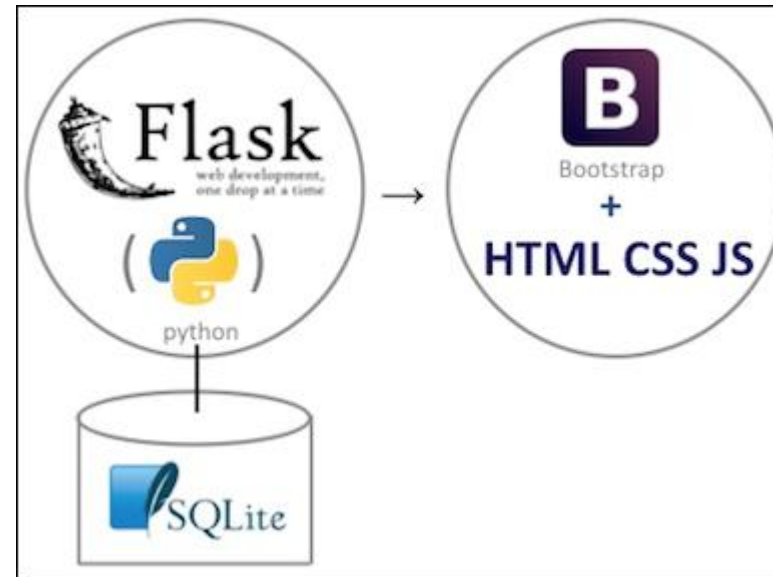
WEB DEVELOPMENT WITH PYTHON

WORKING WITH SQLITE DATABASE

OUTLINE

Working with SQLite Database:

- SQLite Tool
- Loading Data from the Database
- Working with Search Page
- Working with Login Page
- Working with Registration Form
- Working with Shopping Cart Page
 - Add to Cart
 - Modify Cart
 - Delete Cart



Working with SQLite Database:

- Practicing Loading Data from Other Databases
- CRUD - STORAGES

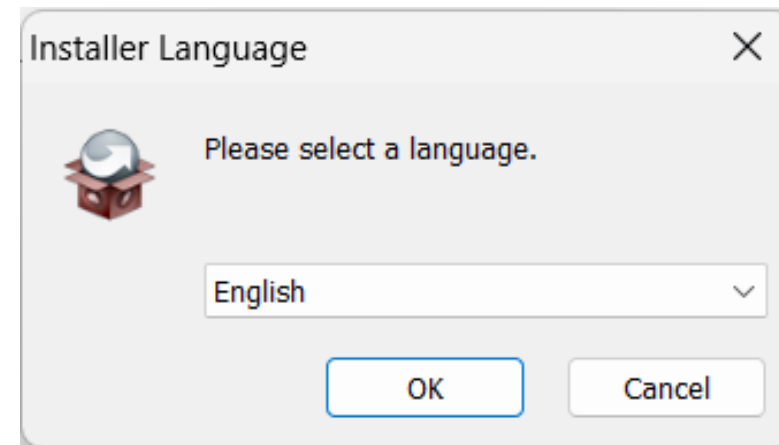


I. WORKING WITH SQLITE DATABASE

- Setup portable tool: <https://portableapps.com/download>
- DB Browser for SQLite

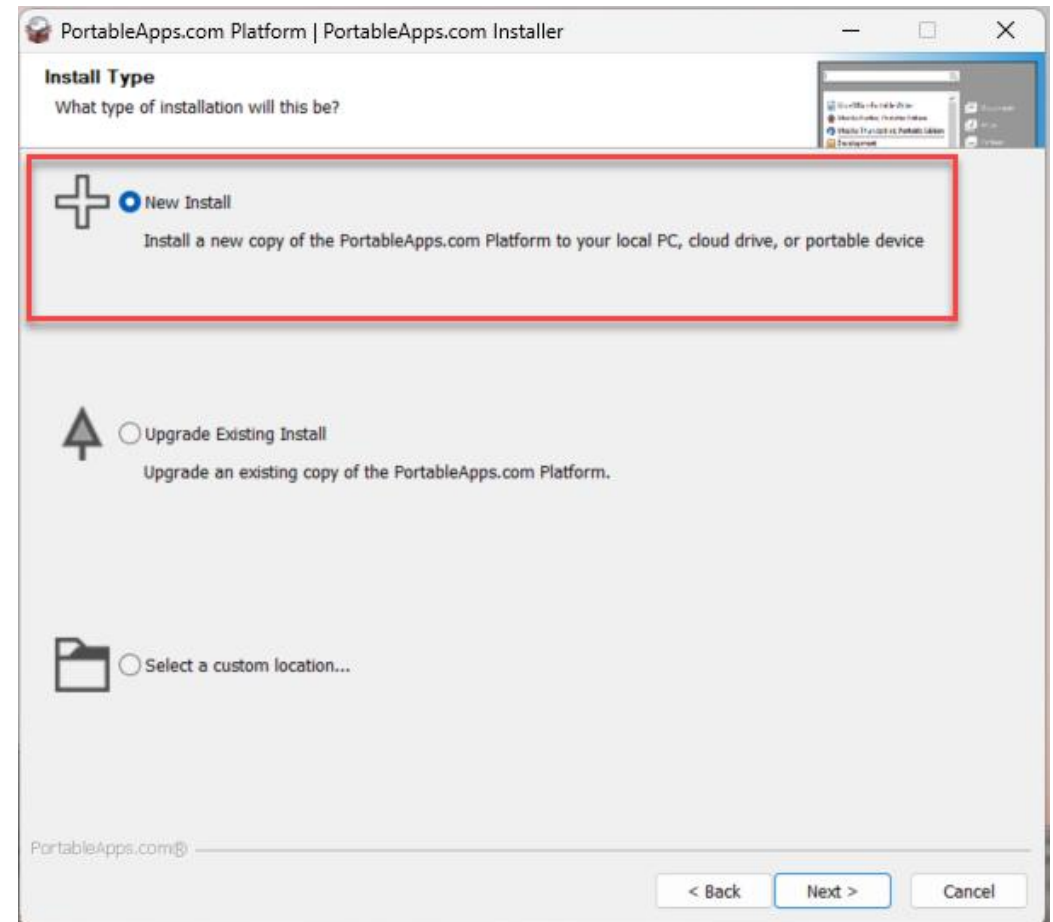
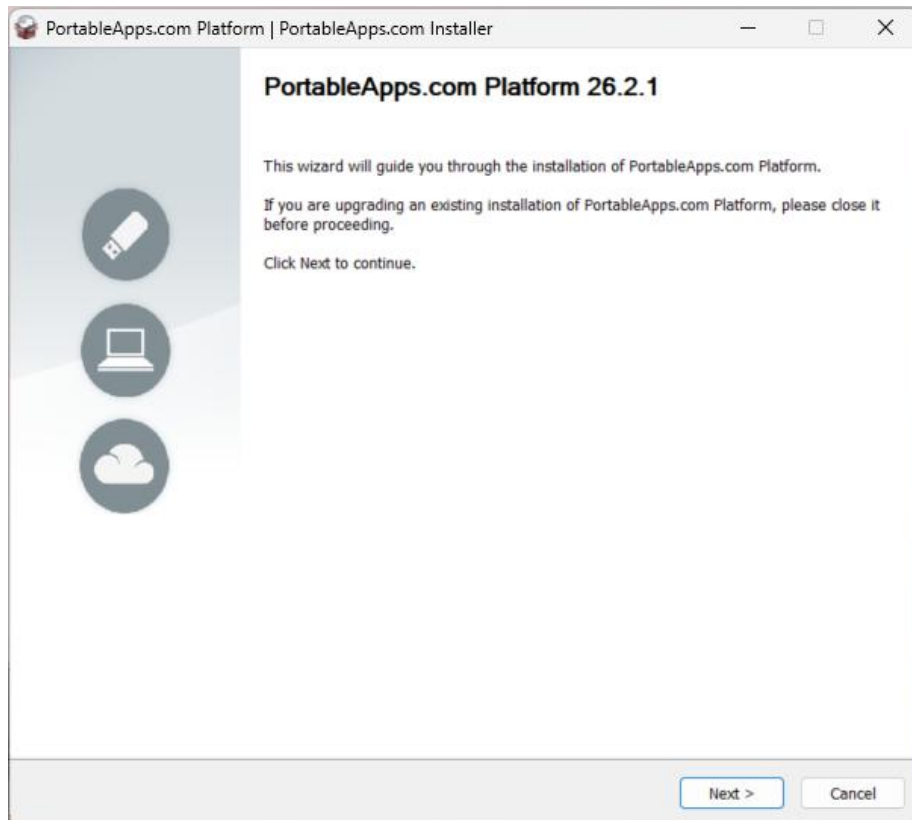
Portable App Issues

- Downloading a Portable App
- Installing a Portable App
- Using a Portable App
- Upgrading a Portable App
- Disabling a Portable App's Splash Screen
- Uninstalling a Portable App
- An app says it 'did not close properly' when started



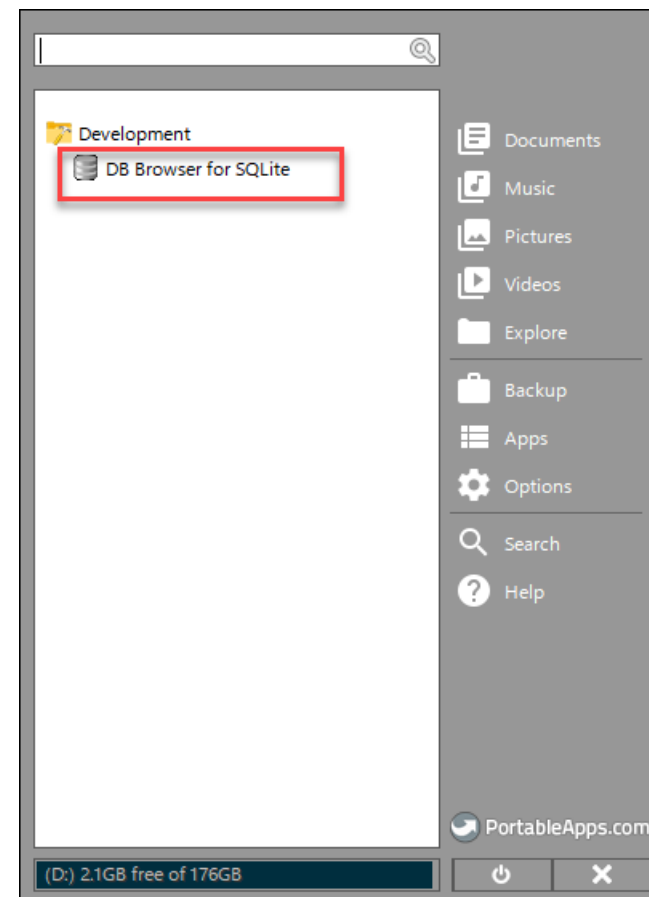
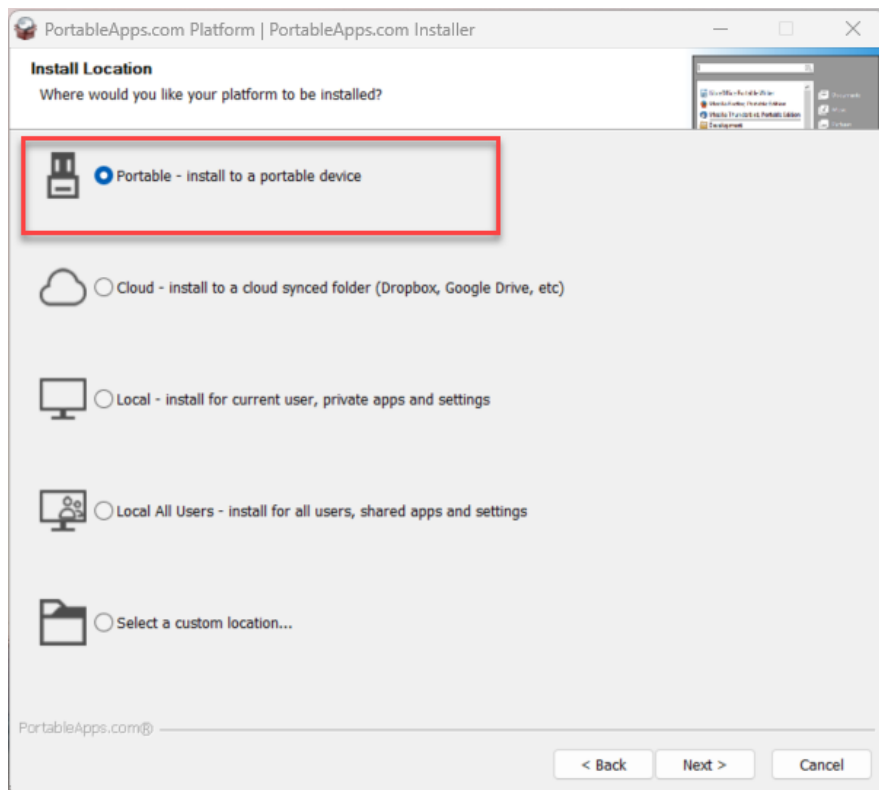
I. WORKING WITH SQLITE DATABASE

■ Setup



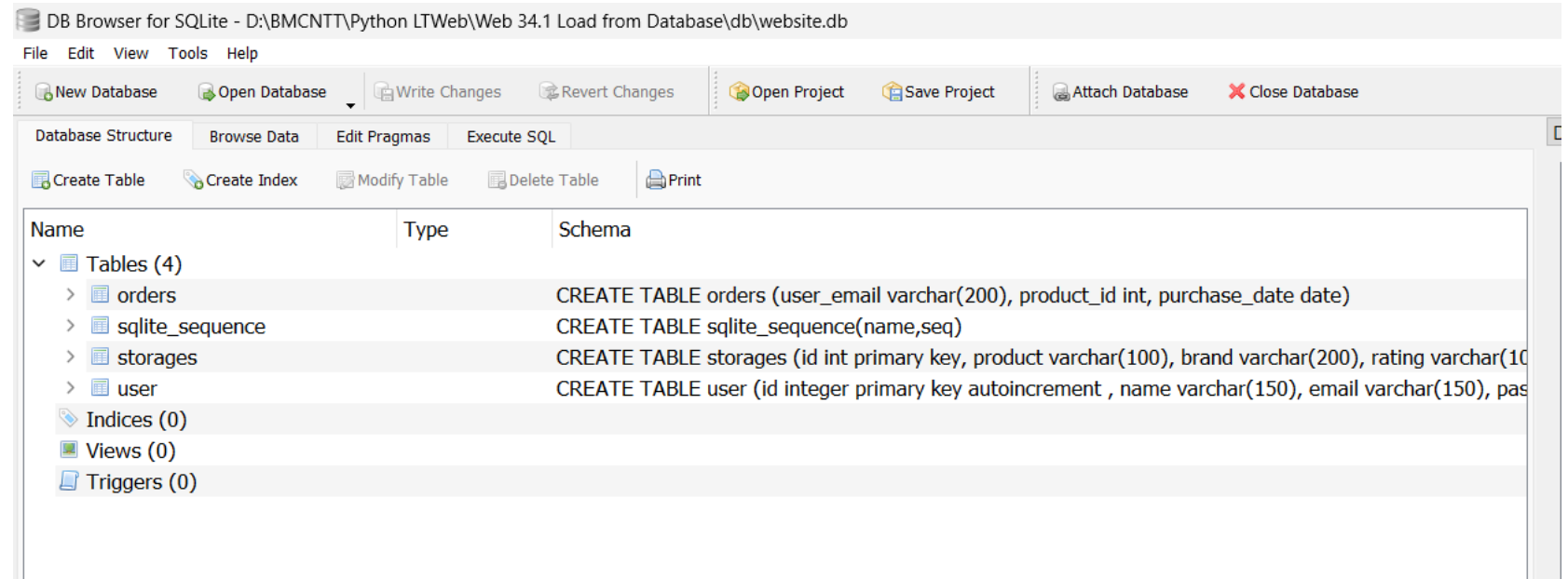
I. WORKING WITH SQLITE DATABASE

■ Setup



I. WORKING WITH SQLITE DATABASE

- Select * from Orders;
- Select * from Storages inner join orders on storages.id = orders.product_id where rating > 4



I. WORKING WITH SQLITE DATABASE

- Select * from Orders;
- Select * from Storages inner join orders on storages.id = orders.product_id where rating>4

DB Browser for SQLite - D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database\db\website.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

Table: storages

	id	product	brand	rating	model	picture	price	RAM	
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	id	product	brand	rating	model	picture	price	RAM	deta
2	1	laptop	HP	4.1	Pavilion 15-ab205tx	https://i.gadgets360cdn.com/product...	49,790	4GB	Size
3	2	laptop	HP	4.8	Pavilion 15-AU626TX	https://i.gadgets360cdn.com/product...	69,980	16GB	Size
4	3	laptop	Asus	4.3	R558UQ-DM539D	https://i.gadgets360cdn.com/product...	35,490	4GB	Size
5	4	laptop	HP	4.3	240 G4	https://i.gadgets360cdn.com/product...	27,500	4GB	Size
6	5	laptop	Acer	4.4	Aspire E E1-572G	https://i.gadgets360cdn.com/product...	59,500	4GB	Size
7	6	laptop	Dell	5	Inspiron 3000 3543	https://i.gadgets360cdn.com/product...	28,099	4GB	Size
8	7	laptop	Asus	4.5	X550LC-XX119H	https://i.gadgets360cdn.com/product...	46,000	4GB	Size
9	8	laptop	HP	3.5	Pavilion 15-AU020TX	https://i.gadgets360cdn.com/product...	72,500	4GB	Size
10	9	laptop	Asus	4.5	G551VW-FI242T	https://i.gadgets360cdn.com/product...	98,500	16GB	Size
11	10	laptop	HP	4	Pavilion 15-au620tx	https://i.gadgets360cdn.com/product...	56,950	8GB	Size
12	11	laptop	HP	4.4	Pavilion 15-au628tx	https://i.gadgets360cdn.com/product...	80,250	8GB	Size
13	12	laptop	HP	4.8	15-ay006tx	https://i.gadgets360cdn.com/product...	35,799	4GB	Size
14	13	laptop	Lenovo	4.3	G50 45	https://i.gadgets360cdn.com/product...	17,990	8GB	Size
15	14	laptop	HP	4.3	Pavilion 15-ba021ax	https://i.gadgets360cdn.com/product...	32,490	4GB	Size
16	15	laptop	HP	4.7	Pavilion 15-au009tx	https://i.gadgets360cdn.com/product...	72,249	8GB	Size
17	16	laptop	MSI	5	PS63 Modern 8RC	https://i.gadgets360cdn.com/product...	102,000	16GB	Size
18	17	laptop	Acer	3.5	TravelMate P2 TMP259-G2-MG	https://i.gadgets360cdn.com/product...	42,900	8GB	Size
19	18	laptop	HP	3.5	Pavilion 15-ab522tx	https://i.gadgets360cdn.com/product...	59,190	8GB	Size
20	19	laptop	HP	4.3	Pavilion 15-ac179tx	https://i.gadgets360cdn.com/product...	43,690	4GB	Size
21	20	laptop	Asus	4.1	R558UQ-DM540D	https://i.gadgets360cdn.com/product...	40,999	4GB	Size
22	21	laptop	Asus	4	R558UQ-DM701D	https://i.gadgets360cdn.com/product...	51,990	8GB	Size
23	22	laptop	Asus	5	R558UQ-DM175D	https://i.gadgets360cdn.com/product...	44,300	4GB	Size

I. WORKING WITH SQLITE DATABASE

Using sqlite3

- Is a module in Python's standard library
- Use the raw SQL (writing SQL queries directly)
- Steps
 - Connect to the database
 - **Fetch data from the table**
 - Close the database connection
- Change command text...

Using SQLAlchemy

- Is an Object Relational Mapper (ORM)
- Can simplifies database interactions with Python objects
- Steps
 - Configure
 - Defining the Object Model (example User Model)
 - Use query.all()
- Use other function: **filter()**...

I. WORKING WITH SQLITE DATABASE

DB Browser for SQLite - D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database Demo\db\website.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Database Close Database

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1

```
1 Select * from storages
```

	id	product	brand	rating	model	picture	p
1	id	product	brand	rating	model	picture	pri
2	1	laptop	HP	4.1	Pavilion 15-ab205tx	https://i.gadgets360cdn.com/product...	49,
3	2	laptop	HP	4.8	Pavilion 15-AU626TX	https://i.gadgets360cdn.com/product...	69,
4	3	laptop	Asus	4.3	R558UQ-DM539D	https://i.gadgets360cdn.com/product...	35,
5	4	laptop	HP	4.3	240 G4	https://i.gadgets360cdn.com/product...	27,
6	5	laptop	Acer	4.4	Aspire E E1-572G	https://i.gadgets360cdn.com/product...	59,
7	6	laptop	Dell	5	Inspiron 3000 3543	https://i.gadgets360cdn.com/product...	28,
8	7	laptop	Asus	4.5	X550LC-XX119H	https://i.gadgets360cdn.com/product...	46,
9	8	laptop	HP	3.5	Pavilion 15-AU020TX	https://i.gadgets360cdn.com/product...	72,
10	9	laptop	Asus	4.5	G551VW-FI242T	https://i.gadgets360cdn.com/product...	98,
11	10	laptop	HP	4	Pavilion 15-au620tx	https://i.gadgets360cdn.com/product...	56,
12	11	laptop	HP	4.4	Pavilion 15-au628tx	https://i.gadgets360cdn.com/product...	80,
13	12	laptop	HP	4.8	15-ay006tx	https://i.gadgets360cdn.com/product...	35,
14	13	laptop	Lenovo	4.3	G50 45	https://i.gadgets360cdn.com/product...	17,
15	14	laptop	HP	4.3	Pavilion 15-ba021ax	https://i.gadgets360cdn.com/product...	32,

Execution finished without errors.
Result: 796 rows returned in 7ms
At line 1:
Select * from storages

Data

id - product - brand

1 - laptop - HP

2 - laptop - HP

3 - laptop - Asus

4 - laptop - HP

5 - laptop - Acer

6 - laptop - Dell

7 - laptop - Asus

```
index.html
</head>
<body>
  <h1>Data</h1>
  <div class="menu">
    <ul>
      {% for row in table %}
        <li>{{ row[0] }} -
          {{ row[1] }} -
          {{ row[2] }}
        </li>
      {% endfor %}
    </ul>
  </div>
</body>
</html>
```

I. WORKING WITH SQLITE DATABASE

Using sqlite3

- **1. Connect to the Database:**
 - Begin by establishing a connection to the SQLite database using the sqlite3 module.
- **2. Fetch Data from the Table:**
 - Execute your SQL queries to fetch data from the desired table.
- **3. Close the Database Connection:**
 - It's essential to close the database connection after your operations are completed to free up resources.
- **4. Modify Command Text (Optional):**
 - If needed, you can modify the command text to perform specific operations like INSERT, UPDATE, DELETE, etc. Just adjust the SQL query within the cursor.execute() method.

```
2 from flask import Flask, render_template
3 import sqlite3
4 app = Flask(__name__)
5 sqldbname = 'db/website.db'
6
7 @app.route('/')
8 def index():
9     # Connect to the SQLite database
10    conn = sqlite3.connect(sqldbname)
11    # Create a cursor object to execute SQL queries
12    cursor = conn.cursor()
13    # Execute an SQL query to fetch data
14    cursor.execute('SELECT * FROM user')
15    data = cursor.fetchall()
16    # Close the database connection
17    conn.close()
18    return render_template(
19        template_name_or_list: 'index.html', table=data)
20    # Press the green button in the gutter to run the script.
21 if __name__ == '__main__':
22     app.run(debug=True)
23
```

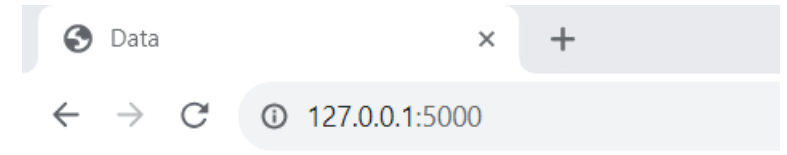
I. WORKING WITH SQLITE DATABASE

```
main_01_Sqlite_1.py x index.html x
1 # This is a sample Python script.
2 from flask import Flask, render_template
3 import sqlite3
4 app = Flask(__name__)
5 sqldbname = 'db/website.db'
6
7 @app.route('/')
8 def index():
9     # Connect to the SQLite database
10    conn = sqlite3.connect(sqldbname)
11    cursor = conn.cursor()
12    # Execute an SQL query to fetch data
13    cursor.execute('SELECT * FROM user')
14    data = cursor.fetchall()
15    # Close the database connection
16    conn.close()
17    return render_template(
18        template_name_or_list: 'index.html', table=data)
19 # Press the green button in the gutter to run the script.
20 if __name__ == '__main__':
21     app.run(debug=True)
22
```

```
index.html x style.css x main_01_Sqlite_1.py x
1 <!DOCTYPE html>
2 <html lang="en">
3 <link rel="stylesheet" type="text/css"
4     href="static/style.css">
5 <head>
6     <meta charset="UTF-8">
7     <title>Data</title>
8 </head>
9 <body>
10     <h1>Data</h1>
11     <div class="menu">
12         <ul>
13             {% for row in table %}
14                 <li>{{ row[0]}} -
15                     {{ row[1]}} -
16                     {{ row[2]}}
17                 </li>
18             {% endfor %}
19         </ul>
20     </div>
21 </body>
22 </html>
```

I. WORKING WITH SQLITE DATABASE

```
index.html x style.css x main_01_Sqlite_1.py x
1  /* Apply styles to the <ul> element */
2  ul {
3      list-style-type: none;
4      padding: 0; /* Remove default padding */
5  }
6
7  /* Apply styles to the <li> elements */
8  li {
9      margin: 0; /* Remove default margin */
10     padding: 8px 0; /* Add padding to top and bottom */
11     color: #333; /* Text color */
12 }
13
14 /* Style for list items on hover */
15 li:hover {
16     background-color: #f0f0f0; /* Background color on hover */
17     cursor: pointer; /* Change cursor to pointer on hover */
18 }
```



Data

1 - abc - abc@gmail.com
2 - def - def@gmail.com

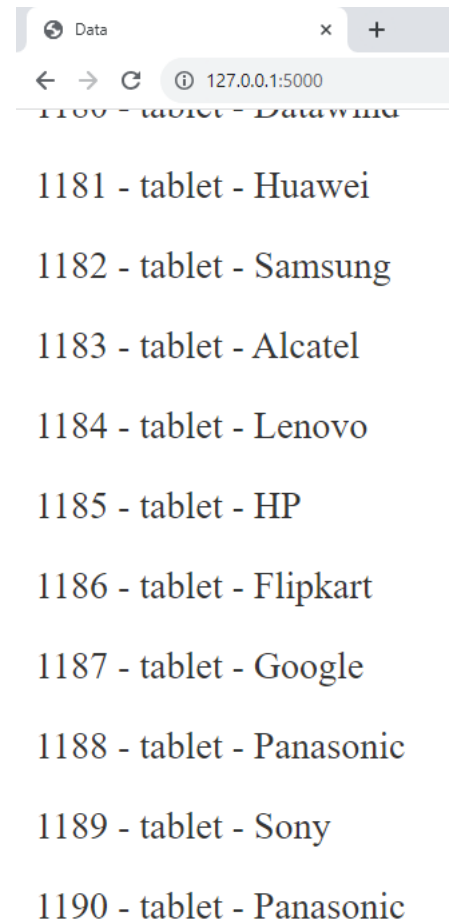
I. WORKING WITH SQLITE DATABASE

- Modify Command Text (Optional):
- Just adjust the SQL query within the `cursor.execute()` method. For example,
 - Load data from [Storages](#)
 - Load data from [Orders](#)
 - Working with [joins](#) and [conditions](#) to filter data
- If needed, you can modify the command text to perform specific operations like INSERT, UPDATE, DELETE, etc.

I. WORKING WITH SQLITE DATABASE

- Practice loading data from other tables.
- Test command text in DB Browser for SQLite Tool
- Using SQLite to filter data by searching from database
- Apply the conditions to Flask web

```
main_01_Sqlite_1.py x
1 # This is a sample Python script.
2 from flask import Flask, render_template
3 import sqlite3
4 app = Flask(__name__)
5 sqldbname = 'db/website.db'
6
7 @app.route('/')
8 def index():
9     # Connect to the SQLite database
10    conn = sqlite3.connect(sqldbname)
11    cursor = conn.cursor()
12
13    # Execute an SQL query to fetch data
14    cursor.execute("Select * from storages")
15    data = cursor.fetchall()
16    # Close the database connection
17    conn.close()
18    return render_template(
19        template_name_or_list: 'index.html', table=data)
20    # Press the green button in the gutter to run the script.
21 if __name__ == '__main__':
22     app.run(debug=True)
```

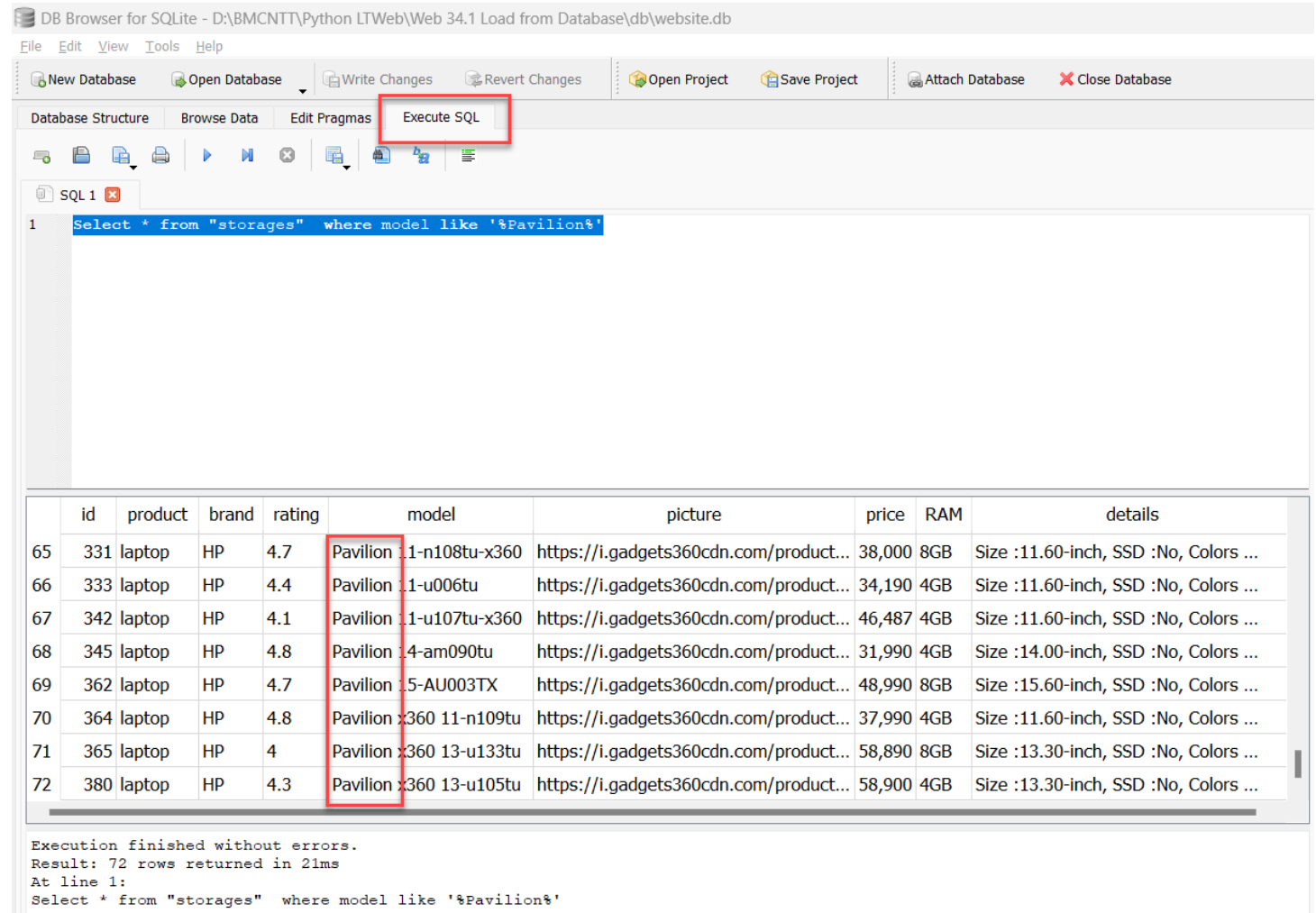


ID	Device
1181	tablet - Huawei
1182	tablet - Samsung
1183	tablet - Alcatel
1184	tablet - Lenovo
1185	tablet - HP
1186	tablet - Flipkart
1187	tablet - Google
1188	tablet - Panasonic
1189	tablet - Sony
1190	tablet - Panasonic

I. WORKING WITH SQLITE DATABASE

- Practice loading data from other tables.
- Test command text in **DB Browser for SQLite Tool**
- Using SQLite to filter data by searching from database:

Select * from
"storages"
where model like
'%Pavilion%'



The screenshot shows the DB Browser for SQLite application. The title bar indicates the database path: D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database\db\website.db. The menu bar includes File, Edit, View, Tools, and Help. The toolbar contains buttons for New Database, Open Database, Write Changes, Revert Changes, Open Project, Save Project, Attach Database, and Close Database. The main interface has three tabs: Database Structure, Browse Data, and Edit Pragma. The 'Execute SQL' button is highlighted with a red box. Below the tabs, the SQL editor shows the query: `Select * from "storages" where model like '%Pavilion%'`. The results are displayed in a table with 10 columns: id, product, brand, rating, model, picture, price, RAM, and details. The 'model' column is highlighted with a red box. The results show 72 rows, with the first 7 rows visible in the screenshot.

	id	product	brand	rating	model	picture	price	RAM	details
65	331	laptop	HP	4.7	Pavilion 1-n108tu-x360	https://i.gadgets360cdn.com/product...	38,000	8GB	Size :11.60-inch, SSD :No, Colors ...
66	333	laptop	HP	4.4	Pavilion 1-u006tu	https://i.gadgets360cdn.com/product...	34,190	4GB	Size :11.60-inch, SSD :No, Colors ...
67	342	laptop	HP	4.1	Pavilion 1-u107tu-x360	https://i.gadgets360cdn.com/product...	46,487	4GB	Size :11.60-inch, SSD :No, Colors ...
68	345	laptop	HP	4.8	Pavilion 4-am090tu	https://i.gadgets360cdn.com/product...	31,990	4GB	Size :14.00-inch, SSD :No, Colors ...
69	362	laptop	HP	4.7	Pavilion 5-AU003TX	https://i.gadgets360cdn.com/product...	48,990	8GB	Size :15.60-inch, SSD :No, Colors ...
70	364	laptop	HP	4.8	Pavilion x360 11-n109tu	https://i.gadgets360cdn.com/product...	37,990	4GB	Size :11.60-inch, SSD :No, Colors ...
71	365	laptop	HP	4	Pavilion x360 13-u133tu	https://i.gadgets360cdn.com/product...	58,890	8GB	Size :13.30-inch, SSD :No, Colors ...
72	380	laptop	HP	4.3	Pavilion x360 13-u105tu	https://i.gadgets360cdn.com/product...	58,900	4GB	Size :13.30-inch, SSD :No, Colors ...

Execution finished without errors.
Result: 72 rows returned in 21ms
At line 1:
Select * from "storages" where model like '%Pavilion%'

I. WORKING WITH SQLITE DATABASE

- Practice loading data from other tables.
- Test command text in DB Browser for SQLite Tool
- Using SQLite to filter data by searching from database
- Apply the conditions to Flask web

```
main_01_Sqlite_1.py x
1  # This is a sample Python script.
2  from flask import Flask, render_template
3  import sqlite3
4  app = Flask(__name__)
5  sqldbname = 'db/website.db'
6
7  @app.route('/')
8  def index():
9      # Connect to the SQLite database
10     conn = sqlite3.connect(sqldbname)
11     cursor = conn.cursor()
12
13     # Execute an SQL query to fetch data
14     cursor.execute("Select * from storages where Brand like '%HP%'")
15     data = cursor.fetchall()
16     # Close the database connection
17     conn.close()
18     return render_template(
19         template_name_or_list: 'index.html', table=data)
20     # Press the green button in the gutter to run the script.
21     if __name__ == '__main__':
22         app.run(debug=True)
```

Data x +
← → ↻ ⓘ 127.0.0.1:5000

378 - laptop - HP

380 - laptop - HP

391 - laptop - HP

824 - tablet - HP

840 - tablet - HP

859 - tablet - HP

949 - tablet - HP

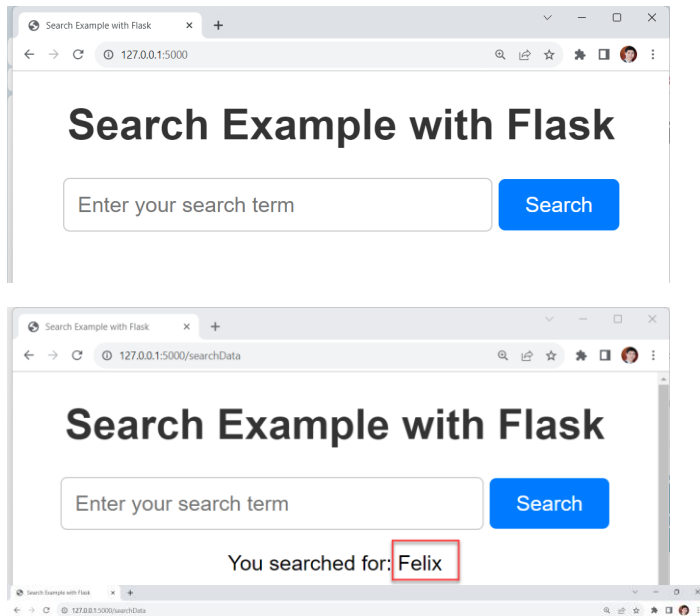
1115 - tablet - HP

1117 - tablet - HP

II. WORKING WITH SQLITE DATABASE

- Implemented Search Page Integrating Data from Database

III. SEARCH FORM



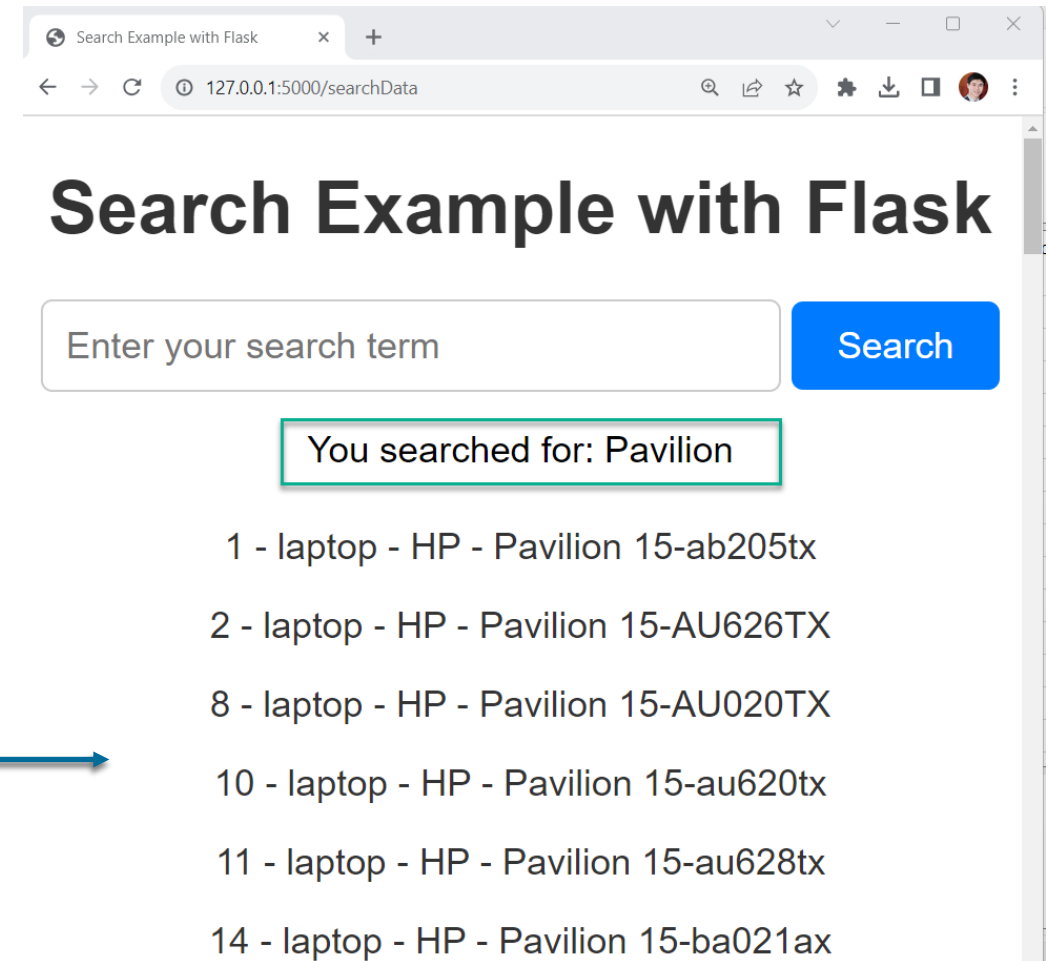
Search Example with Flask

Enter your search term

Search

You searched for: Felix

	fname	lname	gender	age	exercise	hours	grade	address
452	Felix	Spears	male	16	2	11	76.5	27 Greystone Street, Easton, PA 18042
805	Felix	Potter	male	18	4	11	82.1	8458 Ivy Court, Danvers, MA 01923
1689	Felix	Avila	male	16	5	11	89.7	9617 Brookside St., Ocoee, FL 34761
1930	Felix	Coleman	male	17	5	16	87.3	7014 South Riverview Avenue, Glen Burnie, MD 21060
1975	Felix	Freeman	male	15	4	11	75.2	671 Division Ave., Vineland, NJ 08360



III. SEARCH FORM

#Load dữ liệu và lọc ra bản ghi phù hợp

1 usage

```
def load_data(search_text):  
    import pandas as pd  
    df = pd.read_csv('gradedata.csv')  
    dfX = df  
    if search_text != "":  
        dfX = df[(df["fname"] == search_text) |  
                 (df["lname"] == search_text)]  
    print(dfX)  
    html_table = dfX.to_html(classes='data',  
                             escape=False)  
    return html_table
```

usage

```
def load_data_from_db(search_text):  
    sqldbname = 'db/website.db'  
    if search_text != "":  
        # Khai bao bien de tro toi db  
        conn = sqlite3.connect(sqldbname)  
        cursor = conn.cursor()  
        sqlcommand = ("Select * from storages "  
                      "where model like '%" + search_text + "%'")  
        cursor.execute(sqlcommand)  
        data = cursor.fetchall()  
        conn.close()  
    return data
```

III. SEARCH FORM

#Đối với phương thức Search

```
@app.route(rule: '/searchData', methods=['POST'])
```

```
def searchData():
```

#Get data from Request

```
search_text = request.form['searchInput']
```

#Thay bằng hàm load dữ liệu từ DB

```
html_table = load_data_from_db(search_text)
```

```
print(html_table)
```

```
return render_template(template_name_or_list: 'SearchWithCSSDataDB.html',  
                        search_text=search_text,  
                        table=html_table  
                        )
```

```
def load_data_from_db(search_text):
```

```
    sqldbname = 'db/website.db'
```

```
    if search_text != "":
```

Khai báo biến để truy cập db

```
    conn = sqlite3.connect(sqldbname)
```

```
    cursor = conn.cursor()
```

```
    sqlcommand = ("Select * from storages "
```

```
                  "where model like '%" + search_text + "%'"
```

```
    cursor.execute(sqlcommand)
```

```
    data = cursor.fetchall()
```

```
    conn.close()
```

```
    return data
```

```
<h1>Search Example with Flask</h1>
```

```
<form method="POST" action="/searchData">
```

```
    <input type="text" name="searchInput"
```

```
          placeholder="Enter your search term">
```

```
    <button type="submit">Search</button>
```

```
</form>
```

```
<p>
```

```
    {% if search_text %}
```

```
    You searched for: {{ search_text }}
```

```
    <div class="menu">
```

```
        <ul>
```

```
            {% for row in table %}
```

```
                <li>{{ row[0] }} -
```

```
                    {{ row[1] }} -
```

```
                    {{ row[2] }} -
```

```
                    {{ row[4] }}
```

```
                </li>
```

```
            {% endfor %}
```

```
        </ul>
```

```
    </div>
```

```
    {% endif %}
```

```
</p>
```

III. SEARCH FORM

#Đối với phương thức Search

```
@app.route(rule: '/searchData', methods=['POST'])
```

```
def searchData():
```

#Get data from Request

```
search_text = request.form['searchInput']
```

#Thay bằng hàm load dữ liệu từ DB

```
html_table = load_data_from_db(search_text)
```

```
print(html_table)
```

```
return render_template(template_name_or_list: 'SearchWithCSSDataDB.html',  
                        search_text=search_text,  
                        table=html_table  
                        )
```

```
def load_data_from_db(search_text):
```

```
    sqldbname = 'db/website.db'
```

```
    if search_text != "":
```

Khai báo biến để truy cập db

```
    conn = sqlite3.connect(sqldbname)
```

```
    cursor = conn.cursor()
```

```
    sqlcommand = ("Select * from storages "
```

```
                  "where model like '%" + search_text + "%'"
```

```
    cursor.execute(sqlcommand)
```

```
    data = cursor.fetchall()
```

```
    conn.close()
```

```
    return data
```

```
<h1>Search Example with Flask</h1>
```

```
<form method="POST" action="/searchData">
```

```
    <input type="text" name="searchInput"
```

```
          placeholder="Enter your search term">
```

```
    <button type="submit">Search</button>
```

```
</form>
```

```
<p>
```

```
    {% if search_text %}
```

```
    You searched for: {{ search_text }}
```

```
    <div class="menu">
```

```
        <ul>
```

```
            {% for row in table %}
```

```
                <li>{{ row[0] }} -
```

```
                {{ row[1] }} -
```

```
                {{ row[2] }} -
```

```
                {{ row[4] }}
```

```
            </li>
```

```
            {% endfor %}
```

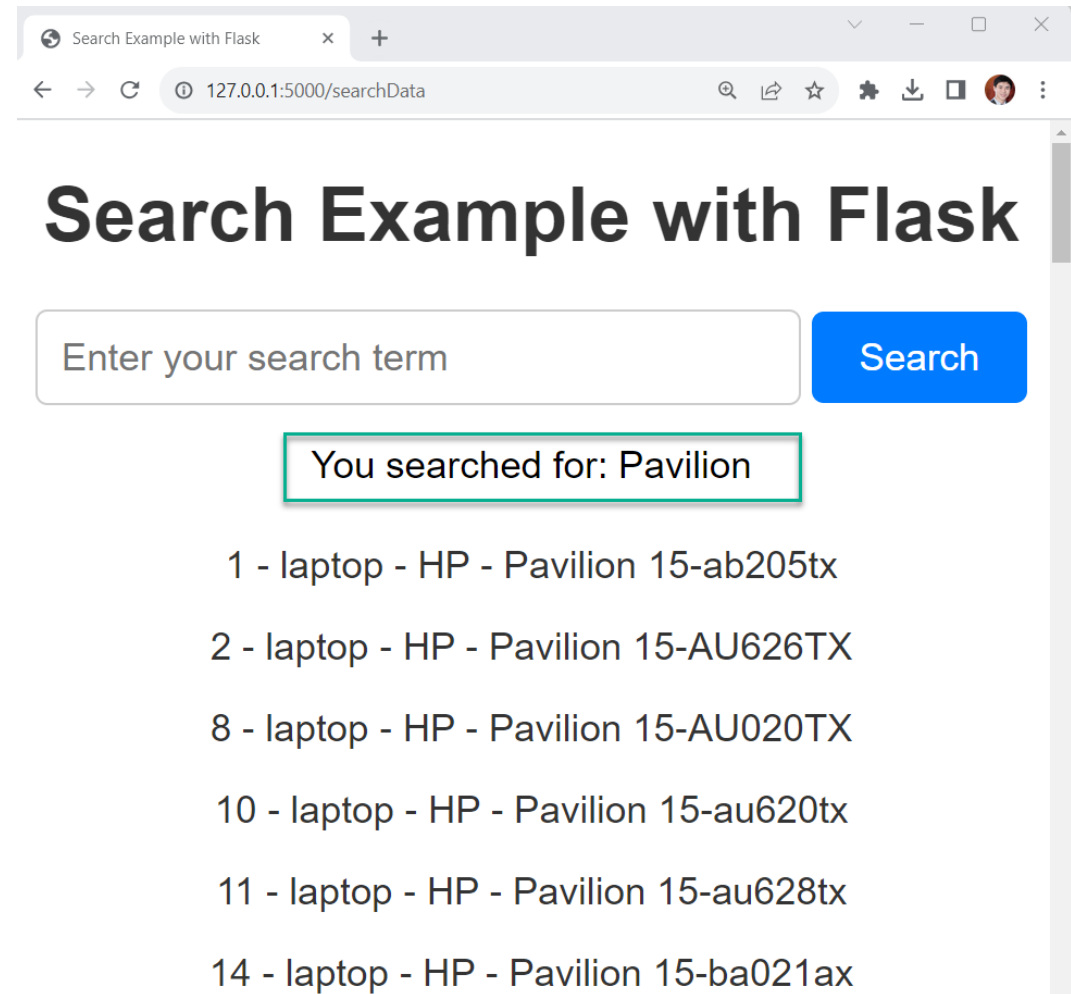
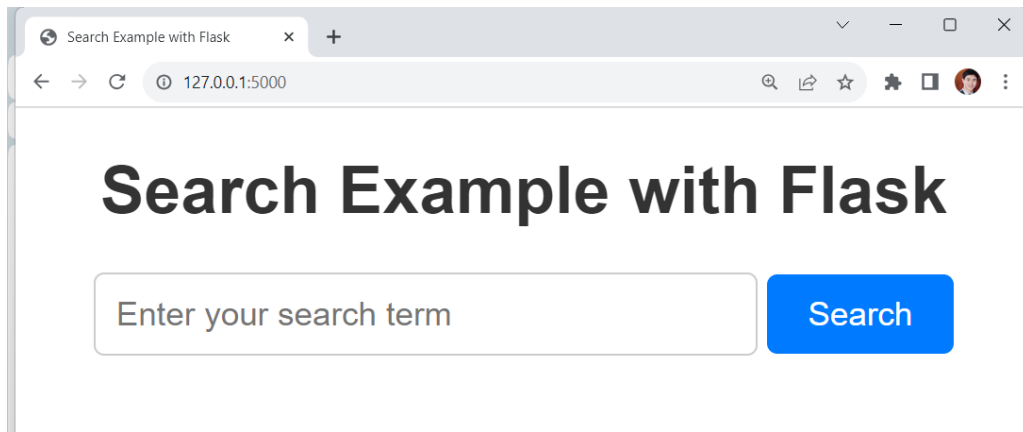
```
        </ul>
```

```
    </div>
```

```
    {% endif %}
```

```
</p>
```

3. SEARCH FORM



IV. WORKING WITH JOIN

- Using SQLite to join multiple table
- Apply the conditions to Flask web
- Do it by your self

The screenshot shows the DB Browser for SQLite interface. The title bar indicates the database is located at D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database\db\website.db. The main window is divided into several panes. The 'Execute SQL' pane on the left contains a SQL query:

```
1 Select * from Orders;
2
3 Select * from Storages inner join orders
4 on storages.id = orders.product_id
5 where rating>4
```

 The 'DB Schema' pane on the right lists the database's structure:

Name	Type	Schema
▼ Tables (4)		
orders	CREATE TABLE orders (user_email varchar(200),	
sqlite_sequence	CREATE TABLE sqlite_sequence(name,seq)	
storages	CREATE TABLE storages (id int primary key, prod	
user	CREATE TABLE user (id integer primary key autoi	
Indices (0)		
Views (0)		
Triggers (0)		

 At the bottom, a results table displays the output of the query:

	id	product	brand	rating	model	
1	7	laptop	Asus	4.5	X550LC-XX119H	https://i.gadget

3. SEARCH FORM

DB Browser for SQLite - D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database\db\website.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Open Project Save Project Attach Databases

Database Structure Browse Data Edit Pragma Execute SQL

SQL 1

```
1 Select * from storages where
2 model like '%DE%' or
3 brand like '%DE%' or
4 details like '%DE%'
5
```

	id	product	brand	rating	model	picture
1	id	product	brand	rating	model	picture
2	1	laptop	HP	4.1	Pavilion 15-ab205tx	https://i.gadgets360cdn.com/product...
3	2	laptop	HP	4.8	Pavilion 15-AU626TX	https://i.gadgets360cdn.com/product...
4	3	laptop	Asus	4.3	R558UQ-DM539D	https://i.gadgets360cdn.com/product...
5	4	laptop	HP	4.3	240 G4	https://i.gadgets360cdn.com/product...
6	5	laptop	Acer	4.4	Aspire E E1-572G	https://i.gadgets360cdn.com/product...
7	6	laptop	Dell	5	Inspiron 3000 3543	https://i.gadgets360cdn.com/product...
8	7	laptop	Asus	4.5	X550LC-XX119H	https://i.gadgets360cdn.com/product...
9	8	laptop	HP	3.5	Pavilion 15-AU020TX	https://i.gadgets360cdn.com/product...
10	9	laptop	Asus	4.5	G551VW-FI242T	https://i.gadgets360cdn.com/product...

Execution finished without errors.
Result: 527 rows returned in 128ms
At line 1:
Select * from storages where
model like '%DE%' or
brand like '%DE%' or
details like '%DE%'

```
# Khai bao bien de tro toi db
conn = sqlite3.connect(sqldbname)
cursor = conn.cursor()
sqlcommand = ("Select * from storages "
              "where model like '%" + search_text + "%'")
cursor.execute(sqlcommand)
data = cursor.fetchall()
```

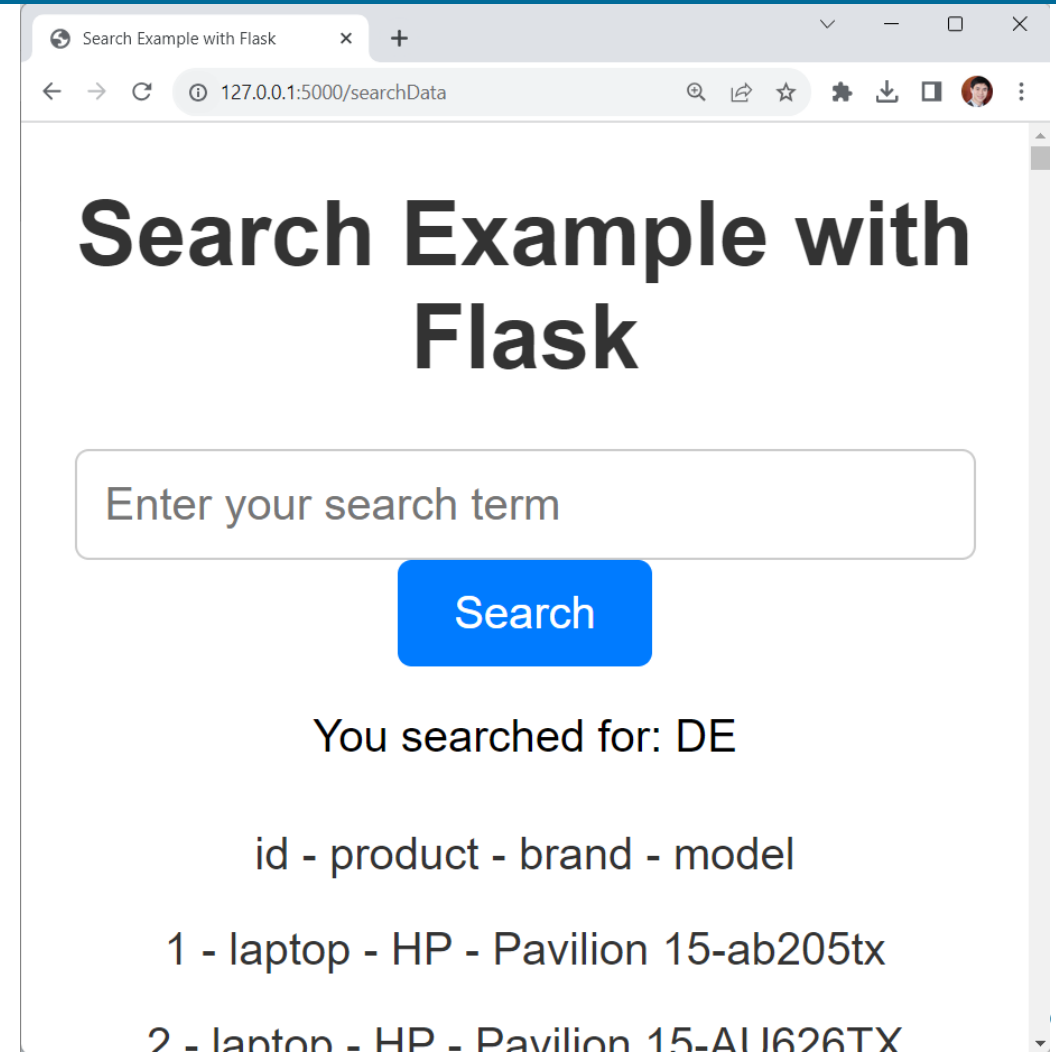
```
def load_data_from_db(search_text):
    sqldbname = 'db/website.db'
    if search_text != "":
        # Khai bao bien de tro toi db
        conn = sqlite3.connect(sqldbname)
        cursor = conn.cursor()

        sqlcommand = "Select * from storages where model like '%" + search_text + "%' "
        sqlcommand = sqlcommand + " or brand like '%" + search_text + "%' "
        sqlcommand = sqlcommand + " or details like '%" + search_text + "%' "

        cursor.execute(sqlcommand)
        data = cursor.fetchall()
        conn.close()
        return data
```

3. SEARCH FORM

```
def load_data_from_db(search_text):  
    sqldbname = 'db/website.db'  
    if search_text != "":  
        # Khai bao bien de tro toi db  
        conn = sqlite3.connect(sqldbname)  
        cursor = conn.cursor()  
  
        sqlcommand = "Select * from storages where model like '%" + search_text + "%'"  
        sqlcommand = sqlcommand + " or brand like '%" + search_text + "%'"  
        sqlcommand = sqlcommand + " or details like '%" + search_text + "%'"  
  
        cursor.execute(sqlcommand)  
        data = cursor.fetchall()  
        conn.close()  
        return data
```



Search Example with Flask

127.0.0.1:5000/searchData

Search Example with Flask

Enter your search term

Search

You searched for: DE

id - product - brand - model

1 - laptop - HP - Pavilion 15-ab205tx

2 - lanton - HP - Pavilion 15-AI I626TX

5. WORKING WITH SESSION – CHECK USER

The diagram illustrates a session-based login process across three browser windows:

- Top Left Window:** Address bar shows `127.0.0.1:5000`. The page content is "Welcome! [Login](#)". A blue arrow points from the [Login](#) link to the login form in the middle window.
- Middle Window:** Address bar shows `127.0.0.1:5000/login`. The page title is "Login". It contains a "Username:" label, a text input field containing "Felix Pham", and a blue "Login" button. A blue arrow points from the "Login" button to the bottom-left window.
- Bottom Left Window:** Address bar shows `127.0.0.1:5000`. The page content is "Hello, Felix Pham! [Logout](#)". A blue arrow points from the [Logout](#) link back to the login form in the middle window.

5. WORKING WITH SESSION – CHECK USER

The screenshot shows a web browser window with the URL `127.0.0.1:5000/login`. The page has a title "Login" and a form with a "Username:" label and a text input field containing "Felix Pham". Below the input field is a blue "Login" button.

Below the browser window is a screenshot of a database browser window titled "DB Browser for SQLite - D:\BMCNTT\Python LTWeb\Web 34.1 Load from Database Demo\db\website.db". The "Execute SQL" tab is active, showing a query: `Select * from user`. Below the query is a table with the following data:

	id	name	email	password
1	1	abc	abc@gmail.com	123456789
2	2	def	def@gmail.com	123456789

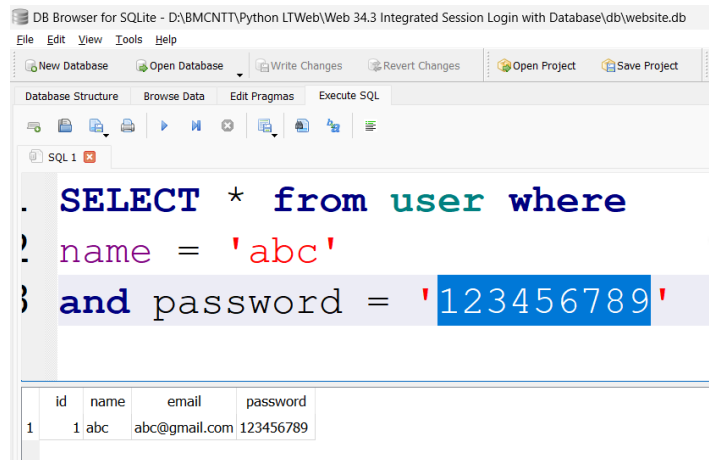
The "name" and "password" columns are highlighted with red boxes.

The screenshot shows a web browser window with the URL `127.0.0.1:5000/login`. The page has a title "Login" and a form with a "Username:" label and a text input field containing "abc". Below the input field is a "Password:" label and a text input field containing "123456789". Below the password field is a blue "Login" button.

Below the browser window is a screenshot of a database browser window titled "DB Browser for SQLite - D:\BMCNTT\Python LTWeb\Web 34.3 Integrated Session Login with Database\db\website.db". The "Execute SQL" tab is active, showing a query: `SELECT * from user where name = 'abc' and password = '123456789'`. Below the query is a table with the following data:

	id	name	email	password
1	1	abc	abc@gmail.com	123456789

5. WORKING WITH SESSION – CHECK USER



```
def check_exists(username, password):  
    result = False;  
    sqldbname = 'db/website.db'  
    # Khai bao bien de tro toi db  
    conn = sqlite3.connect(sqldbname)  
    cursor = conn.cursor()  
    # sqlcommand = "Select * from storages where "  
    sqlcommand = "Select * from user where name = '"+username+"' and password = '"+password+"'"  
    cursor.execute(sqlcommand)  
    data = cursor.fetchall()
```

5. WORKING WITH SESSION – CHECK USER

```
app.py x app_login_db.py x styles.css x login.html x
5 1 <!DOCTYPE html>
2 2 <html>
3 3 <head>
4 4 <title>Login</title>
5 5 <link rel="stylesheet" href="{{ url_for('static',filename='styles.css') }}">
6 6 </head>
7 7 <body>
8 8 <h1>Login</h1>
9 9 <form method="POST">
10 10 <label for="username">Username:</label>
11 11 <input type="text" id="username" name="username" required>
12 12 <br><br>
13 13 <label for="password">Password:</label>
14 14 <input type="text" id="password" name="password" required>
15 15 <br> <br> <br>
16 16 <button type="submit">Login</button>
17 17 </form>
18 18 </body>
19 19 </html>
```

5. WORKING WITH SESSION – CHECK USER

```
@app.route(rule: '/login', methods=['GET', 'POST'])
def login():
    # Khi nhận dữ liệu từ hành vi post, sau khi nhận
    # từ session sẽ gọi định tuyến sang trang index
    if request.method == 'POST':
        username = request.form['username']
        # Store 'username' in the session
        session['username'] = username
        return redirect(url_for('index'))
    # Trường hợp mặc định là vào trang login
    return render_template('login.html')
```

```
@app.route(rule: '/login', methods=['GET', 'POST'])
def login():
    # Khi nhận dữ liệu từ hành vi post, sau khi nhận dữ liệu
    # từ session sẽ gọi định tuyến sang trang index
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        # Store 'username' in the session
        if check_exists(username, password):
            session['username'] = username

        return redirect(url_for('index'))
    # Trường hợp mặc định là vào trang login
    return render_template('login.html')
```

5. WORKING WITH SESSION – CHECK USER

```
@app.route(rule: '/login', methods=['GET', 'POST'])
def login():
    # Khi nhận dữ liệu từ hành vi post, sau khi nhận dữ liệu
    # từ session sẽ gọi định tuyến sang trang index
    if request.method == 'POST':
        uname = request.form['username']
        pword = request.form['password']
```

```
if checkExists(uname, pword):
```

- Create function 'checkExists' >
- Create parameter 'checkExists' >
- Rename reference >
- Ignore an unresolved reference 'app.checkExists' >
- Mark all unresolved attributes of 'app' as ignored >
- Create new scratch file from selection >
- Invert 'if' condition >
- Adjust code style settings >

```
18 def checkExists(uname, pword):
19     pass
```

1 usage

```
def checkExists(uname, pword):
    pass
```

```
@app.route(rule: '/login', methods=['GET', 'POST'])
```

```
def login():
```

```
# Khi nhận dữ liệu từ hành vi post, sau khi nhận dữ liệu
# từ session sẽ gọi định tuyến sang trang index
```

```
if request.method == 'POST':
```

```
    uname = request.form['username']
```

```
    pword = request.form['password']
```

```
if checkExists(uname, pword):
```

```
# Store 'username' in the session
```

```
session['username'] = uname
```

```
return redirect(url_for('index'))
```

```
# Trường hợp mặc định là vào trang login
```

```
return render_template('login.html')
```

5. WORKING WITH SESSION – CHECK USER

```
app = Flask(__name__)  
app.secret_key = 'felix_pham'
```

```
@app.route('/')  
def index():
```

```
    # Check if 'username' key exists in the session
```

```
    if 'username' in session:
```

- Import this name >
- Import this name locally >
- Create parameter 'session' >
- Rename reference >
- Ignore an unresolved reference 'app.session' >
- Mark all unresolved attributes of 'app' as ignored >
- Create new scratch file from selection >
- Invert 'if' condition >
- Adjust code style settings >

Press Ctrl+Q to toggle preview

2 request, redirect, url_for, session)

a>')

/a>'

1 usage

```
def check_exists(username, password):
```

```
    result = False;
```

```
    sqldbname = 'db/website.db'
```

```
    # Khai bao bien de tro toi db
```

```
    conn = sqlite3.connect(sqldbname)
```

```
    cursor = conn.cursor()
```

```
    # sqlcommand = "Select * from storages where "
```

```
    sqlcommand = "Select * from user where name = '"+username+"' and password = '"+password+"'"
```

```
    cursor.execute(sqlcommand)
```

```
    data = cursor.fetchall()
```

```
    print(type(data))
```

```
    if len(data)>0:
```

```
        result = True
```

```
    conn.close()
```

```
    return result;
```

5. WORKING WITH SESSION – CHECK USER

The screenshot displays an IDE window titled "Web 34.3 Integrated Session Login with Database - app_login_db.py". The left sidebar shows a project structure with folders "db", "static", and "templates", and files "website.db", "styles.css", "login.html", "app.py", and "app_login_db.py". The main editor shows the following Python code:

```
38 conn = sqlite3.connect(sqldbname)    conn: <sqlite3.Connection object at 0x0000023C9C421120>
39 cursor = conn.cursor()    cursor: <sqlite3.Cursor object at 0x0000023C9C3E7180>
40 # sqlcommand = "Select * from storages where "
41
42 sqlcommand = "Select * from user where name = '"+username+"' and password = '"+password+"'"
43
44 cursor.execute(sqlcommand)
45 data = cursor.fetchall()
46 print(type(data))
```

The code is being debugged. The "Debugger" tab is active, showing a list of breakpoints on the left and a "Console" window on the right. The console displays the following output:

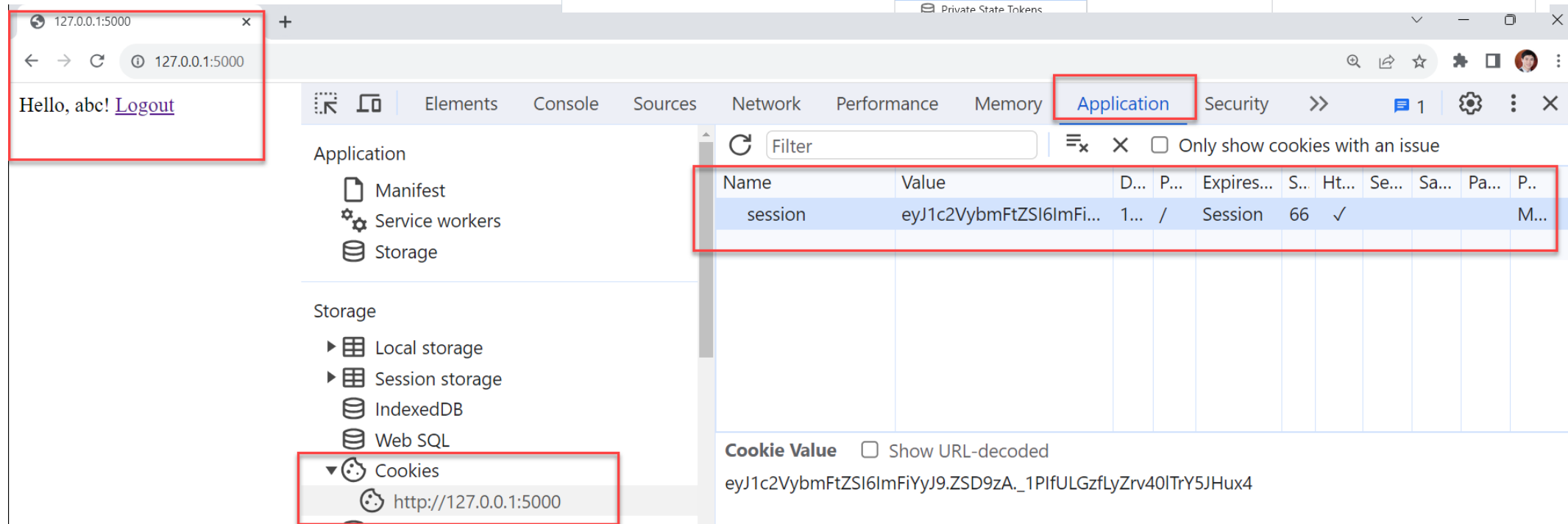
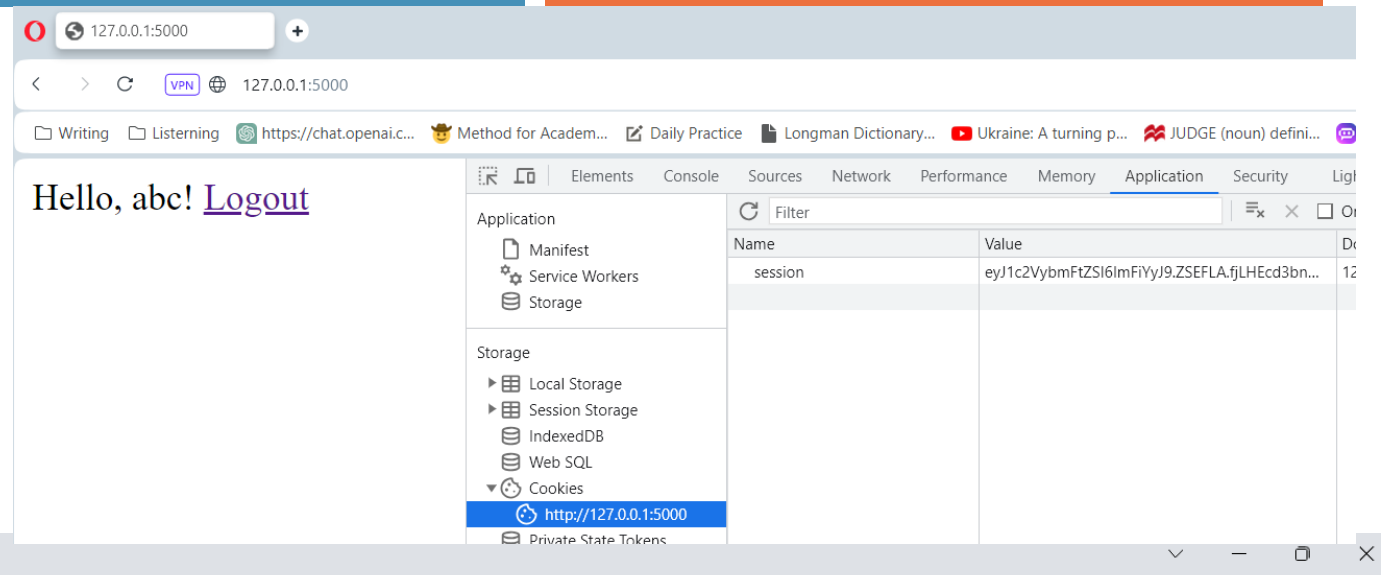
```
Evaluate expression (Enter) or add a watch (Ctrl+Shift+Enter)
> conn = {Connection} <sqlite3.Connection object at 0x0000023C9C421120>
> cursor = {Cursor} <sqlite3.Cursor object at 0x0000023C9C3E7180>
01 password = {str} '123123123123'
01 result = {bool} False
01 sqlcommand = {str} "Select * from user where name = 'abc' and password = '123123123123'"
01 sqldbname = {str} 'db/website.db'
01 username = {str} 'abc'
```

5. WORKING WITH SESSION – CHECK USER

```
css style_cart.css | H cart.html | SearchwithDataAddtoCartable_Update.p
import sqlite3

from flask import Flask, request, render_template, session

app: Flask = Flask(__name__, static_url_path='/static')
app.secret_key = "FelixPham"
```



5. WORKING WITH SESSION – CHECK USER

Web 34.3 Integrated Session Login with Database - app_login_db.py

Project

- Web 34.3 Integrated Session Login with Database [Web]
 - db
 - website.db
 - static
 - styles.css
 - templates
 - login.html
 - app.py
 - app_login_db.py
- External Libraries
- Scratches and Consoles

```
19
20 @app.route(rule: '/login', methods=['GET', 'POST'])
21 def login():
22     # Khi nhận dữ liệu từ hành vi post, sau khi nhận dữ liệu
23     # từ session sẽ gọi định tuyến sang trang index
24     if request.method == 'POST':
25         username = request.form['username']
26         password = request.form['password']
27         # Store 'username' in the session
28         if check_exists(username, password):
29             session['username'] = username
30
31         return redirect(url_for('index'))
32         # Trường hợp mặc định là vào trang login
33         return render_template('login.html')
34
35 1 usage
36 def check_exists(username, password):
37     result = False;
38     sqldbname = 'db/website.db'
39     # Khai báo biến để truy vấn db
40     conn = sqlite3.connect(sqldbname)
41     cursor = conn.cursor()
```



NATIONAL ECONOMICS UNIVERSITY

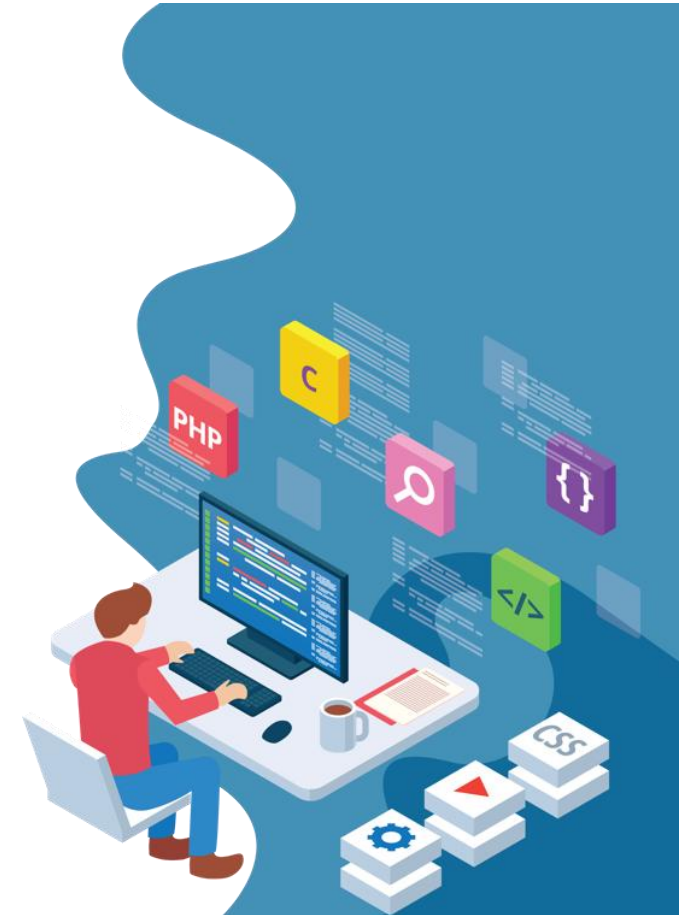
NEU COLLEGE OF TECHNOLOGY

FACULTY OF INFORMATION TECHNOLOGY

CHAPTER IV
WEB DEVELOPMENT WITH PYTHON
WORKING WITH WEB API

OUTLINE

- Create Web API
 - Select All Method
 - Select by ID Method
 - Insert Method
 - Update Method
 - Delete Method
- Working with PostMan
- Create App Client
- Create Web Client



CREATE WEB API

- Create new Project
- Import sqlite3
- Import flask

```
main.py x
1  # Import flask and sqlite3 modules
2  from flask import Flask, request, jsonify
3  import sqlite3
4
5  # Create a flask app
6  app = Flask(__name__)
7
8  # Connect to the database
9  sqldbname = 'db/website.db'
10
11 # Run the app
12 if __name__ == '__main__':
13     app.run(debug=True)
14
```

```
main.py x
1  # Import flask and sqlite3 modules
2  from flask import Flask, request, jsonify
3  import sqlite3
4
5
6
7
8
9
10
11
12 # Create a Customer table if it does not exist
```

Install package flask >

Rename reference >

Ignore an unresolved reference 'main.flask' >

Mark all unresolved attributes of 'main' as ignored >

Convert to 'import flask' >

Press Ctrl+Q to toggle preview

Packages installed successfully

Installed packages: 'flask'

WEB API – SELECT ALL METHOD

```
main.py
16
17 # Define a route for getting all users
18 @app.route(rule: '/users', methods=['GET'])
19 def get_users():
20     conn = sqlite3.connect(sqldbname)
21     cur = conn.cursor()
22     # Query the database for all users
23     cur.execute('SELECT * FROM user')
24     users = cur.fetchall()
25     # Convert the result to a list of dictionaries
26     users_list = []
27     for user in users:
28         users_list.append({'id': user[0], 'name': user[1],
29                             'email': user[2], 'password': user[3]})
30     # Return the list as a JSON response
31     return jsonify(users_list)
32
33 # Run the app
34 if __name__ == '__main__':
35     app.run(debug=True)
```

```
127.0.0.1:5000/users
127.0.0.1:5000/users

[
  {
    "email": "abc@gmail.com",
    "id": 1,
    "name": "abc",
    "password": "123456789"
  },
  {
    "email": "def@gmail.com",
    "id": 2,
    "name": "def",
    "password": "123456789"
  },
  {
    "email": "thaofami@gmail.com",
    "id": 3,
    "name": "abc",
    "password": "abc"
  },
  {
    "email": "thaofami@gmail.com",
    "id": 4,
    "name": "Felix Pham",
    "password": "abc123"
  }
]
```

WORKING WITH POST MAN

- Install Postman to test the app
- Install Postman

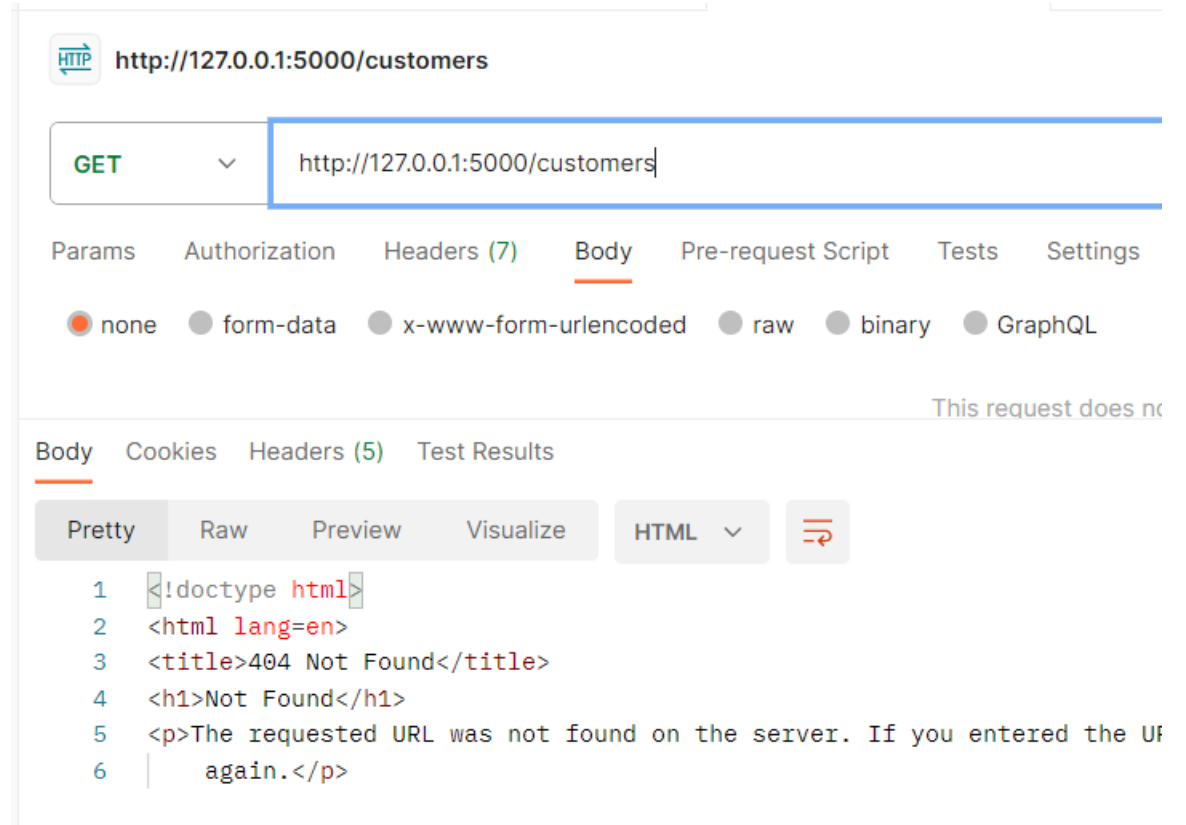
Install Postman to test the app

This tutorial uses Postman to test the API.

- Install [Postman](#)
- Start the web app.
- Start Postman.
- Select **Workspaces** > **Create Workspace** and then select **Next**.
- Name the workspace *TodoApi* and select **Create**.
- Select the settings gear icon > **Settings** (General tab) and disable SSL certificate verification.

⚠ Cảnh báo

Re-enable SSL certificate verification after testing the sample app.

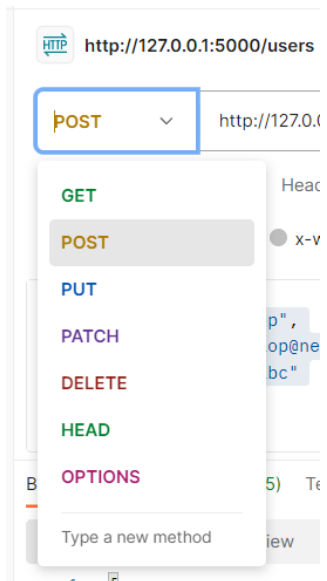
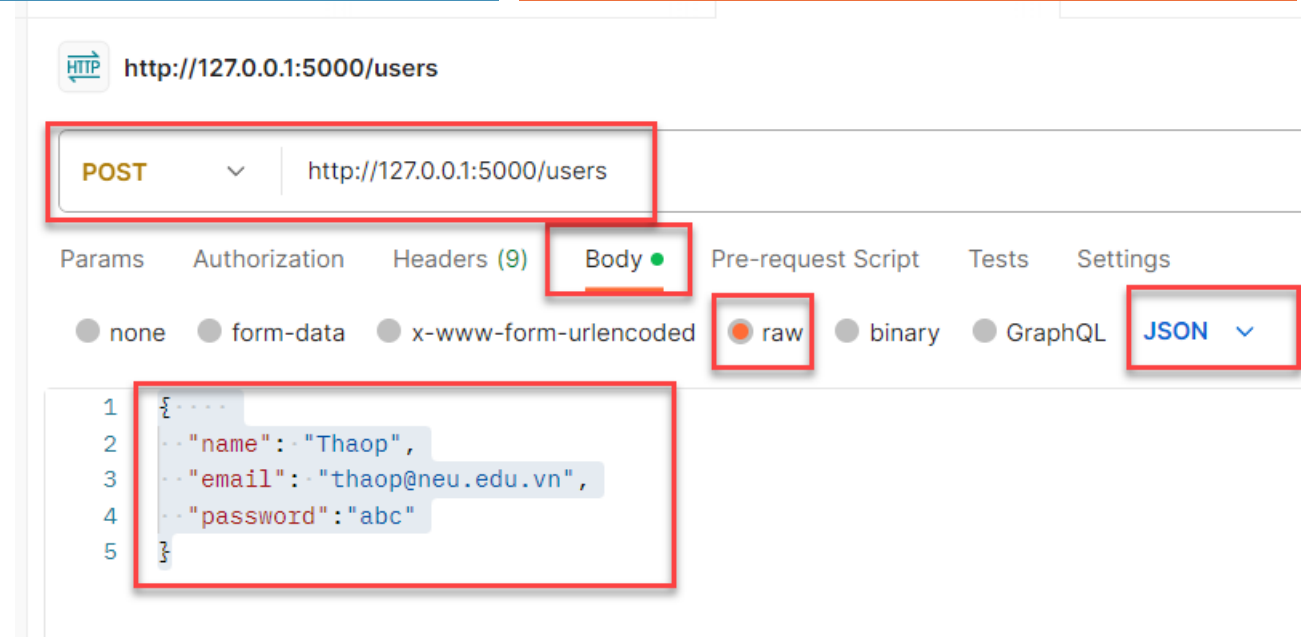


- <https://learning.postman.com/docs/sending-requests/requests/>

RUN POST MAN

Use the POST endpoint to add data to the app:

- In Postman, create a new HTTP request by selecting **New > HTTP**.
- Set the HTTP method to **POST**.
- Set the URI to `https://localhost:<port>/todoitems`. For example:
`https://localhost:5001/todoitems`
- Select the **Body** tab.
- Select **raw**.
- Set the type to **JSON**.
- In the request body enter JSON for a to-do item:



```
{  
  "name": "Thaop",  
  "email": "thaop@neu.edu.vn",  
  "password": "abc"  
}
```

WEB API – SELECT ALL METHOD

```
# Define a route for getting all users
```

```
@app.route(rule: '/users', methods=['GET'])
```

```
def get_users():
```

```
    conn = sqlite3.connect(sqldbname)
```

```
    cur = conn.cursor()
```

```
    # Query the database for all users
```

```
    cur.execute('SELECT * FROM user')
```

```
    users = cur.fetchall()
```

```
    # Convert the result to a list of dictionaries
```

```
    users_list = []
```

```
    for user in users:
```

```
        users_list.append({'id': user[0], 'name': user[1],  
                           'email': user[2], 'password': user[3]})
```

```
    # Return the list as a JSON response
```

```
    return jsonify(users_list)
```

```
# Run the app
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True)
```

HTTP http://127.0.0.1:5000/Users

GET

http://127.0.0.1:5000/Users

Params

Authorization

Headers (9)

Body

Pre-request Script

Query Params

	Key	Value
	Key	Value

Body

Cookies

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

HTML

⌵

```
1 <!doctype html>  
2 <html lang=en>  
3 <title>404 Not Found</title>  
4 <h1>Not Found</h1>  
5 <p>The requested URL was not found on the server. If  
6   again.</p>
```

WEB API – SELECT ALL METHOD

Define a route for getting all users

```
@app.route(rule: '/users', methods=['GET'])
```

```
def get_users():
```

```
    conn = sqlite3.connect(sqldbname)
```

```
    cur = conn.cursor()
```

Query the database for all users

```
    cur.execute('SELECT * FROM user')
```

```
    users = cur.fetchall()
```

Convert the result to a list of dictionaries

```
    users_list = []
```

```
    for user in users:
```

```
        users_list.append({'id': user[0], 'name': user[1],
```

```
                           'email': user[2], 'password': user[3]})
```

Return the list as a JSON response

```
    return jsonify(users_list)
```

The screenshot shows a web API client interface with the following elements:

- Overview Tab:** Displays the request URL `http://127.0.0.1:5000/users` and the method `GET`.
- Params Tab:** Shows no parameters.
- Headers Tab:** Shows no headers.
- Body Tab:** Shows the response body in JSON format, which is a list of user objects:

```
[{"email": "abc@gmail.com", "id": 1, "name": "abc", "password": "123456789"}, {"email": "def@gmail.com", "id": 2, "name": "def", "password": "123456789"}, {"email": "thaofami@gmail.com", "id": 3, "name": "abc", "password": "abc"}, {"email": "thaofami@gmail.com", "id": 4, "name": "Felix Pham"}]
```
- Status Bar:** Shows the response status `200 OK`, time `6 ms`, and size `573 B`.

CREATE WEB API

```
main.py x
32
33 # Define a route for getting a user by id
34 @app.route(rule: '/users/<int:id>', methods=['GET'])
35 def get_user(id):
36     conn = sqlite3.connect(sqldbname)
37     cur = conn.cursor()
38     # Query the database for the user with the given id
39     cur.execute(_sql: 'SELECT * FROM user WHERE id = ?', _parameters: (id,))
40     user = cur.fetchone()
41     # Check if the user exists
42     if user:
43         # Convert the result to a dictionary
44         user_dict = {'id': user[0], 'name': user[1],
45                     'email': user[2], 'password': user[3]}
46         # Return the dictionary as a JSON response
47         return jsonify(user_dict)
48     else:
49         # Return a 404 error if the user does not exist
50         return 'User not found', 404
```

```
127.0.0.1:5000/users/1 x +
127.0.0.1:5000/users/1
{
  "email": "abc@gmail.com",
  "id": 1,
  "name": "abc",
  "password": "123456789"
}
```

WEB API – SELECT BY ID METHOD

```
# Define a route for getting a user by id
@app.route(rule: '/users/<int:id>', methods=['GET'])
def get_user(id):
    conn = sqlite3.connect(sqldbname)
    cur = conn.cursor()
    # Query the database for the user with the given id
    cur.execute(_sql: 'SELECT * FROM user WHERE id = ?', _parameters: (id,))
    user = cur.fetchone()
    # Check if the user exists
    if user:
        # Convert the result to a dictionary
        user_dict = {'id': user[0], 'name': user[1],
                    'email': user[2], 'password': user[3]}
        # Return the dictionary as a JSON response
        return jsonify(user_dict)
    else:
        # Return a 404 error if the user does not exist
        return 'User not found', 404
```

The screenshot shows a web browser interface for testing HTTP requests. The address bar displays the URL `http://127.0.0.1:5000/users/1`. The method dropdown is set to `GET`. The response status is `200 OK` with a response time of `8 ms` and a body size of `251 B`. The response body is displayed in JSON format, showing the user details: `{ "email": "abc@gmail.com", "id": 1, "name": "abc", "password": "123456789" }`.

WEB API – INSERT METHOD

```
main.py x
51
52 # Define a route for adding a new user
53 @app.route(rule: '/users', methods=['POST'])
54 def add_user():
55     conn = sqlite3.connect(sqldbname)
56     cur = conn.cursor()
57     # Get the user name, email and password from the request body
58     user_name = request.json.get('name')
59     user_email = request.json.get('email')
60     user_password = request.json.get('password')
61     # Check if the user name, email and password are valid
62     if user_name and user_email and user_password:
63         # Insert the user into the database
64         cur.execute(_sql: 'INSERT INTO user (name, email, password) '
65                     'VALUES (?, ?, ?)', _parameters: (user_name, user_email, user_password))
66         conn.commit()
67         # Get the id of the inserted user
68         user_id = cur.lastrowid
69         # Return the id as a JSON response
70         return jsonify({'id': user_id})
71     else:
72         # Return a 400 error if the user name, email or password are invalid
73         return 'User name, email and password are required', 400
```

WEB API – INSERT METHOD

HTTP <http://127.0.0.1:5000/users>

POST <http://127.0.0.1:5000/users>

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```
1 {
2   "name": "Thaop",
3   "email": "thaop@neu.edu.vn",
4   "password": "abc"
5 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON**

```
1 {
2   "id": 5
3 }
```

```
main.py x
52 # Define a route for adding a new user
53 @app.route(rule: '/users', methods=['POST'])
54 def add_user():
55     conn = sqlite3.connect(sqldbname)
56     cur = conn.cursor()
57     # Get the user name, email and password from the request body
58     user_name = request.json.get('name')
59     user_email = request.json.get('email')
60     user_password = request.json.get('password')
61     # Check if the user name, email and password are valid
62     if user_name and user_email and user_password:
63         # Insert the user into the database
64         cur.execute(_sql: 'INSERT INTO user (name, email, password) '
65                     'VALUES (?, ?, ?)', _parameters: (user_name, user_email, user_password))
66         conn.commit()
67         # Get the id of the inserted user
68         user_id = cur.lastrowid
69         # Return the id as a JSON response
70         return jsonify({'id': user_id})
71     else:
72         # Return a 400 error if the user name, email or password are invalid
73         return 'User name, email and password are required', 400
```

WEB API – UPDATE METHOD

main.py

```
74 # Define a route for updating a user by id
75 @app.route(rule: '/users/<int:id>', methods=['PUT'])
76 def update_user(id):
77     conn = sqlite3.connect(sqldbname)
78     cur = conn.cursor()
79     # Get the user name, email and password from the request body
80     user_name = request.json.get('name')
81     user_email = request.json.get('email')
82     user_password = request.json.get('password')
83     # Check if the user name, email and password are valid
84     if user_name and user_email and user_password:
85         # Update the user in the database
86         cur.execute(_sql: 'UPDATE user SET name = ?, email = ?, '
87                     'password = ? WHERE id = ?', _parameters: (user_name, user_email, user_password, id))
88         conn.commit()
89         if cur.rowcount > 0: # Check if the user exists
90             # Return a success message as a JSON response
91             return jsonify({'message': 'User updated successfully'})
92         else:
93             return 'User not found', 404 # Return a 404 error if the user does not exist
94     else:
95         # Return a 400 error if the user name, email or password are invalid
96         return 'User name, email and password are required', 400
```

WEB API – UPDATE METHOD

HTTP <http://127.0.0.1:5000/users/2>

PUT <http://127.0.0.1:5000/users/2>

Params Authorization Headers (9) **Body** Pre-request Script

● none ● form-data ● x-www-form-urlencoded ● **raw** ● binary

```
1 {
2   "name": "ThaoPHAM",
3   "email": "thaop@neu.edu.vn",
4   "password": "thaoPham"
5 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON**

```
1 {
2   "message": "User updated successfully"
3 }
```

```
main.py x
74 # Define a route for updating a user by id
75 @app.route(rule='/users/<int:id>', methods=['PUT'])
76 def update_user(id):
77     conn = sqlite3.connect(sqldbname)
78     cur = conn.cursor()
79     # Get the user name, email and password from the request body
80     user_name = request.json.get('name')
81     user_email = request.json.get('email')
82     user_password = request.json.get('password')
83     # Check if the user name, email and password are valid
84     if user_name and user_email and user_password:
85         # Update the user in the database
86         cur.execute(_sql: 'UPDATE user SET name = ?, email = ?, '
87                     'password = ? WHERE id = ?', _parameters: (user_name, user_email, user_password, id))
88         conn.commit()
89         if cur.rowcount > 0: # Check if the user exists
90             # Return a success message as a JSON response
91             return jsonify({'message': 'User updated successfully'})
92         else:
93             return 'User not found', 404 # Return a 404 error if the user does not exist
94     else:
95         # Return a 400 error if the user name, email or password are invalid
96         return 'User name, email and password are required', 400
```

WEB API – DELETE METHOD

```
# Define a route for deleting a user by id
@app.route(rule: '/users/<int:id>', methods=['DELETE'])
def delete_user(id):
    conn = sqlite3.connect(sqldbname)
    cur = conn.cursor()
    # Delete the user from the database
    cur.execute(_sql: 'DELETE FROM user WHERE id = ?', _parameters: (id,))
    conn.commit()
    # Check if the user exists
    if cur.rowcount > 0:
        # Return a success message as a JSON response
        return jsonify({'message': 'User deleted successfully'})
    else:
        # Return a 404 error if the user does not exist
        return 'User not found', 404
```

HTTP http://127.0.0.1:5000/users/1

DELETE

http://127.0.0.1:5000/users/1

Params Authorization Headers (7) Body Pre-request Script Tests Set



none



form-data



x-www-form-urlencoded



raw



binary



GraphQL

This request

Body Cookies Headers (5) Test Results

Pretty

Raw

Preview

Visualize

JSON



1

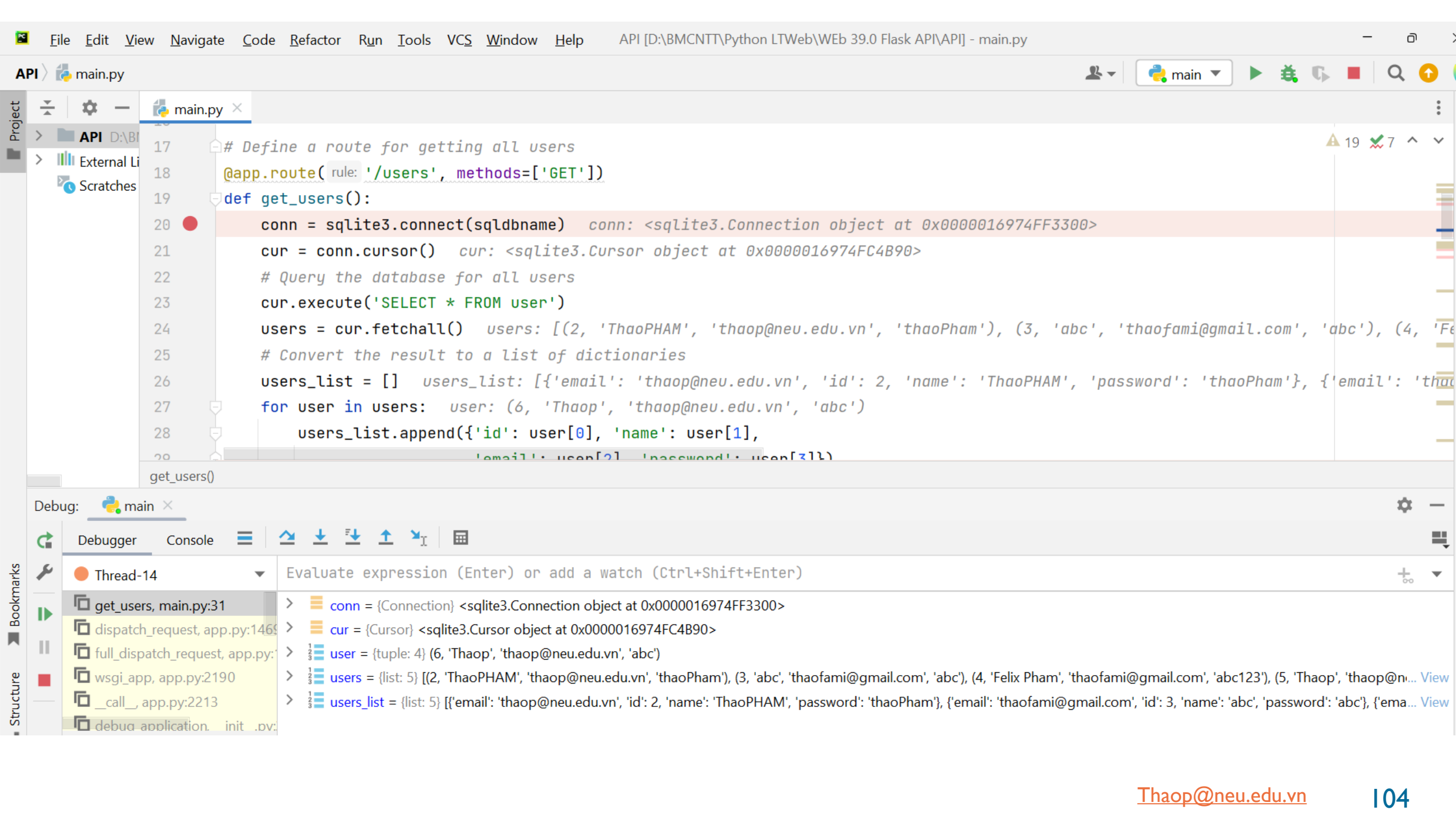
2

3

"message": "User deleted successfully"



DEBUG



HTTP http://127.0.0.1:5000/users/2

GET

http://127.0.0.1:5000/users/2

Params Authorization Headers (7) Body Pre-request Script Tests

Query Params

Key	Value
Key	Value

HTTP http://127.0.0.1:5000/users/2

GET

http://127.0.0.1:5000/users/2

Params Authorization Headers (7) Body Pre-request Script

Query Params

Key	Value
-----	-------

Body Cookies Headers (5) Test Results

Pretty

Raw

Preview

Visualize

JSON



```
1 {
2   "email": "thaop@neu.edu.vn",
3   "id": 2,
4   "name": "ThaoPHAM",
5   "password": "thaoPham"
6 }
```

Save

```
36 # Define a route for getting a user by id
37 @app.route(rule: '/users/<int:id>', methods=['GET'])
38 def get_user(id): id: 2
39     conn = sqlite3.connect(sqldbname) conn: <sqlite3.Connection object at 0x0
40     cur = conn.cursor() cur: <sqlite3.Cursor object at 0x0000016974FC4E30>
41     # Query the database for the user with the given id
42     cur.execute(_sql: 'SELECT * FROM user WHERE id = ?', _parameters: (id,))
43     user = cur.fetchone() user: (2, 'ThaoPHAM', 'thaop@neu.edu.vn', 'thaoPham
44     # Check if the user exists
45     if user:
46         # user = {tuple: 4} (2, 'ThaoPHAM', 'thaop@neu.edu.vn', 'thaoPham')
47         u
48         Array {str} 'ThaoPHAM'
49         {str} 'thaop@neu.edu.vn'
50         01 3 = {str} 'thaoPham'
51         01 _len_ = {int} 4
52     else:
53         > Protected Attributes
```

get_user() > if user

in x

Set value F2



CLIENT WEB

CREATE NEW PROJECT

■ Create project

```
main.py x
1 # Import flask, requests and json modules
2 from flask import Flask, render_template, request, redirect, flash
3 import requests
4 import json
```

```
main.py x
1 # Import flask, requests and json modules
2 from flask import Flask, render_template, request, redirect, flash
3 import requests
4 import json
5 # Create a flask app
6 app = Flask(__name__)
7 # Set the secret key for flash messages
8 app.secret_key = 'FelixPham'
9 # Set the base url for the web api
10 base_url = 'http://127.0.0.1:5000/users'
11
12 # Run the app
13 if __name__ == '__main__':
14     app.run(debug=True, port=5001)
15
```

```
main.py x
1 # Import flask, requests and json modules
2 from flask import Flask, render_template, request, redirect, flash
3 import requests
4
5
6
7
8
```

Install package requests >

Rename reference >

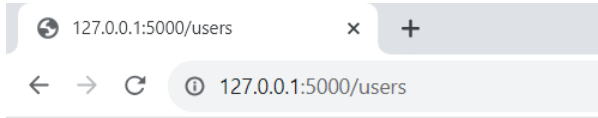
Ignore an unresolved reference 'requests' >

Press Ctrl+Q to toggle preview

Packages installed successfully

Installed packages: 'flask'

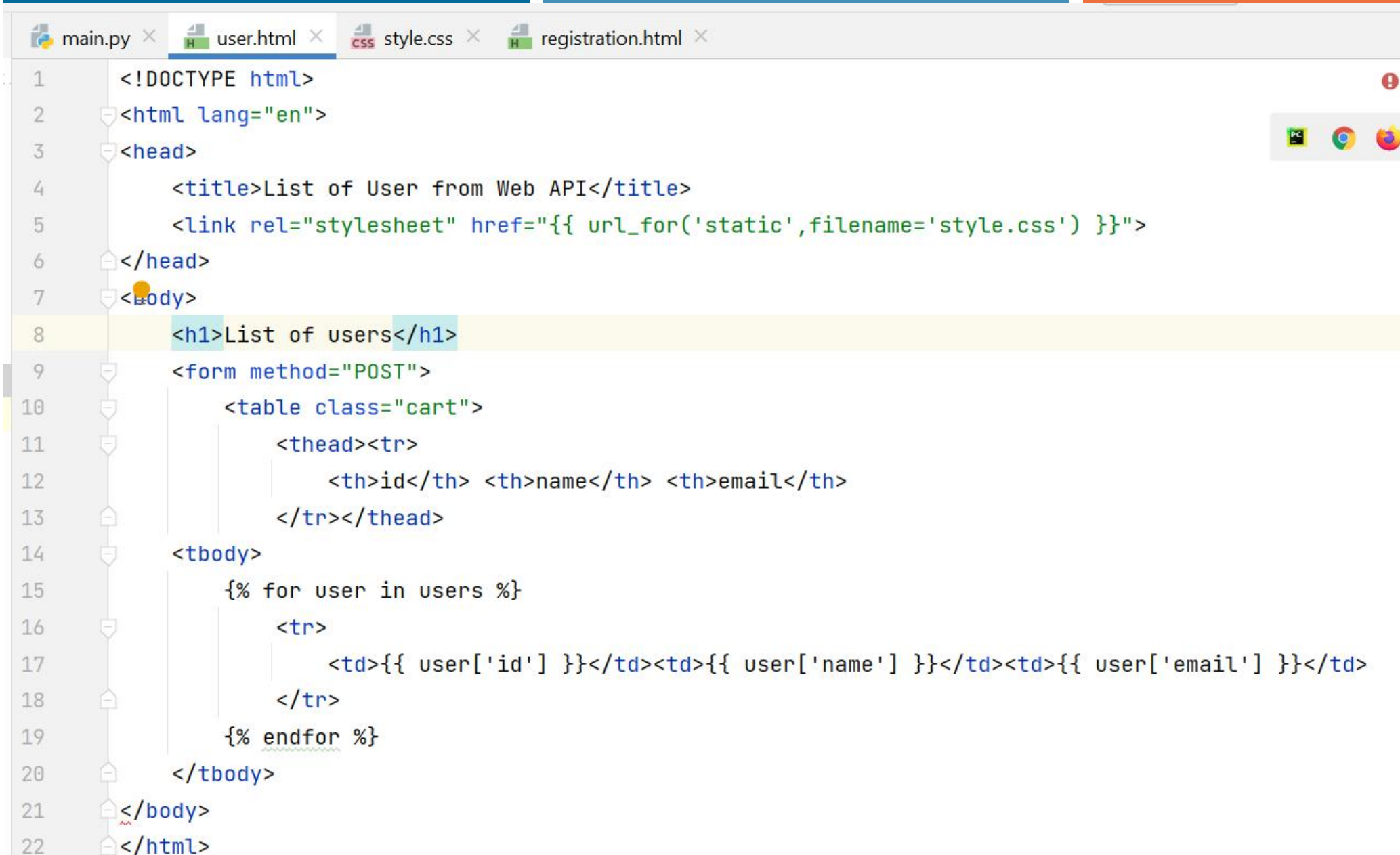
WEB CLIENT API – INDEX METHOD - PYTHON



```
[
{
  "email": "thaop@neu.edu.vn",
  "id": 2,
  "name": "ThaoPHAM",
  "password": "thaoPham"
},
{
  "email": "thaofami@gmail.com",
  "id": 3,
  "name": "abc",
  "password": "abc"
},
{
  "email": "thaofami@gmail.com",
  "id": 4,
  "name": "Felix Pham",
  "password": "abc123"
},
{
  "email": "thaop@neu.edu.vn",
  "id": 5,
  "name": "Thaop",
  "password": "abc"
},
{
  "email": "thaop@neu.edu.vn",
  "id": 6,
  "name": "Thaop",
  "password": "abc"
}
]
```

```
# Set the base url for the web api
base_url = 'http://127.0.0.1:5000/users'
# Define a route for the home page
@app.route('/')
def index():
    # Get all users from the web api
    response = requests.get(base_url)
    # Check if the response is successful
    if response.status_code == 200:
        # Parse the response as a JSON object
        users = response.json()
        # Render the index.html template with the users data
        return render_template(template_name_or_list='user.html', users=users)
    else:
        # Display an error message
        flash('Something went wrong. Please try again later.')
        return render_template('user.html')
```

WEB CLIENT API – INDEX METHOD - HTML



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <title>List of User from Web API</title>
5     <link rel="stylesheet" href="{{ url_for('static',filename='style.css') }}">
6 </head>
7 <body>
8     <h1>List of users</h1>
9     <form method="POST">
10         <table class="cart">
11             <thead><tr>
12                 <th>id</th> <th>name</th> <th>email</th>
13             </tr></thead>
14             <tbody>
15                 {% for user in users %}
16                 <tr>
17                     <td>{{ user['id'] }}</td><td>{{ user['name'] }}</td><td>{{ user['email'] }}</td>
18                 </tr>
19                 {% endfor %}
20             </tbody>
21         </form>
22 </body>
</html>
```

WEB CLIENT API – INDEX METHOD - CSS

```
/* styles.css */
.shopping-cart-icon {
    width: 30px; /* Set the width of the icon */
    height: 30px; /* Set the height of the icon */
}

.cart {
    width: 100%;
    border-collapse: collapse;
    margin-bottom: 20px;
}

.cart th, .cart td {
    border: 1px solid #ddd;
    padding: 10px;
    text-align: left;
}

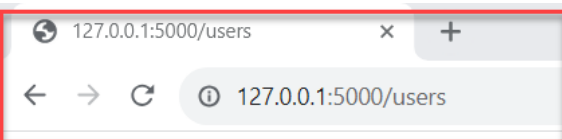
.cart th {
    background-color: #f2f2f2;
}
```

```
.cart tr:nth-child(even) {
    background-color: #f2f2f2;
}

.cart button {
    padding: 10px 20px;
    background-color: #007bff;
    color: #fff;
    border: none;
    border-radius: 5px;
    font-size: 16px;
    cursor: pointer;
    width: 150px;
}

.cart button:hover {
    background-color: #0056b3;
}
```

WEB CLIENT API – INDEX METHOD



```
[
  {
    "email": "thaop@neu.edu.vn",
    "id": 2,
    "name": "ThaoPHAM",
    "password": "thaoPham"
  },
  {
    "email": "thaofami@gmail.com",
    "id": 3,
    "name": "abc",
    "password": "abc"
  },
  {
    "email": "thaofami@gmail.com",
    "id": 4,
    "name": "Felix Pham",
    "password": "abc123"
  },
  {
    "email": "thaop@neu.edu.vn",
    "id": 5,
    "name": "Thaop",
    "password": "abc"
  },
  {
    "email": "thaop@neu.edu.vn",
    "id": 6,
    "name": "Thaop",
    "password": "abc"
  }
]
```

A screenshot of a web browser displaying a page titled "List of User from Web API". The browser's address bar shows the URL "127.0.0.1:5001". The page content includes a heading "List of users" and a table with three columns: "id", "name", and "email". The table contains five rows of user data. The browser window is highlighted with a red rectangle.

List of users

id	name	email
2	ThaoPHAM	thaop@neu.edu.vn
3	abc	thaofami@gmail.com
4	Felix Pham	thaofami@gmail.com
5	Thaop	thaop@neu.edu.vn
6	Thaop	thaop@neu.edu.vn

EXERCISE

- User
 - Add Method for Manipulate new User
 - Add Method for Add new User
 - Add Method for Edit new User
 - Add Method for Delete new User
- Orders

WEB CLIENT API – ADD METHOD

```
main.py x edit.html x index.html x main_v2.py x add.html x user.html x style.css x registration.hti
28 # Define a route for the add user page
29 @app.route(rule: '/add', methods=['GET', 'POST'])
30 def add():
31     # Check if the request method is POST
32     if request.method == 'POST':
33         # Get the user name, email and password from the form
34         user_name = request.form.get('name')
35         user_email = request.form.get('email')
36         user_password = request.form.get('password')
37         # Check if the user name, email and password are valid
38         if user_name and user_email and user_password:
39             # Send a POST request to the web api with the user name, email and password as JSO
40             response = requests.post(base_url,
41                                     json={'name': user_name, 'email': user_email, 'password': user_password})
42             # Check if the response is successful
43             if response.status_code == 200:
44                 # Parse the response as a JSON object
45                 user = response.json()
46                 # Display a success message
47                 flash(f'User {user["id"]} added successfully.')
48                 # Redirect to the home page
49                 return redirect('/')
50             else:
else:
    # Display an error message
    flash('Something went wrong. Please try again later.')
    return render_template('users/add.html')
else:
    # Display an error message
    flash('User name, email and password are required.')
    return render_template('add.html')
else:
    # Render the add.html template
    return render_template('users/add.html')
```

WEB CLIENT API – EDIT METHOD

```
main.py × edit.html × index.html × main_v2.py × add.html × user.html × style.css × registration.html ×
61
62 # Define a route for the edit user page
63 @app.route(rule: '/edit/<int:id>', methods=['GET', 'POST'])
64 def edit(id):
65     # Check if the request method is POST
66     if request.method == 'POST':
67         # Get the user name, email and password from the form
68         user_name = request.form.get('name')
69         user_email = request.form.get('email')
70         user_password = request.form.get('password')
71         # Check if the user name, email and password are valid
72         if user_name and user_email and user_password:
73             # Send a PUT request to the web api with the user name, email and password as JSON data
74             response = requests.put(url: f'{base_url}/{id}',
75                                     json={'name': user_name, 'email': user_email, 'password': user_password})
76             # Check if the response is successful
77             if response.status_code == 200:
78                 # Display a success message
79                 flash(f'User {id} updated successfully.')
80                 # Redirect to the home page
81                 return redirect('/')
82             else:
83                 # Display an error message
```

⚠ 8 ⚠ 10

WEB CLIENT API – EDIT METHOD

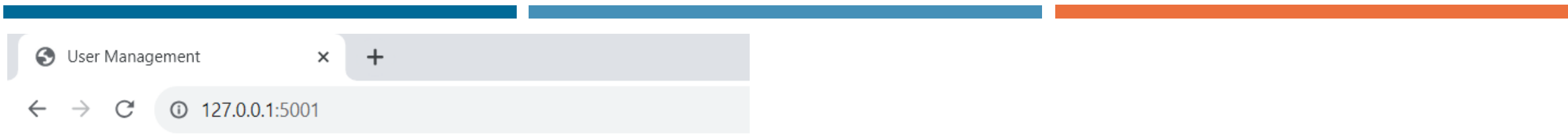
```
main.py x edit.html x index.html x main_v2.py x add.html x user.html x style.css x registration.html x
61
62 # Define a route for the edit user page
63 @app.route(rule: '/edit<int:id>', methods=['GET', 'POST'])
64 def edit(id):
65     # Check if the request method is POST
66     if request.method == 'POST':
67         # Get the user name, email and password from the form
68         user_name = request.form.get('name')
69         user_email = request.form.get('email')
70         user_password = request.form.get('password')
71         # Check if the user name, email and password are valid
72         if user_name and user_email and user_password:
73             # Send a PUT request to the web api with the user name, email and password as JSON data
74             response = requests.put(url: f'{base_url}/{id}',
75                                     json={'name': user_name, 'email': user_email, 'password': user_password})
76             # Check if the response is successful
77             if response.status_code == 200:
78                 # Display a success message
79                 flash(f'User {id} updated successfully.')
80                 # Redirect to the home page
81                 return redirect('/')
82             else:
83                 # Display an error message
```

```
# Redirect to the home page
return redirect('/')
else:
    # Display an error message
    flash('Something went wrong. Please try again later.')
    return render_template('edit.html')
else:
    # Display an error message
    flash('User name, email and password are required.')
    return render_template('edit.html')
else:
    # Send a GET request to the web api to get the user by id
    response = requests.get(f'{base_url}/{id}')
    # Check if the response is successful
    if response.status_code == 200:
        # Parse the response as a JSON object
        user = response.json()
        # Render the edit.html template with the user data
        return render_template(template_name_or_list: 'edit.html', user=user)
    else:
        # Display an error message
        flash('User not found.')
        return render_template('edit.html')
```

WEB CLIENT API – DELETE METHOD

```
main.py x edit.html x index.html x main_v2.py x add.html x user.html x CSS
103
104 # Define a route for the delete user page
105 @app.route(rule: '/delete/<int:id>', methods=['GET', 'POST'])
106 def delete(id):
107     # Check if the request method is POST
108     if request.method == 'POST':
109         # Send a DELETE request to the web api to delete the user by id
110         response = requests.delete(f'{base_url}/{id}')
111         # Check if the response is successful
112         if response.status_code == 200:
113             # Display a success message
114             flash(f'User {id} deleted successfully.')
115             # Redirect to the home page
116             return redirect('/')
117         else:
118             # Display an error message
119             flash('Something went wrong. Please try again later.')
120             return render_template('delete.html')
121     else:
122         # Send a GET request to the web api to get the user by id
123         response = requests.get(f'{base_url}/{id}')
124         # Check if the response is successful
125         if response.status_code == 200:
```

```
        if response.status_code == 200:
            # Parse the response as a JSON object
            user = response.json()
            # Render the delete.html template with the user
            return render_template(
                template_name_or_list: 'delete.html', user=user)
        else:
            # Display an error message
            flash('User not found.')
            return render_template('delete.html')
```



User Management

Add User

Name:

Email:

Password:

Users

ID	Name	Email	Password	Actions
2	ThaoPHAM	thaop@neu.edu.vn	thaoPham	Edit Delete
3	abc	thaofami@gmail.com	abc	Edit Delete
4	Felix Pham	thaofami@gmail.com	abc123	Edit Delete
5	Thaop	thaop@neu.edu.vn	abc	Edit Delete
6	Thaop	thaop@neu.edu.vn	abc	Edit Delete

EXERCISE

■ User

- Add Method for Manipulate new User
- Add Method for Add new User
- Add Method for Edit new User
- Add Method for Delete new User
- Check Exists User from Database

■ Orders

- Return information for an Order by OrderID
- Return a list of Orders from one date to another date

■ Products

- Get the full list of products
- Get the list of products that match the search criteria
- Get the list of products with their corresponding quantity sold and total sales amount